

From Sparse Selection to Physical Savings: A Unified Runtime and Measurement Contract for Progressive Retrieval Attention

PRA Research Program

September 2026

Abstract

We ask how Progressive Retrieval Attention (PRA) can become a portable runtime abstraction without confusing a small logical selection with a systems-level saving. We organize deployment into three depths: Selected Context serializes chosen evidence as visible text (E0); Native Memory supplies selected model-native key/value state outside the visible prompt (E2); and Native Serving gives the engine scheduler direct ownership of placement and reuse (E3). The implementation unifies authenticated resources, typed records, tasks, sessions, routing, physical materialization, storage lifecycle, engine capability negotiation, and evidence receipts behind one runtime and gateway. The contract keeps ordinary prefix K/V distinct from query-addressed PRA state and falls back to E0 when a deeper capability is unavailable.

The evaluation separates quality, context, and economics. On frozen routed evidence, MLX-LM, SGLang-MLX, and vLLM-Metal remove 90.6–93.0% of visible prompt tokens and preserve 6,258/6,258 paired outputs across 149 questions per engine. Lossless WARM storage preserves 265/265 outputs over 17 model–dataset cohorts and survives restart, whereas int8 COLD storage is exact in only 61/265 sequences and remains opt-in. Qwen3-32B reaches a warm E2/E0 cost ratio of 1.011 and a cold usable-context ratio of 0.976 on Apple Silicon. A scheduler-invisible CUDA vLLM prototype preserves 128/128 outputs in each of FULL, E0, E2-HOT, and E2-WARM, but E0 sustains 56.3 requests/s versus 45.5 and 42.4 for the two native modes. Thus semantic parity and scheduler invisibility do not yet imply an economic win. On a 48 GiB M4 Pro, a Qwen3.5-27B MLX 4-bit checkpoint runs at 31.1 decode tokens/s and fits in 15.4 GiB at smoke scale. At a 20% selected-context budget, it removes 77–79% of visible prompt tokens and reduces mean TTFT by 76–79% on 32 held-out QASPER and HotpotQA questions. Its learned router helps QASPER but hurts HotpotQA, and detached native K/V is unavailable because 48 of 64 layers require DeltaNet recurrent state beyond K/V.

Using MultiHop-RAG only as a locked engine workload, MLX-LM contiguous Native Memory preserves every selector-frozen output. Retained K/V reduces warm total latency from 3.031 to 0.638 s at 2K and from 6.121 to 0.799 s at 4K, while cold native execution remains 1.8–3.3% slower across Qwen3-4B/8B. Independently cached chunks reduce runtime but change outputs, so they remain unqualified. Papers 3.2 and 3.3 own the corresponding retrieval and selector conclusions.

1 Introduction

PRA changes what a model must keep logically available. A long-lived resource may remain addressable through a compact routing representation while only a small selected part of its native K/V participates in one attention operation. That distinction is algorithmically useful, but it leaves a systems question unanswered. A selected set can be small while the runtime still stores every detail tensor in HBM, launches many tiny gathers, copies the same bytes repeatedly, constructs temporary concatenations, or serializes requests around Python control flow. In that case sparsity exists in a trace but not in the resource bill.

This paper asks how to turn PRA into a portable inference primitive before a serving engine becomes semantically aware of PRA memory. The immediate task is not to design a new scheduler. It is to define an execution boundary that can be measured, optimized, and later replaced without changing the meaning of a selected reference.

Three previously separate interfaces must meet at that boundary. The Paper 2 SDK supplies PRAForCausalLM, model-family injection, routing, and generation. Paper 4 supplies authenticated resolvers and the cold, warm, and hot external-memory lifecycle. Paper 6.5 supplies stable typed identities, callable-derived tool schemas, declarative skills, lazy disclosure, and host-authorized tool execution. Paper 7 supplies type-aware compact views, scoped lossless backing, size-gated native indexing, lazy selected-region encoding, selective materialization, and cursors for large results. Paper 8 supplies versioned task state, task-scoped record admission, and durable user/session resolution. We retain these responsibilities and place one orchestration facade above them. This is integration by contract rather than by collapsing distinct policies.

The report is organized around four falsifiable claims:

- **Correct live-history semantics.** PRA must fork from the same resident state as the ordinary request. Reconstructing or re-prefilling a transcript is a different treatment and cannot establish cache reuse.
- **Engine portability contract.** Same-subset exactness, original- position sparse selection, absence of physical K/V copy, and lifecycle safety are independent gates. Passing one never implies the others.
- **Agent retention semantics.** Agent state is a typed causal graph, not an undifferentiated token stream. Task statements, action–observation pairs, source evidence, mutations, progress, and verification require stable record identities and explicit retention floors.
- **Coding-agent utility frontier.** Only official task outcomes can admit context savings. The locked comparison is No PRA versus PRA at 100%, 90%, and conditionally 75%, crossed with adaptor on/off while reporting success, calls, selected tokens, realized retention, K/V reuse, copy, and time.

The unified runtime, gateway, authenticated resource lifecycle, typed tool and result surfaces, and engine-specific adapters are the implementation needed to test those claims. They are not additional evidence categories. In particular, this paper treats correction of an overstated result as evidence about the measurement contract: cache identity, residence, source position, physical copy, and realized retention must be observed independently.

2 PRA Runtime Abstraction

2.1 Logical and physical memory are different quantities

Consider L decoder layers. Each reference has T tokens, H_{kv} physical key/value heads, head width D , and b bytes per scalar. Storing complete K/V for the reference requires

$$B_{\text{detail}} = 2LH_{kv}TD b, \quad (1)$$

where the factor two accounts for keys and values. If routing selects S unique tokens, the active payload is

$$B_{\text{active}} = 2L_c H_{kv} S D b, \quad (2)$$

where L_c is the number of memory-consuming layers. The ratio $B_{\text{active}}/B_{\text{detail}}$ is a logical opportunity. It is not peak HBM. Peak HBM also includes model weights, local K/V, routing indices, allocator slack, transfer staging, and temporary packed tensors.

Grouped-query and multi-query attention make the distinction especially important. Warm storage should preserve H_{kv} physical heads. Expanding them to the larger number of query heads before materialization duplicates memory without adding information. The runtime therefore represents every native tensor as

$$K, V \in \mathbb{R}^{B \times H_{kv} \times T \times D} \quad (3)$$

and leaves head expansion to the model-family attention adapter.

These equations require four layer sets, not one overloaded set of “PRA layers.” Let L_A store compact addresses, L_{KV} store detailed native K/V, L_R perform request-time routing, and L_C consume selected memory. The runtime enforces

$$L_C \subseteq L_{KV}, \quad L_R \subseteq L_A \quad \text{for layer-native routing.} \quad (4)$$

The persistent detail term is therefore

$$B_{\text{detail}} = 2|L_{KV}|H_{kv}TD b, \quad (5)$$

whereas one request pays active attention over $|L_C|S$ layer-token states. Address layers may be broader than detail-K/V layers when compact search is useful at several depths. Production defaults do the opposite kind of conservatism: detail K/V is encoded only for the resolved consumer profile, unless profile switching or calibration explicitly requests a union.

2.2 Latency is a sum of visible stages

For one request, PRA overhead is not a single routing timer. We decompose it as

$$t_{\text{PRA}} = t_{\text{lookup}} + t_{\text{graph}} + t_{\text{plan}} + t_{\text{gather}} + t_{\text{transfer}} + t_{\text{pack}} + t_{\text{position}} + t_{\text{metadata}}. \quad (6)$$

Generation and attention follow this preparation. Cold requests also pay source fetch and native encoding; compiled paths pay graph capture and code generation. Reporting only warm attention can therefore reverse the practical conclusion.

3 Deployment and Integration Model

3.1 One facade, separate authorities

Figure 1 shows the runtime. The facade does not make every component interchangeable in meaning. It gives them one lifecycle and one inspection surface while retaining their ownership boundaries.

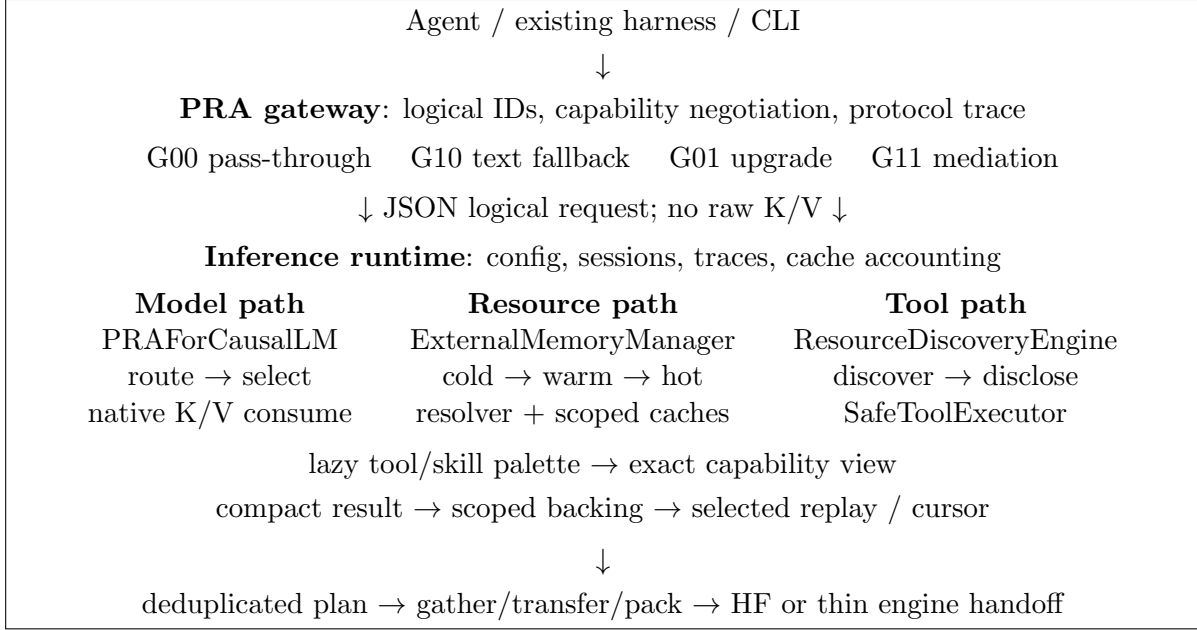


Figure 1: The distributed product boundary. Stable URIs join the paths, but the agent owns task meaning, the gateway owns translation, and the engine owns native K/V. Routing does not authorize source access or tool side effects.

The public configuration has two layers. **PRAConfig** specifies semantic behavior and all four model-layer roles: **address_layers**, **detail_kv_layers**, **routing_layers**, and **consumption_layers**, plus chunking, closure mode, and materialization budget. **PRARuntimeConfig** specifies physical behavior: backend, compilation mode, K/V layout, page size, cache bounds, prefetch policy, and timing synchronization. Both serialize to a versioned JSON artifact. A runtime optimization is valid only when changing the second layer preserves selected identities and output parity under fixed model semantics.

Layer selections use topology-normalized forms: all layers, last n , last fraction, evenly spaced n , or an explicit list. A versioned registry resolves request override, workload/model profile, family balanced fallback, then the all-eligible correctness reference. The named objectives are **reference_correctness**, **quality_max**, **balanced**, and **economy**; the resolved trace records model ID and revision, workload, materialization class, physical layer lists, profile source, and registry version. These are internal optimization objectives. The product registry labels smoke-only realizations of **quality_max** as **QUALITY_MAX_CANDIDATE**. A profile name is not portable evidence by itself.

Address and detail construction have independent lifecycle states (**NOT_BUILT**, **PARTIAL**, **BUILT**, **SKIPPED**, or **DEFERRED**). The detail policy is minimal, profile union, all layers, or explicit. The address policy is routing-only, all candidate routing layers, external-only, or explicit. If a new consumer requires absent detail K/V, the runtime must re-encode missing layers, fail, or follow an explicitly configured profile downgrade. It never silently substitutes a shallower profile.

3.2 Engine instances and runtime model identity

PRA distinguishes engine-instance identity from loaded-model identity. One engine process may expose one or more loaded models. Model-specific PRA state—including bundle, profile, qualification, session state, storage, and native memory—is scoped by a stable runtime-model identity and model fingerprint. Engines supporting only one base model use the same contract with a single **default** loaded model. This keeps multi-model behavior in the management container rather than turning the semantic runtime for one model into a fleet manager.

Internally, the management provider maps each `runtime_model_id` to a small runtime-state record containing the observed model descriptor, storage manager, session source, capabilities, and effective configuration. The old single-model constructor is normalized to one such record. Canonical source records may be shared across models, but derived native identity includes the model fingerprint together with resource identity, version, selected span or representation, and layout. Model-native K/V can therefore neither cross a model switch nor survive an unload as an apparently compatible prefix.

The Registry retains one engine instance with a `models[]` observation and accepts either the legacy singular desired fields or a `desired_models[]` list. Drift is matched by runtime identity and reported both per model and in aggregate. The gateway similarly routes to the pair (*instance, runtime model*), which distinguishes one model replicated on several engines from several models resident behind one upstream. Dynamic load and unload are optional capabilities; a single-model runtime returns `NOT_APPLICABLE`, `ACTION_NOT_SUPPORTED`, or `RESTART_REQUIRED` rather than pretending that process replacement is an in-process model transition.

3.3 A multi-axis execution policy

PRA does not imply one routing frequency. We represent one request with four orthogonal choices. The *selection stage* is request, prefill/completion phase, or token. The *layer scope* is shared logical identities or an independent selection per active layer. The *materialization scope* is request, phase, layer, or token. The *residency policy* retains a payload for that lifetime or releases it after one layer window. Shared identities never mean shared tensors: chunk *C₇* resolves to the native K/V independently encoded at each consuming layer.

The global default is request/shared/request/keep. A model default can override individual fields, and a request override takes final precedence. The resolved policy records field-level provenance. Unsupported combinations raise before generation; there is no implicit downgrade. The engine-neutral `PRASelectionPlan` contains stable URI/chunk identities and row-local spans, never tensors. A layer resolver maps that plan to native payloads, and a separate materialization manager applies physical lifetime and residency.

The HF reference path preserves the old high-level request/shared behavior and the old low-level token/per-layer behavior. It adds request/per-layer by capturing all active layers in one memory-disabled prompt pass, routing each once, and freezing each result. It adds token/shared through a request-owned bridge: layers before the designated routing layer receive no current-token PRA memory; that layer selects once, and later layers resolve the same identities to their own K/V. Phase/shared execution uses a cache-free routing probe at the first active PRA layer, seeds the shared logical plan, and then performs one cache-producing prefill. Decode reselects once at the same boundary and reuses the resulting plan. Requiring the routing layer to be the first active consumer prevents an earlier layer from entering the cache with a different memory view.

3.4 Stable resources and authenticated sessions

A stable URI is the common identity for text memory, native K/V, tools, and typed observations. Paper 4’s `PRASession` retains tenant, user, session, cache scope, admitted resources, and warm/hot handles. Credentials remain opaque callbacks passed only to resolvers; they are absent from prompts, snapshots, and artifacts. Cache presence never grants authorization.

3.5 Durable logical sessions and task scope

An interactive agent needs continuity longer than one model process. The runtime therefore separates logical `AgentSessionState` from physical `PRASession`. Logical state is resolved by user and session identity and contains an ordered typed-record stream, a task descriptor, and an idempotent event

log. Physical state owns admitted resources, native K/V, and result-store handles. Closing it releases accelerator and resolver state without deleting the conversation or task graph.

SessionService is the persistence boundary. Its in-memory implementation provides thread-safe ephemeral operation. Its local implementation hashes path identities, writes versioned JSON through atomic replacement, and rejects stale optimistic versions. A database can implement the same create, resolve, save, list, and delete contract without changing the model path.

Task events create, link, activate, block, resume, complete, or cancel nodes in a validated DAG. Typed records carry producing-task, consuming-task, relation, and event-sequence provenance. Before ordinary PRA ranking, **TaskScopeSelector** admits the active task, its structural closure, or a bounded adaptive neighborhood. Scope controls which records may compete; record and chunk routing still control which admitted detail is materialized.

3.6 Agent and toolset product surface

Toolset couples a callable to its generated schema, namespace, tenant, tags, and declared side-effect class. Bundles compose through the existing **CapabilitySDK**. The built-in local bundle provides file listing, bounded reads and search, exact writes and replacements, Git status, and workspace-rooted command execution. Path operations reject traversal outside the configured workspace. Shell commands remain write-class actions: workspace selection is not a security sandbox.

PRAAgent assembles model, capability discovery, lazy skill folders, safe execution, compact results, durable sessions, and task scope. A turn persists the user record, selects task-relevant records, discloses a bounded tool palette, generates, validates at most a configured number of tool rounds, stores compact output with exact-backing provenance, and persists the final assistant record. The terminal adds resumable sessions and slash commands for tasks, context, and tools. Write and destructive calls are denied by default and can receive one host confirmation; disclosure alone never grants authority.

Paper 6.5's tool resources enter through the same runtime but a separate decision path. Discovery can identify a tool. Disclosure can expose its schema. A model can propose a call. Only **SafeToolExecutor**, given a request-scoped **ExecutionAuthorization**, can validate identity, arguments, write authority, and destructive authority. Its result becomes a typed observation resource with producer and call provenance. This separation prevents a high retrieval score from turning into ambient execution privilege.

3.7 Lazy tools and skills

Paper 6.5's latest capability contract is now part of the facade rather than an adjacent experiment. **AgentConfig** accepts ordinary Python callables, explicit declarative **Skill** objects, and a parent directory containing OpenAI- or Anthropic-style skill folders. Callables are inspected into stable, provider-neutral schemas. Folder loaders parse declarative metadata and instructions but never execute bundled scripts.

The default is a two-stage model-side lifecycle. Discovery and Phase A encode only compact selection views under count and token budgets. Once the model or host chooses a stable record ID, **activate_capability** activates the complete tool schema or skill instructions from immutable local backing. It does not rerun semantic discovery. Encoded views may be cached, while active bytes and resident cached bytes remain separate counters. Eager selection or full-view encoding is an explicit policy override.

3.8 Compact typed result backing

A successful tool output need not return as one flat prompt block. Every open **PRASession** now owns an **AdaptiveContextRuntime** with an exact, hash-verified backing store scoped by tenant and session.

The runtime infers tabular, log, graph, terminal, or generic structured shape for tool and API results. Type-specific compactors retain schema, counts, statistics, errors, and bounded representative units while lexical, entity, rare-term, schema, and optional summary addresses remain retrieval-only.

The public operations preserve stable record identity: compact inspection, address search, full or selected materialization, and bounded cursor actions. `execute_tool_and_record` joins authorized execution to this record lifecycle while retaining producer, call, and observation provenance. Closing an ephemeral session removes its backing bytes.

Paper 7’s final result changes the product default. Native PRA is appropriate for typed backing only while its ingestion and resident-state cost stays inside an explicit budget. Above that budget, the record must remain recoverable through cheaper addresses, search, cursors, and bounded lazy native encoding; it must not be truncated and labeled as a complete native index.

Accordingly, `ContextPolicy` exposes `max_native_index_tokens` and `max_native_index_bytes`, plus per-`RecordType` overrides. The byte gate can reject an oversized descriptor before tokenization; the token gate more directly bounds model work when token accounting is available. The reference defaults are 4,096 tokens and 65,536 bytes, but these are deployment policy values rather than claimed universal optima.

Every record has one auditable native-index lifecycle state:

NOT_REQUESTED BUILT SKIPPED_SIZE_LIMIT DEFERRED

The ordinary Hugging Face loader automatically prepares in-budget records. `BUILT` means complete backing passed through the production PRA encoder. `SKIPPED_SIZE_LIMIT` preserves compact and cheap-index recovery. `DEFERRED` is an explicit synchronous lifecycle decision, not a claim of background work. After a cheap selector identifies a bounded row, field, line, item, or chunk, `encode_result_region_native` authorizes that exact region and creates native K/V only for it. Tear-down removes compact, full, and lazy-region references. A shared multi-tenant model cache is not claimed safe without engine-level partitioning.

3.9 User workflow

The supported workflow is deliberately small:

```
agent = AgentConfig(tools=(lookup,), skills_path="./skills")
context = ContextPolicy(
    max_native_index_tokens=4096,
    max_native_index_bytes=65536,
    record_policies={DB_RESULT: TypeContextPolicy(...)})
runtime = PRARuntime.from_pretrained(
    model_id, runtime_config=config,
    agent_config=agent, context_policy=context)
session = runtime.open_session(...)
palette = runtime.activate_capability_candidates(record_ids)
runtime.activate_capability(selected_id)
result = runtime.execute_tool_and_record(..., session=session)
audit = runtime.result_native_index_audit(
    session, result.record.record_id)
if audit.native_index_state == SKIPPED_SIZE_LIMIT:
    detail = runtime.encode_result_region_native(
        session, result.record.record_id, {"rows": [20, 30]})
trace = runtime.inspect()
```

The same facade still accepts direct references, an `ExternalMemoryManager`, a custom discovery engine, and a safe executor. The executed notebook demonstrates model memory, physical materialization, lazy capabilities, compact results, cursors, size-gated native result routing, and selected-region native promotion with an offline Hugging Face model and pure in-memory tools.

4 Physical Materialization

4.1 Planning before copying

Routing produces logical half-open intervals $I_i = (u_i, \ell_i, a_i, b_i)$, naming URI u_i , layer ℓ_i , and token range $[a_i, b_i)$. The planner groups intervals by URI and layer, sorts them, merges overlaps, and then applies a global token budget. If two requested intervals overlap, their common tokens are copied once. The trace reports requested, unique, and dropped token counts, making deduplication and budget truncation auditable.

The portable materializer slices source tensors, transfers selected slices when needed, and packs them by consumption layer. It reports output bytes, transfer bytes, and temporary bytes. It does not infer peak HBM from these values; CUDA profiling records peak allocator state separately.

4.2 Native-reference semantic gate

Reference K/V is captured at source-relative positions. The direct query must therefore begin strictly after every source key that it may attend to. For independent records r with source lengths L_r , the runtime uses

$$p_{q,j} = \max_r L_r + j, \quad (7)$$

not j and not $\sum_r L_r + j$. The maximum avoids query/source position collisions while preserving each record’s independent coordinate system. Earlier code started $p_{q,0} = 0$, which invalidated full-reference RoPE parity. This was a runtime bug, not a routing or training deficiency.

The corrected path enters selected native K/V and local K/V into the same family-native softmax. It retains grouped-query K/V heads physically, preserves the model’s scaling and causal mask, and keeps the immutable materialization plan active through prefill and every decode call. Compact telemetry records query width, local-cache width, and active native width without retaining attention tensors.

The storage fingerprint now distinguishes fixed `post_rope` keys from optional `pre_rope` keys. A pre-RoPE entry carries a contract over the model revision, every layer’s exact frequency tensor and scaling policy, RoPE dimensions, and rotation layout. At request time an adapter may apply the selected packed positions before exposing the entry to attention. Contract mismatch invalidates the derived cache. Post-RoPE remains the default because it avoids request-time rotation when positions are fixed; pre-RoPE is an experimental composition facility, not a claim that rotation alone restores the causal history missing from independently encoded records.

Papers 3.2 and 3.3 own the detailed document-selection, positional-rebinding, and answer-quality studies. Paper 4.5 imports only their runtime decision: independently encoded pre-RoPE records remain `CALIBRATION_PENDING`. An engine adapter must qualify its RoPE contract and numerical path, and that qualification does not recreate cross-record causal contextualization.

Table 1: HF native-reference correctness ladder. E0 is an offline FP32 mechanism test; E2–E3 reuse the 15-case frozen Qwen3-0.6B Paper 8 ablation.

Gate	Condition	Accuracy	Native tokens	Result
E0	One complete reference; visible/native logits	—	8	5.97×10^{-8} max error
E1	Multiple complete records; boundary/dedup audit	—	144	pass
E2	Selected complete task records; all 28 layers	60.0%	80.2	functional
E3	Paper-3 radius-zero evidence; all 28 layers	0.0%	35.7	insufficient

The bug fix closes the semantic blocker but does not make every sparse geometry adequate. Visible full task scope reaches 73.3%, corrected selected full-record native consumption reaches 60.0%, and sparse late-layer or radius-zero paths remain at 0% on this frozen checkpoint. The first expected token is often ranked correctly even when later generation fails. The remaining boundary is autoregressive composition across materialization depth and layer coverage; training is not proposed as a remedy for a broken native path.

4.3 Cross-model semantic reference gate

We next apply one architecture-neutral protocol to three frozen pretrained models. The exact checkpoints are Qwen/Qwen3-0.6B revision `c1899de2`, unsloth/Llama-3.2-1B revision `9535bd9b`, and google/gemma-3-1b-it revision `dcc83ea8`. The official Meta Llama revision `4e20de36` remained license-gated to the available account; the experiment therefore uses a public weight mirror and records that substitution rather than implying that the official repository was tested. All runs use eager FP16 CUDA on the GTX 950M, one model per process.

Before the semantic cases, each model must preserve logits, hidden states, greedy generation, and the complete dynamic cache exactly with PRA disabled. Reference registration must retain native K/V-head count and head width without query-head expansion, preserve source-relative positions, and remain active for two explicit decode calls. The four measured conditions are a visible prefix, the same complete reference as native K/V, an explicitly selected complete record, and a family-declared sparse layer profile. Three one-token continuations test a capital, color, and numeric code. This small diagnostic localizes integration failures; it is not an answer-quality benchmark.

Table 2: Cross-model HF semantic gate. Selected-full and sparse columns report one-token exact accuracy over three cases. Full-native errors are against the same visible prefix. KV-head fidelity and request-lifetime checks pass for all rows.

Model	Disabled	Full native	Selected full	Sparse profile	KV heads
Qwen3-0.6B	exact	0.0364 max	66.7%	8/28 layers; 66.7%	8/16 preserved
Llama-3.2-1B mirror	exact	0.0234 max	66.7%	8/16 layers; 33.3%	8/32 preserved
Gemma-3-1B	exact	partial; 9.5410 max	100.0%	2/4 global; 100.0%	1/4 preserved

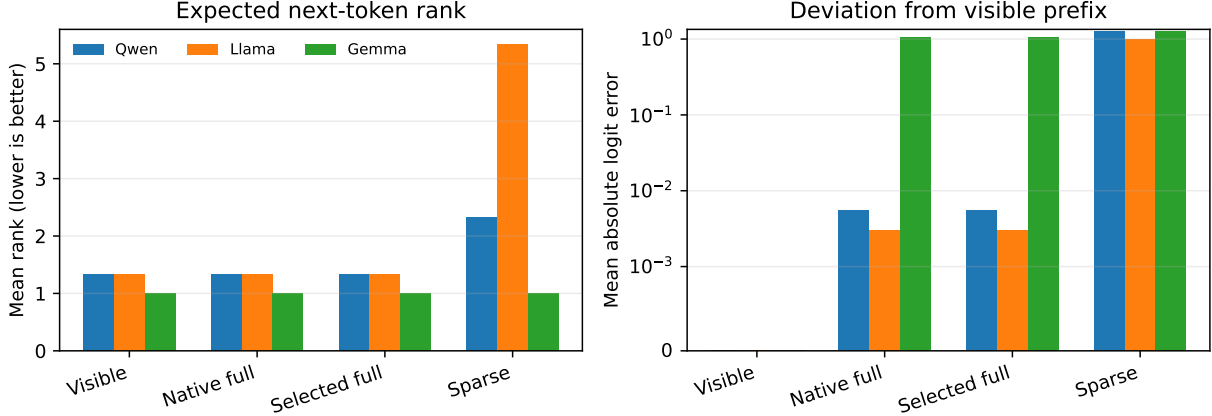


Figure 2: The full and selected-full conditions preserve expected-token rank for Qwen and Llama. Reducing consumer-layer coverage changes semantics despite selecting the same complete record. Gemma’s global-only path retains the top token in these cases but has large logit deviation because unchanged local sliding layers do not consume external K/V.

For Qwen and Llama, full-native replay preserves the visible-prefix top token in all three cases. Maximum FP16 logit error is 0.0364 and 0.0234, respectively. The sparse condition exposes the three independent materialization axes: record selection remains fixed, token width remains the complete selected record, but consumer layers fall from 28 to 8 for Qwen and from 16 to 8 for Llama. Mean expected-token rank consequently moves from 1.33 to 2.33 and 5.33, and Llama accuracy falls from 66.7% to 33.3%. Sparse routing therefore does not establish a semantically adequate consumption profile.

Gemma is a useful failed abstraction boundary. Its 26-layer decoder contains four global-attention layers; the adapter intentionally leaves 22 local sliding-window layers native. Disabled parity, single-head MQA storage, positions, and decode lifetime pass, but global-only native replay differs from the visible prefix by 9.5410 maximum and 1.0667 mean absolute logit error. The one-token top answers remain unchanged, but that coarse agreement is not full-prefix equivalence. We therefore report *partial topology*, not cross-family semantic parity.

The runtime now also carries an attention-only MoE contract inherited from the HF mechanism study. Injection resolves ordinary and nested text decoders, snapshots every sparse MLP, router, and expert-container identity, replaces only eligible attention modules, and rejects the result if any host-owned MoE object changed. Tiny structurally faithful tests cover Qwen3-MoE, nested Mistral 3, and GPT-OSS; GPT-OSS sliding layers remain untouched and its learned attention sinks are forwarded to the native kernel. Qwen’s hybrid GDN/QSA family is described by a fail-closed structural plan: QSA layers are eligible, whereas recurrent GDN state and the sparse indexer remain host-owned. These are implementation contracts, not model-scale qualification results.

The pre-publication queue therefore keeps five gates separate: contract test, exact model load, optional adaptor training, matched No-PRA/PRA-without-adaptor/ PRA-with-adaptor measurement, and immutable bundle publication. Its order is Qwen3-30B-A3B, Gemma 3 12B and 27B, Mistral Small 3.2 24B, GPT-OSS 20B, QwQ 32B, and Llama 3.3 70B. Qwen3-30B-A3B additionally has an exact loaded MLX 4-bit identity and five-seed native-no-adaptor measurements on QASPER, HotpotQA, and 2Wiki. That receipt does not close the three-way gate: its No-PRA and trained-adaptor arms, qualification, and publication remain pending. The other targets currently establish only their structural contracts. Reduced consumer-layer profiles remain **CALIBRATION_PENDING**; **BALANCED** retains all eligible consumer layers.

The executable qualification path fixes the candidate chunks once, compares generic cosine with a five-seed evidence-supervised router, and replays each selection twice: as visible selected text and as all-consumer native K/V. Its five named conditions are full No-PRA, generic selected text, generic native K/V, learned-router selected text, and learned-router native K/V. Thus a router gain cannot be mistaken for a transport gain, and native parity is measured against the exact selected-text twin rather than an independently retrieved context.

4.4 Corrected layer-profile calibration

The earlier sparse column was a diagnostic, not a calibrated deployment profile. We therefore inject all eligible layers once and replay one fixed, complete reference identity through a common profile sweep. Qwen uses all 28 decoder layers; Llama uses all 16; Gemma normalizes over its four eligible global-attention layers. Candidate profiles are all layers; contiguous late suffixes of 1, 4, 8, 12, 14, 16, 20, and 24 where defined; early and middle blocks; and evenly spaced blocks. Reference width, query offset, native post-position K/V, GQA/MQA head count, and shared-softmax attention remain fixed. The 102 rows comprise three frozen one-token cases per model. They calibrate SDK behavior but do not replace Paper 3’s workload-scale placement study.

Table 3: Corrected fixed-identity layer calibration. ΔLP is target-token log probability relative to the visible-prefix control. Rank is averaged over three cases. “All” means all eligible layers, not necessarily every physical decoder layer for Gemma.

Model	Profile	Consumers	Active layer-tokens	ΔLP	Rank
Qwen3-0.6B	all	28	196.0	+0.001	1.33
	last 24	24	168.0	+0.389	1.00
	last 8	8	56.0	−0.650	2.33
	last 1	1	7.0	−1.705	20.33
Llama-3.2-1B	all	16	106.7	−0.002	1.33
	last 8	8	53.3	−1.986	5.33
	last 1	1	6.7	−2.708	16.67
Gemma-3-1B	all four global	4	28.0	−0.038	1.00
	last global	1	7.0	−7.818	72.33

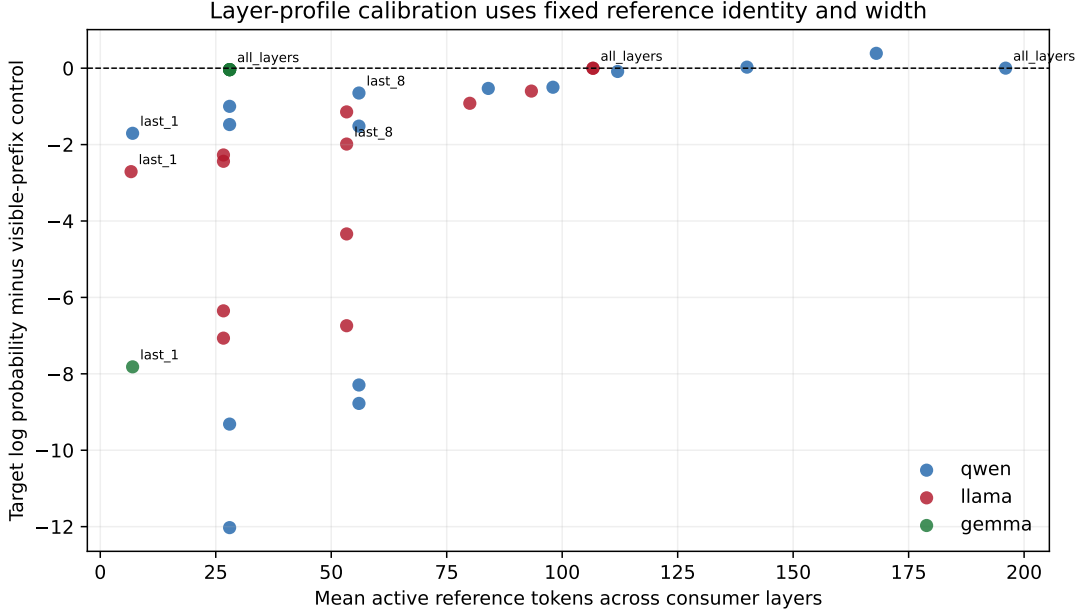


Figure 3: Target-token quality against active layer-token cost with reference identity and token width fixed. The sharp family-dependent trajectories rule out a universal “last eight” product default.

All-layer Qwen and Llama replay are the only tested profiles with maximum full-logit error below .1. Qwen last-24 has the best mean target log probability on these three cases, but its maximum full-logit deviation is 7.14; it is therefore registered under the internal `quality_max` objective but published as `QUALITY_MAX_CANDIDATE`, not as a production-grade `QUALITY_MAX` profile or the balanced product default. Llama requires all 16 layers for the same fidelity criterion. Gemma requires all four eligible global layers to preserve the tested target tokens, yet still cannot reproduce the hidden computation of its 22 unchanged local layers. Thus `balanced` and `reference_correctness` resolve to all eligible layers for all three families in this registry version. The economy profile is an explicit last-one stress point and carries no quality guarantee.

The storage consequence is intentionally separate. Routing-only address state is retained at one late layer, while detail K/V is retained at the resolved consumer layers. On these short references, Qwen detail storage falls from 802,816 bytes for all 28 layers to 688,128 for last 24 and 229,376 for last 8; the routing index remains 2,048 bytes. A service that needs rapid profile switching may ingest the union of candidate detail layers. The production default instead stores only the chosen profile and re-encodes explicitly if a later request asks for missing detail.

4.5 Larger-model profile scaling on Apple Silicon

The small semantic cases above establish a profile contract, not a scaling law. We therefore apply the same normalized suffix space to pinned 4-bit Qwen3-8B, 14B, and 32B checkpoints and Qwen3-30B-A3B on an M4 Pro with 48 GiB unified memory. Each model sees the same five unique annotated-evidence examples per dataset from QASPER, HotpotQA, and 2Wiki. Selection identity and the 384-token source cap are fixed across conditions. This is workload-backed calibration evidence; it is not a routed end-task confirmation because the dataset annotations provide the selected documents.

Table 4: Large-model MLX profile calibration. Concat and segmented agreement are greedy-sequence agreement with selected-text E0. PRA MiB counts active all-layer selected K/V. Every reduced-layer profile remains CALIBRATION PENDING.

Model	Layers	E0 F1	Concat agr.	Seg. agr.	$ \Delta LP$	PRA MiB	Seg./E0
Qwen3-8B	36	0.236	1.000	1.000	0.189	39.6	1.08
Qwen3-14B	40	0.277	1.000	1.000	0.027	43.9	1.05
Qwen3-32B	64	0.231	1.000	0.867	0.039	70.3	1.03
Qwen3-30B-A3B	48	0.256	1.000	0.933	0.146	26.4	1.12

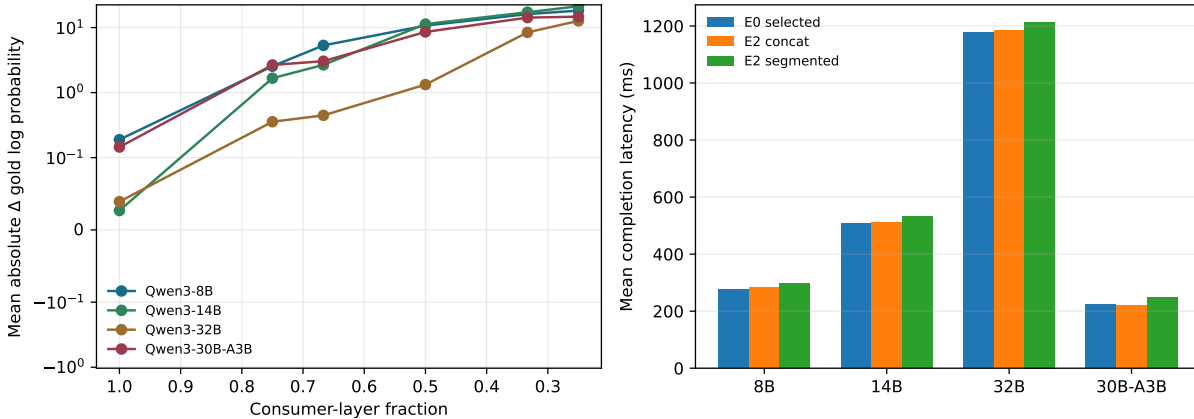


Figure 4: Profile sensitivity and execution time across the dense Qwen ladder and the 30B-A3B control. Larger dense models reduce likelihood sensitivity to missing early consumers, but do not establish sequence-stable sparse profiles.

Full concatenated native K/V preserves all 60 selected-text sequences and is therefore the `REFERENCE_CORRECTNESS` execution path for these rows. The live segmented one-softmax implementation preserves 57/60, with mean absolute gold-log-probability differences of .189, .027, .039, and .146 nats at 8B, 14B, 32B, and 30B-A3B. It remains a kernel candidate rather than a replacement for the exact reference path. The dense last-three-quarter profile becomes less likelihood-sensitive with scale: its mean absolute delta falls from 2.545 to 1.665 to .356 nats. Sequence agreement does not improve monotonically (.267, .467, and .133), so this trend cannot justify a family-level sparse default. The MoE profile uses less active K/V but is more layer-sensitive than dense 32B. Parameter sparsity and semantic-context sparsity are separate profile axes.

These results leave product naming unchanged but strengthen its basis. `REFERENCE_CORRECTNESS` remains all eligible layers. A reduced `QUALITY_MAX_CANDIDATE` may be emitted only with model, revision, workload, materialization, and evidence tier attached. `QUALITY_MAX` remains unassigned; neither a favorable likelihood mean nor a large-model label can promote it without held-out sequence/task validation.

Long-context pressure receipt. The same frozen selections were then held at at most 384 source tokens while same-dataset distractors expanded the full condition to 8K. Selected segmented E2 preserves every selected-text sequence at the measured 8B and 14B points, but the sign of full-minus-selected answer likelihood changes from +1.192 nats at 8B to -1.485 at 14B. This confirms that a profile cannot be promoted from a monotonic larger-model context-tolerance assumption. The physical profile is stable: selected all-layer K/V is 39.6 MiB at 8B and 43.9 MiB at 14B, versus 1,152 and 1,280 MiB for

the 8K dense source. At 32B it is 70.3 MiB versus 2,048 MiB; selected segmented E2 preserves 13/15 sequences with a +0.030-nat mean likelihood shift. At the three-case 8B 32K boundary, selected K/V remains 39.6 MiB while dense native K/V reaches 4,608 MiB. These are CALIBRATION rows. The 64K request exceeds the pinned 40,960-position limit and is recorded as unsupported, not silently shortened.

Corrected 48-GiB scaling and profile-policy receipt. The matched M4 Pro run adds one graph warm-up per path and compares warm with warm and cold with cold. Over 30 natural-QA examples and five seeds per checkpoint, concatenated E2 preserves every E0 output at 4B, 8B, 14B, and 32B. Warm E2/E0 ratios contract from 1.054 to 1.031, 1.018, and 1.011; cold usable- context ratios are 0.932, 0.932, 0.950, and 0.976. The 32B warm 95% paired- bootstrap interval is [1.009, 1.013], so this exact path remains slightly above E0 rather than crossing below it. Unfused all-layer segmented ratios contract from 1.165 to 1.102, 1.055, and 1.047, but sequence agreement is only 0.833 at 32B. An earlier 17.18-GB host correctly recorded 32B as NOT_RUN_CAPACITY_GATE; the 48-GiB result supersedes that capacity limitation without rewriting the earlier receipt.

A held-out noncontiguous Qwen3-4B calibration also fails to remove any layer group under a 0.05-nat likelihood and 0.90 first-token constraint. The calibrated set remains 36/36 layers and all 12 held-out outputs match E0. This is a negative sparsity result, not evidence for a learned reduced profile. Accordingly, BALANCED means all eligible consumer layers; every reduced or segmented realization remains CALIBRATION_PENDING.

Finally, corrected Qwen3-8B occupied-context rows keep selected detail at 39.6 MiB and selected-E2 completion at 387.4 ms across 2K, 8K, and 32K source occupancy, with 18/18 selected-text sequence agreement. The full visible path grows from 6.37 to 31.85 GiB peak allocation. These receipts are imported into the product matrix with model revision, host, seed count, and measurement status; they do not upgrade unfused segmented execution to a production kernel.

4.6 Cross-family learned-router qualification

Structural compatibility does not answer whether an extra routing adapter is useful. We therefore compare two PRA configurations while holding the model, tokenizer, examples, chunking, and extracted hidden states fixed. The generic condition scores each mean chunk gist by normalized cosine similarity to the last query state. The learned condition uses a rank-128 asymmetric linear router trained with a multi-positive softmax objective. Both conditions retain the rest of PRA’s typed-resource, chunk-index, selection, and consumer-layer machinery. Thus the comparison isolates the routing adapter rather than comparing PRA with and without memory.

For each exact checkpoint identity, independent 128-token encoding blocks are divided into 32-token routing chunks. We extract the normalized attention input from the final decoder layer while retaining each architecture’s native normalization, masks, RoPE, grouped-query attention, and, for Gemma, mixed sliding/global topology. Identity-disjoint splits contain 48 training, 16 validation, and 32 held-out test examples, evenly divided between QASPER and HotpotQA. The HF runs use bfloat16 after a finite-feature gate; the gate rejects non-finite extraction before optimization or publication. Five seeds (11, 23, 37, 53, and 71) are trained for 512 steps; the seed with the highest combined validation AUC_{0-30} is selected before the test split is evaluated.

Table 5 separates four conclusions. First, the adapter is useful on QASPER across model scale and family: R_{20} rises from 0.413 to 0.640 for Qwen3-4B, 0.318 to 0.679 for Qwen3-14B, 0.318 to 0.468 for Llama-3.1-8B, 0.226 to 0.454 for Gemma-3-1B, and 0.375 to 0.610 for Qwen3.5-27B. The matched Qwen2.5-1.5B checkpoints extend the result to post-trained HF identities: general instruction tuning rises from 0.296 to 0.381, while code-and-instruction tuning rises from 0.173 to 0.394. Second, this gain is not a universal property of additional parameters. HotpotQA R_{20} declines by 0.044, 0.180, 0.195, 0.018, and 0.210 for the five MLX checkpoints, and by 0.051 and 0.183 for the two HF checkpoints.

Table 5: Held-out routing comparison for exact post-trained HF and quantized MLX model identities. Cohort sizes are retained in the source artifact. The learned router is selected from five seeds using validation data; R_{20} is evidence recall at a 20% chunk budget.

Model	QASPER R_{20}		HotpotQA R_{20}		Combined MRR	
	Generic	Learned	Generic	Learned	Generic	Learned
Qwen2.5-1.5B-Instruct	0.296	0.381	0.385	0.334	0.354	0.423
Qwen2.5-Coder-1.5B-Instruct	0.173	0.394	0.398	0.215	0.346	0.412
Qwen3-4B	0.413	0.640	0.386	0.342	0.333	0.549
Qwen3-4B 8-bit	0.446	0.566	0.407	0.180	0.404	0.499
Qwen3-14B	0.318	0.679	0.494	0.314	0.401	0.645
Qwen3.5-27B 4-bit	0.375	0.610	0.518	0.308	0.501	0.483
Llama-3.1-8B	0.318	0.468	0.616	0.421	0.404	0.537
Gemma-3-1B	0.226	0.454	0.336	0.318	0.316	0.494

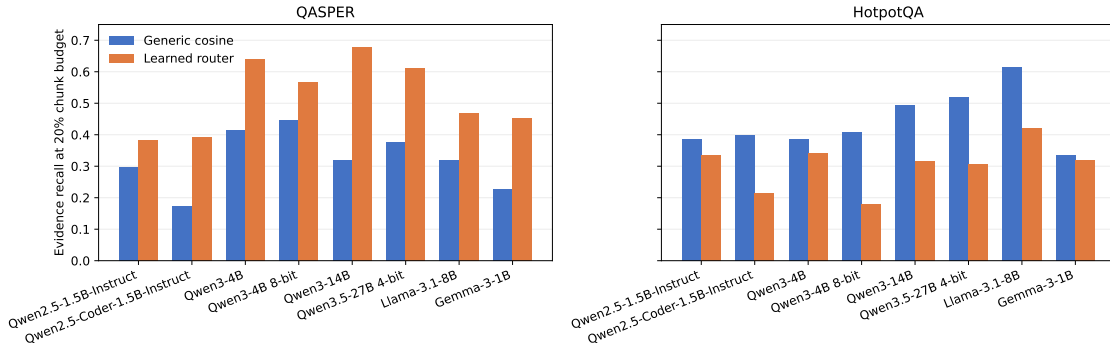


Figure 5: Parameter-free generic routing and the validation-selected learned router on held-out identities. Learned routing consistently improves QASPER, but HotpotQA recall is flat or lower. The adapter is consequently a dataset-qualified option, not the catalog default.

Gemma’s HotpotQA MRR nevertheless rises from 0.385 to 0.538, showing that even the direction can depend on the metric. Third, model size alone does not remove domain dependence: Qwen3.5-27B retains a substantial QASPER gain while producing the largest Qwen HotpotQA loss, whereas Qwen3-14B has the largest QASPER gain. Fourth, post-training changes routing geometry even when architecture and size are matched. The code-tuned checkpoint starts from lower generic QASPER recall but receives the larger learned gain; its qualification still concerns document evidence routing, not source-code retrieval or coding-task accuracy.

The SDK policy follows the measurement. Every public bundle exposes canonical **quality**, **balanced**, and **economy** profiles, but only **balanced** is currently qualified and recommended. Bundles with a learned asymmetric router additionally expose **qasper-learned** as an opt-in research profile. These are exact checkpoint identities; no result is transferred between MLX and HF, between base and post-trained checkpoints, or across tokenizer revisions. Routing diagnostics remain separate from generation quality, equivalence, and serving economics.

4.7 Qwen3.5-27B hybrid-attention qualification

The 27B Qwen3.5 checkpoint tests a materially different deployment boundary. We run the exact `mlx-community/Qwen3.5-27B-4bit` revision `45797d2...9553` on an M4 Pro with 48 GiB unified memory. The model has 64 decoder layers in a repeating three-DeltaNet/one-full-attention pattern, so only 16 layers expose ordinary attention K/V. A short warm-up decodes at 31.1 tokens/s with 15.4 GiB

Table 7: Qwen3.5-27B end-task qualification. PRA conditions retain 20% of source chunks; TTFT includes selection and visible selected-context prefill.

Dataset	Condition	Token F1	Evidence recall	Visible tokens	TTFT (ms)
QASPER	Full / No PRA	0.532	1.000	3648	32945
QASPER	PRA generic	0.385	0.344	773	7095
QASPER	PRA learned	0.459	0.553	773	7079
QASPER	Oracle control	0.470	0.958	773	7042
HOTPOTQA	Full / No PRA	0.887	1.000	1669	15305
HOTPOTQA	PRA generic	0.602	0.506	375	3642
HOTPOTQA	PRA learned	0.320	0.292	375	3638
HOTPOTQA	Oracle control	0.900	1.000	375	3640

peak unified allocation. PRA can select and serialize evidence for the whole model, and the router can use the final attention-input representation. It cannot honestly detach only K/V: the remaining 48 layers also require recurrent and convolution state. The bundle therefore advertises Selected Context and fails closed for Native Memory.

The routing protocol matches the catalog experiment above: 48 training, 16 validation, and 32 identity-disjoint test examples; five seeds; 32-token chunks; and a rank-128 asymmetric router with 1,310,720 parameters. The validation-selected seed improves QASPER R_{20} from 0.375 to 0.610 and MRR from 0.346 to 0.683. It simultaneously lowers HotpotQA R_{20} from 0.518 to 0.308 and MRR from 0.656 to 0.282. This is not a general 27B routing upgrade. The public `qasper-learned` profile is opt-in; generic routing remains the default.

Table 6: Qwen3.5-27B held-out routing at a 20% chunk budget.

Dataset	Generic R_{20}	Learned R_{20}	Generic MRR	Learned MRR
QASPER	0.375	0.610	0.346	0.683
HOTPOTQA	0.518	0.308	0.656	0.282

We then execute the same 32 test questions under full context, generic PRA, learned PRA, and an evidence-aware oracle control. The selected conditions use about 375 visible tokens instead of 1,669 on HotpotQA and 773 instead of 3,648 on QASPER. Mean TTFT falls from 15.31 to 3.64 s and from 32.94 to 7.08–7.09 s, respectively. Quality depends on routing. Generic HotpotQA F1 is 0.602 versus 0.887 full, while the oracle reaches 0.900 with the same selected-token budget. On QASPER, learned routing improves F1 from 0.385 generic to 0.459, narrowing but not closing the gap to 0.532 full. The oracle’s QASPER evidence recall is 0.958 but F1 is only 0.470, which also exposes answer-generation error and the small cohort’s sensitivity to yes/no response formatting.

The immutable bundle is published as `EInnovator/pr-qwen3-5-27b-mlx-4bit` at commit `48d50f2...fde0`. It contains the structural contract, the router, exact base/tokenizer revisions, aggregate and row-level qualification receipts, and the native-memory exclusion. An adapter can therefore improve a qualified domain without being enabled globally, while unsupported physical state never silently degrades to a misleading K/V-only implementation.

Evidence-to-card publication. A schema-checked importer now reads the paired MLX artifacts rather than copying numbers into model cards. It rejects a mismatch in model ID, immutable revision, quantization, engine/version, profile, or execution mode. For the 8B, 14B, and 32B Qwen identities, the exported **BALANCED** headline uses the concatenated all-consumer native path: all 15 paired natural-QA outputs match selected text, token F1 is unchanged, and visible prompt tokens fall by 89.1%. Mean

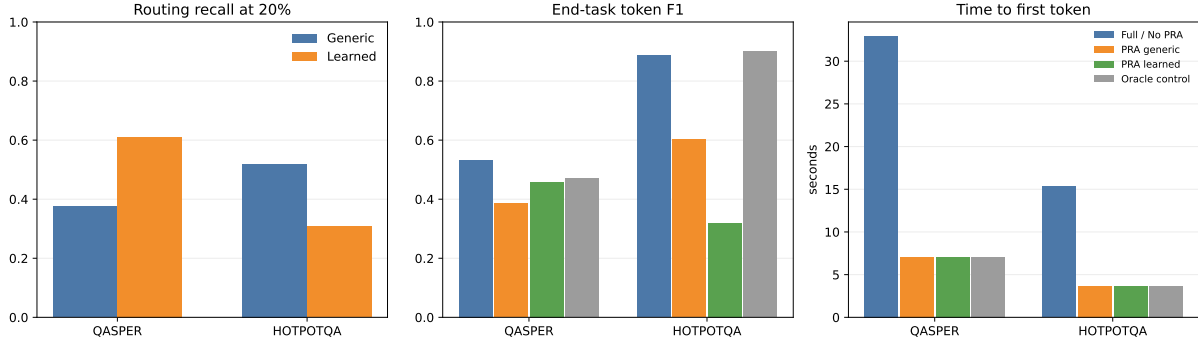


Figure 6: Qwen3.5-27B qualification separates routing recall, answer F1, and TTFT. The learned router is useful on QASPER but reverses direction on HotpotQA. Oracle quality at the same budget shows that better selection has substantial headroom; model size alone does not supply it.

completion-latency deltas are respectively +3.0%, +1.1%, and +0.5%; the 32B median-TTFT point estimate is -6.6%. These small cohorts support ENGINE QUALIFIED, not production-wide qualification. Router recall is printed only under research diagnostics.

4.8 Product profile and benchmark matrix

The registry also emits a strict machine-readable product row rather than mixing numbers and labels such as “not measured.” Schema 2.0 keeps numeric values, measurement status, and provenance in separate fields for every quality, latency, memory, transfer, scheduler, and reuse metric. Missing values are null with explicit NOT_MEASURED or NOT_APPLICABLE status; they are never encoded as zero. Unknown columns, duplicate row identities, unsupported integration levels, throughput without a paired quality measure, and invalid status values fail validation. Table 8 is generated from the same JSON artifact used by the SDK.

Table 8: Compact product matrix. Sparse quality-max settings remain candidates until held-out workload calibration closes their evidence tier.

Model	Profile	Level	Quality Δ	Visible $\downarrow$	Active KV $\downarrow$	TTFT	Status
Qwen	REFERENCE_CORRECTNESS	E2	0.0%	—	0.0%	—	MEASURED
Qwen	QUALITY_MAX_CANDIDATE	E2	4.3%	—	14.3%	—	CALIBRATION PENDING
Qwen	BALANCED	E2	0.0%	—	0.0%	—	MEASURED
Qwen	ECONOMY	E2	-8.8%	—	96.4%	—	MEASURED
Llama	REFERENCE_CORRECTNESS	E2	0.0%	—	0.0%	—	MEASURED
Llama	QUALITY_MAX_CANDIDATE	E2	0.0%	—	0.0%	—	CALIBRATION PENDING
Llama	BALANCED	E2	0.0%	—	0.0%	—	MEASURED
Llama	ECONOMY	E2	-31.1%	—	93.8%	—	MEASURED
Gemma	REFERENCE_CORRECTNESS	E2	0.0%	—	0.0%	—	NOT MEASURED
Gemma	QUALITY_MAX_CANDIDATE	E2	0.0%	—	0.0%	—	CALIBRATION PENDING
Gemma	BALANCED	E2	0.0%	—	0.0%	—	NOT MEASURED
Gemma	ECONOMY	E2	-87.4%	—	75.0%	—	NOT MEASURED

The calibrated layer candidates become useful to an SDK only after their semantics and evidence are inspectable. The current registry publishes four profiles: REFERENCE_CORRECTNESS, QUALITY_MAX_CANDIDATE, BALANCED, and ECONOMY. The accepted QUALITY_MAX name is deliberately reserved and has no evidence row until workload-scale validation supports it. A semantic row is keyed by model ID, immutable revision, workload, and profile. Engine, engine version, hardware, and dtype identify a physical realization of that row; they do not redefine the profile. This separation prevents one GPU measurement from becoming an implied portable latency property.

Table 9 is generated directly from the packaged JSON registry. Quality is mean target-token probability over the same three fixed continuation cases used above. Retention and delta are recomputed against the model’s reference-correctness row. Materialized tokens count selected source tokens, whereas active K/V counts those tokens across consuming layers. Detail savings measure persistent native K/V, so they remain distinct from active attention savings. The compact manuscript table shows active K/V; materialized tokens, detail storage, and the complete physical realization remain in the generated registry and technical table.

Table 9: Model-specific product profiles. This is SMOKE evidence over three fixed cases, not workload-scale quality. Quality-max candidates are explicitly CALIBRATION PENDING; Gemma rows also remain partial-topology at the mechanism level.

Model	Product profile	Quality	Ret.	Δ	Active K/V	Save	Evidence	Status
Qwen3-0.6B	Ref. correctness	0.4364	100.0%	+0.0000	196.0	0.0%	Smoke	Measured
Qwen3-0.6B	Quality max candidate	0.4790	109.8%	+0.0427	168.0	14.3%	Smoke	Calibration Pending
Qwen3-0.6B	Balanced	0.4364	100.0%	+0.0000	196.0	0.0%	Smoke	Measured
Qwen3-0.6B	Economy	0.3488	79.9%	-0.0875	7.0	96.4%	Smoke	Measured
Llama-3.2-1B	Ref. correctness	0.5083	100.0%	+0.0000	106.7	0.0%	Smoke	Measured
Llama-3.2-1B	Quality max candidate	0.5083	100.0%	+0.0000	106.7	0.0%	Smoke	Calibration Pending
Llama-3.2-1B	Balanced	0.5083	100.0%	+0.0000	106.7	0.0%	Smoke	Measured
Llama-3.2-1B	Economy	0.1972	38.8%	-0.3111	6.7	93.8%	Smoke	Measured
Gemma-3-1B	Ref. correctness	0.8753	100.0%	+0.0000	28.0	0.0%	Smoke	Partial Topology
Gemma-3-1B	Quality max candidate	0.8753	100.0%	+0.0000	28.0	0.0%	Smoke	Calibration Pending
Gemma-3-1B	Balanced	0.8753	100.0%	+0.0000	28.0	0.0%	Smoke	Partial Topology
Gemma-3-1B	Economy	0.0009	0.1%	-0.8744	7.0	75.0%	Smoke	Partial Topology

Qwen’s last-24 candidate has 109.8% mean-probability retention while reducing active and detail K/V by 14.3%; this narrow positive result motivates its quality-max-candidate label but does not establish general superiority or a production **QUALITY_MAX** profile. Its registry row pairs **evidence_tier=SMOKE** with **profile_status=CALIBRATION_PENDING**. Qwen economy retains 79.9% on the smoke cases with 96.4% K/V savings. The corresponding economy points are poor for Llama (38.8% retention) and unusable for Gemma (0.1%), which is precisely why the SDK does not ship one universal sparse layer rule. Balanced remains all eligible layers for this registry version.

The semantic label resolves to inspectable mechanism settings. Explicit request fields are applied afterward and therefore take precedence over profile defaults. The runtime trace records **profile_requested**, the resolved profile, its source, registry version, evidence tier, measurement status, and separate product-readiness **profile_status**. A request for the reserved **QUALITY_MAX** label or an uncalibrated model/workload pair returns **CALIBRATION_PENDING** rather than borrowing candidate evidence.

Table 10: Resolved technical realization. Counts refer to eligible physical layers. Compression and search are fixed controls in this semantic smoke.

Model	Profile	Compression	Search	Materialization	Address	Detail	Route	Consumer
Qwen3-0.6B	REFERENCE.CORRECTNESS	none_semantic.smoke	fixed_full_reference	full_selected_record	1	28	1	28
Qwen3-0.6B	QUALITY_MAX.CANDIDATE	none_semantic.smoke	fixed_full_reference	full_selected_record	1	24	1	24
Qwen3-0.6B	BALANCED	none_semantic.smoke	fixed_full_reference	full_selected_record	1	28	1	28
Qwen3-0.6B	ECONOMY	none_semantic.smoke	fixed_full_reference	full_selected_record	1	1	1	1
Llama-3.2-1B	REFERENCE.CORRECTNESS	none_semantic.smoke	fixed_full_reference	full_selected_record	1	16	1	16
Llama-3.2-1B	QUALITY_MAX.CANDIDATE	none_semantic.smoke	fixed_full_reference	full_selected_record	1	16	1	16
Llama-3.2-1B	BALANCED	none_semantic.smoke	fixed_full_reference	full_selected_record	1	16	1	16
Llama-3.2-1B	ECONOMY	none_semantic.smoke	fixed_full_reference	full_selected_record	1	1	1	1
Gemma-3-1B	REFERENCE.CORRECTNESS	none_semantic.smoke	fixed_full_reference	full_selected_record	1	4	1	4
Gemma-3-1B	QUALITY_MAX.CANDIDATE	none_semantic.smoke	fixed_full_reference	full_selected_record	1	4	1	4
Gemma-3-1B	BALANCED	none_semantic.smoke	fixed_full_reference	full_selected_record	1	4	1	4
Gemma-3-1B	ECONOMY	none_semantic.smoke	fixed_full_reference	full_selected_record	1	1	1	1

Runtime fields use an explicit missingness contract. The registry contains TTFT, inter-token

latency, tokens/s, p50/p95/p99, throughput, peak HBM and RAM, H2D bytes and time, cache hit rate, index build time, prefetch overlap, and concurrency. None was measured by the layer-quality calibration, so the generated view says NOT MEASURED; it does not reuse the separate microbenchmark later in this paper.

Table 11: Physical realization fields for balanced semantic rows. Missing serving measurements remain explicit.

Model	Engine	Hardware	dtype	TTFT	tok/s	p95	Runtime status
Qwen3-0.6B	huggingface_eager	NVIDIA GeForce GTX 950M	torch.float16	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT_MEASURED
Llama-3.2-1B	huggingface_eager	NVIDIA GeForce GTX 950M	torch.float16	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT_MEASURED
Gemma-3-1B	huggingface_eager	NVIDIA GeForce GTX 950M	torch.float16	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT_MEASURED

4.9 Inherited typed-record product policy

The runtime also forward-ports Paper 7’s finalized typed-record policy without reinterpreting its metrics as layer-profile evidence. Per-record compression accepts target-token, maximum-token, and ratio bounds. Typed postings, indexed BM25, and a small embedding index are built independently of the full-body native Q/K size gate. Above that gate, cheap hybrid discovery returns an exact selector; only the selected region is eligible for lazy native encoding and detail-K/V materialization.

In Paper 7’s controlled protocol, type-aware compression reduced mean visible tokens from 180.8 to 54.1, a compact ratio of 25%. Automatic recovery was 71.9%, and cheap-hybrid recall at four was 87.5% over 24 cases. These are inherited measurements with their original cohort and provenance, not new Paper 4.5 quality results. Their role here is to define canonical runtime behavior: full native indexing is budgeted, while cheaper address modes and exact backing identities remain available above the gate.

The task smoke invokes the inherited Paper 8 task graph, provenance attachment, and task-structural selector rather than copying expected IDs into the report. It selects the expected record in all 18 model/condition rows (three tasks, three models, visible and selected-full conditions). Selected-full one-token accuracy equals visible-prefix accuracy for every model. Tool and skill disclosure, safe execution, compact results, and session persistence are shared control-plane contracts and pass the executed SDK tests and notebook; we do not label them model-generated tool-use results. The base Llama mirror has no chat template, so no Llama tool-call claim is made.

4.10 Caching and reload amplification

The runtime includes a thread-safe, byte-bounded LRU for opaque warm or hot payloads. Besides hits, misses, and evictions, it records loaded and reused bytes. We define reload amplification as

$$A_{\text{reload}} = \frac{B_{\text{loaded}}}{\max(B_{\text{resident}}, 1)}. \quad (8)$$

An apparently high hit rate can still be costly if the misses repeatedly load large references. Conversely, byte reuse can be high even when object-level hit rate is modest. Both views are needed. Native reusable keys include tenant, user, session, resource, layer, payload variant, source revision, positional frame, materialization layout, and scope signature. Raw backing can be reused across compatible encodings, but per-layer, rebound, and packed payloads require exact geometry identity. The audit accepts only the identical key and rejects changed source, position, packing, or request scope. Per-tenant byte caps evict within that tenant before the global cap is considered, and scope-local cleanup cannot evict another tenant’s entries. A concurrent same-URI fixture confirms tenant isolation under pressure and cancellation cleanup. This is logical/runtime isolation, not hard GPU partitioning.

5 Engine-Portability Model

5.1 Agent, gateway, and engine are independent

The distributed contract has three authorities. The agent owns user-visible task and session semantics, typed records, provenance, and authorization references. The gateway owns transport sessions, logical resource mapping, protocol translation, and explicit capability negotiation. The inference engine owns tensor layout, physical K/V blocks, device placement, attention, and scheduling. The normal wire carries JSON logical identities rather than raw K/V tensors.

The agent boundary is now explicit in code. `AgentTurnContext` contains the conventional chat trajectory and separate context, tool, skill, and task records. `PRAAgent` performs semantic selection but does not choose a wire representation. `NegotiatedRemoteBackend` probes `/v1/prai/capabilities` once at launch or reconnect and resolves TEXT, PRA-FULL, or PRA-DELTA from advertised features. A reachable endpoint without the capability route is ordinary OpenAI; an unreachable endpoint is an error, not a reason to silently downgrade.

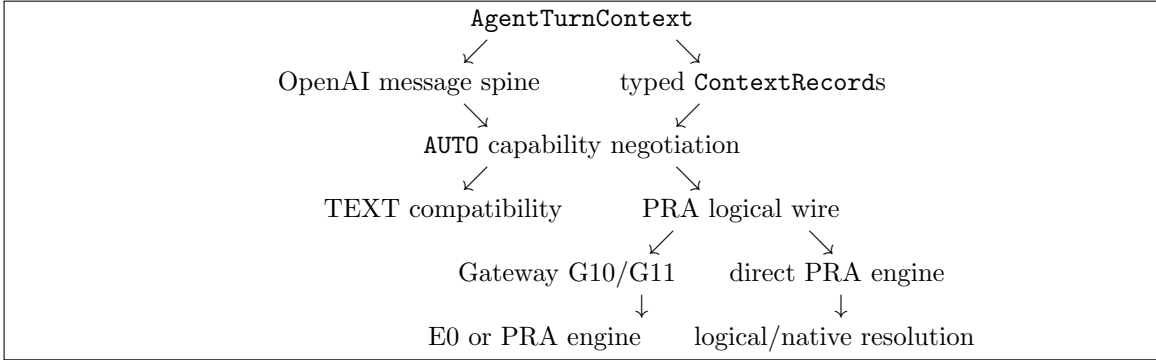


Figure 7: The agent preserves the pretrained conversational spine separately from detached typed context. Compatibility loss occurs only at an explicit TEXT or G10 downgrade; native K/V remains engine-side.

The canonical record projection preserves ID, type, URI, version, source fingerprint, provenance, authorization scope, task ownership/status, available views, selected view, and sharing scope. It never exports Python policy objects, native tensors, or physical page IDs. G10 advertises that its immediate gateway accepts typed records even when its downstream engine is E0; consequently `AUTO` sends logical records to the gateway and the gateway owns deterministic text injection. G11 forwards record, task, and session semantics. An ordinary server receives no `prai` field.

The implemented `PRAWireRequest` preserves messages, tenant/session/task IDs, resources, facets, budgets, execution-policy hints, provenance, and required capabilities in a version-1 extension of the OpenAI request. `PRAEngineAdapter` reports static capabilities, prepares and closes sessions, and offers generation plus streaming. Four deterministic gateway modes cover both sides of the migration: G00 passes ordinary traffic to an ordinary engine; G10 converts explicitly selected resources to labeled text for a PRA-unaware engine; G01 infers typed resources from system and tool-result boundaries before calling a PRA-aware engine; and G11 preserves an explicit PRA request for a PRA-aware engine. G10 is control-plane PRA with text materialization, not native PRA execution.

Capability negotiation is fail-closed. A request that requires native K/V is rejected by an E0 engine unless it explicitly allows text fallback and selects G10. The trace then names the unsupported capabilities, downgrade, selected logical IDs, gateway mode, correlation ID, and whether native K/V was used. Cross-tenant resources and credential-like request metadata are rejected before translation. Relevance still does not grant source or tool authorization. G01 promotes only explicit record

boundaries—system records, tool results, attachments, and already typed messages. It does not infer semantic records from arbitrary unstructured user traffic.

We freeze four integration levels. E0 receives ordinary selected text. E1 understands logical PRA identity, versions, intervals, authorization, session deltas, and fallback semantics, but still realizes context through an ordinary model path. E2 consumes selected resources directly as detached native K/V or an equivalent non-prefix representation with matched geometry, one attention normalization, request-local lifetime, and isolation. E3 adds scheduler-owned placement, prefetch, promotion, eviction, affinity, batching, and distributed movement on top of E2. The local HF adapter is the E2 semantic reference. The OpenAI-compatible HTTP adapter and standalone `pra gateway serve` process implement E0. FreeToken remains reachable through that E0 surface [3]. Companion Papers 6.1 and 6.2 exercise native SGLang-MLX and MLX-LM E2 mechanisms; their provider declarations are available here, but their engine measurements do not promote Paper 4.5 to E3. Paper 6 now exercises native vLLM-Metal V1 generation and scheduler-disjoint pages under a pinned provider, while its generic provider remains E0 unless that bridge is explicitly installed. The portable HF adapter implements E2 native execution and streaming but remains stateless between gateway calls. The configurable remote E1 envelope carries incremental messages and resource operations when the selected engine advertises those capabilities. The HTTP gateway emits OpenAI-compatible server-sent events. Cancellation is cooperative at token boundaries; request-owned references are released only after the decode worker stops. No backing content is included in trace metadata. The capability object names native K/V, logical references, streaming, selected-interval materialization, request lifetime, host/device residency, tenant isolation, phase selection, and scheduler hints independently. The HF reference advertises phase selection through the constrained cache-correct handoff above, but still rejects scheduler hints. This prevents an E2 adapter from being mistaken for an E3 scheduler merely because both accept the same logical wire request.

5.2 Session-aware realization and cumulative context reuse

PRA does not replace ordinary prefix caching. A session may simultaneously retain a conventional sequential-prefix K/V cache and separately address PRA records or native detail. The gateway preserves append-only prefixes for ordinary engines and can send logical deltas to PRA-aware remote engines. Between selection and request construction, an engine-independent realization planner reconciles a visible-materialization ledger against the actual serialized model context. Identity is strict: resource ID, version, selected interval, rendering profile, and rendered-content digest must all agree. Compatible occurrences remain in place; dropped, superseded, removed, or task-closed occurrences materialize again. This rule survives process restart through a durable logical snapshot, while opaque engine handles and inferred physical residency are intentionally discarded.

The planner classifies each selected item as already visible, natively available, requiring materialization, optionally refreshable, or invalid. Its accounting keeps three quantities disjoint. Logical visibility prevents a duplicate input occurrence. An engine prefix observation reports only physical reuse of a sequential prefix and may be confirmed, likely, unknown, or absent. Native Memory reports selected model-native state only on a qualified path. Correctness never depends on a likely or unknown prefix hit, and worker, model-fingerprint, or engine-restart changes invalidate unproven continuity. The first PRA-DELTA turn sends a full inventory. Later unchanged resources emit body-free UNCHANGED operations; changed identity, content fingerprint, authorization, task ownership, or shareability emits ADD or UPDATE, while retirement emits REMOVE. The gateway retains bodies outside its public trace so G10 can render unchanged resources without requiring retransmission. Endpoint restart, reconnect, protocol mismatch, or explicit refresh clears the agent’s acknowledged inventory and forces a full resynchronization before delta mode resumes. Figure 8 makes these state classes explicit. The inference engine physically owns the ordinary prefix cache. PRA logically owns resource identities,

versions, addresses, and selection; the physical owner of their indexes or selected native K/V depends on E2–E3 integration. A hit or invalidation on one axis therefore says nothing by itself about the other.

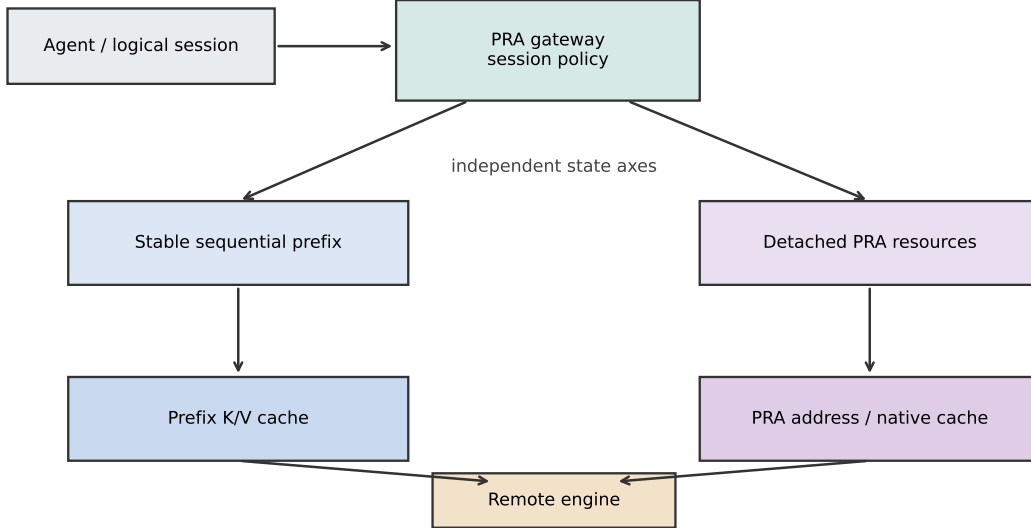


Figure 8: The session-aware gateway maintains two independent state axes. Sequential prompt K/V follows the engine’s ordinary prefix-cache policy; detached PRA resources follow the PRA lifecycle and may be text-materialized at E0 or retained natively at E2+.

The wire contract promotes `history_mode`, engine-session identity, prefix handle, cache-affinity key, maximum decode length, and resource operations to typed fields. `FULL` means that the request supplies canonical complete history; `DELTA` means only appended messages; `AUTO` resolves from capabilities rather than message count. Stable resources similarly produce `ADD`, `UPDATE`, `REMOVE`, or `UNCHANGED` operations while preserving ID, URI, version digest, record type, and authorization scope.

A versioned engine registry resolves OpenAI-generic, vLLM, SGLang, FreeToken, Ollama, Hugging Face, llama.cpp, MLX, and custom profiles. Unknown compatible servers safely remain E0, full-history, and cache-telemetry-unknown. vLLM and SGLang profiles represent automatic prefix-cache capability but do not infer native PRA from an engine label. Companion E2 mechanism results are registered through explicit providers and remain separate from Paper 4.5 evidence. The local HF reference remains E2-native but stateless across calls; the session/resource-delta row is exercised by a deterministic remote-engine contract fixture. Explicit configuration may override registry defaults; resolved capabilities, rather than labels, control transport.

For G10, the former message-zero synthetic system block remains only as a legacy experimental control. The default inserts selected evidence into the current user turn, after the byte-identical prior serialized conversation. Role-sequence tests cover Qwen, Llama, Gemma, and generic OpenAI-style transports. This placement preserves template validity and the previous prefix. It does not imply that changing evidence is cacheable: only the prior sequential prefix is reusable.

The sidecar is keyed by tenant, logical session, and model. It retains only ephemeral engine handles, prefix fingerprints, resource versions, policy digests, and affinity hints; durable canonical history continues to use `SessionService`. Preparation is idempotent, explicit close releases engine state, and cancellation of one stream does not close its reusable session. Model revision, chat-template, visible-prefix, history-rewrite, engine-restart, and invalid-handle changes invalidate the relevant handle. Updating one detached resource emits a resource delta without invalidating an otherwise stable

conversation prefix. The cache-affinity key is only a future scheduler hint; no worker-placement result is claimed.

5.3 Agent-side compatibility before native engine work

The pre-SGLang deployment gate asks whether PRA’s logical record semantics can survive an existing agent without patching its model runtime. The `DeepSeekHarnessPRAAdapter` consumes durable tool-result, session-reference, and attachment events. The `PiCodingAgentPRAAdapter` consumes Pi’s public tool-completion and completed-message events. Both deduplicate event identities, retain typed provenance and task/session IDs, and emit the same tensor-free `PRAWireRequest` used by the HTTP gateway.

Across 10 deterministic five-seed contract cases, the DeepSeek and Pi bridges each pass 100.0% and 100.0% of identity, typing, task-metadata, deduplication, and explicit-fallback checks. In every G10 case the ordinary engine receives labeled evidence text, zero logical resources, and no native-K/V claim. These tests validate the integration boundary, not the quality or latency of either upstream agent.

5.4 Compilation ladder

The eager path is the correctness baseline. The first compiled primitive is a paired `index_select` over K and V with dynamic selected-token count. Cold compile time and warm latency are separate rows, and parity is checked against eager outputs. Failure never falls back into an eager row.

Triton, custom CUDA, and fusion follow profiling rather than precede it. The candidate hot paths are interval gather, deduplication/remap, packing, and position preparation. A kernel is justified only if it improves the complete request after launch, transfer, and synchronization costs. This ordering prevents a fast isolated kernel from becoming a misleading end-to-end result.

5.5 Thin vLLM handoff

PagedAttention showed how virtualized K/V blocks improve serving utilization and batching [6]. Deep PRA integration would additionally make logical PRA blocks, semantic residency, and retrieval-aware prefetch visible to the scheduler. That is outside this paper’s boundary.

The implemented `VLLMThinBackend` instead emits a `VLLMThinRequest`: prompt, stable selected URIs, materialized token count, and ordinary K/V metadata. Semantic scores do not cross the boundary. An ordinary engine can consume already selected memory without owning retrieval policy. When no executor is installed, the backend produces the handoff for inspection and raises on generation rather than pretending to serve.

6 Relation to Existing Systems

PRA is complementary to several kinds of memory optimization. H2O and SnapKV select or retain influential cache entries inside an active sequence [11, 7]. PRA begins from stable external identities and query-time addressability, so its inactive detail K/V may live outside the accelerator. These methods can eventually share page layouts or eviction mechanisms, but their selection semantics differ.

Prompt Cache and CacheBlend reuse precomputed prompt or retrieved-knowledge state [4, 10]. Their lesson is directly relevant: recomputation and transfer can dominate. PRA adds bounded semantic selection before native K/V consumption. RetrievalAttention similarly joins vector retrieval with long-context attention, reinforcing the need to count index and transfer work rather than only selected attention work.

GQA reduces the number of physical K/V heads [1]. Our runtime preserves this layout through warm storage and materialization. It does not expand K/V to query-head count as an implementation convenience.

7 Canonical Evaluation Methodology

7.1 Questions and claim levels

The initial gate asks four narrow questions. First, does the portable selected- K/V implementation preserve exact tensor parity? Second, how large are eager gather and interval-pack costs? Third, how much does CPU residency add for the same selected payload? Fourth, can the local toolchain execute the compiled path? Quality and routing selection remain frozen.

Claims are labeled by evidence. *Measured* means executed on the recorded host. *Inherited* means read from a checked-in earlier experiment with its own protocol. *Contract only* means code exists but no external engine ran. *Architectural* means only an interface comparison is made.

7.2 Superseded evidence ledger

Several early agent-cache results remain useful diagnostics but no longer support the stronger claim originally associated with them. We retain their artifacts rather than deleting inconvenient evidence. Table 12 states the demotion rule applied throughout the report.

Failure found	Evidence superseded or demoted	Admissible replacement evidence
Cache reconstruction	Fresh-prefill, cache-off, and independently reconstructed “same-state” comparisons across the early HF, MLX, vLLM, and SGLang probes. They measure rematerialization behavior, even when outputs match.	Ordinary and PRA continuations fork from one live resident source; 100% selection is a no-op; source ownership remains pinned; selected history is never re-tokenized or prefilled.
Position-base error	Any sparse row that used compact selected width as the decode position or failed to carry original source extent through every decode step. Token agreement from such a row cannot qualify sparse K/V.	A reference consumes the identical subset under the same original positions; selected length, source-position extent, and request suffix are separate recorded fields.
Retention-ceiling semantics	Early nominal-90% whole-bundle runs and v1 sparse-selection rows whose realized retention could fall to 67.1%. Their labels do not describe a 90% treatment.	The corrected causal policy treats 90% as a floor, rounds up indivisible mandatory bundles, and reports both requested and realized retention (90.32–95.66% in the frozen seven-turn cross-engine probe).
Physical copy mistaken for reuse	Packed or detached selected-K/V paths, including the superseded vLLM CUDA worker-side path that moved 804,651,008 bytes. These rows may support same-subset semantics, but not zero-copy or prefix-cache reuse.	Exact source block/storage identities plus independent counters for K/V copy, host-device transfer, re-encoded tokens, transient attention bytes, and offload/restore traffic.
Cross-consumer comparator	Sparse-versus-packed rows that changed both K/V storage and the attention reduction implementation. The earlier HF 0.029785, MLX 1.0, and SGLang 1.109375 final-logit deltas confound selection with backend-specific rounding.	Alias-backed and packed-value paths retain the same logical segment boundaries and original positions and execute the same consumer. Cross-consumer drift remains a diagnostic, not a selection- correctness failure.

Table 12: Superseded agent-cache evidence. “Superseded” removes a systems claim, not the historical artifact: demoted rows remain available for debugging and causal comparison.

7.3 Gateway session and prefix protocol

The controlled gateway fixture executes five turns with changing selected records under four conditions: legacy message-zero G10 insertion, prefix-preserving G10, E0 message deltas, and an E1 logical message/resource-delta contract. It records whether each prior serialized message sequence remains an exact prefix, the reusable-message fraction, invalidations, JSON message and resource bytes, delta bytes, and session reuse. The workload is deterministic and uses a simulated adapter so that transport semantics can be isolated from model quality and network variance. Byte counts refer to the fixture’s JSON envelopes, not tokenizer output or K/V allocation. Physical cache hits remain unknown unless adapter telemetry supplies them.

7.4 Negotiated agent-transport protocol

A second deterministic fixture follows one task through open, tool-result, waiting, resume, and complete turns. Its detached stream contains a long document, an atomic tool schema, a skill record, versioned task state, and a large tool result. The conversational spine grows normally. We freeze the same selected record identities and compare complete TEXT rendering, PRA-FULL, and PRA-DELTA JSON requests. Reported bytes include OpenAI fields and PRA metadata; visible-token estimates

count message text only. Selection parity, task-status projection, and provider-native tool-schema preservation are hard assertions. The fixture does not call a language model, so it measures protocol semantics and retransmission rather than answer quality or TTFT.

7.5 Portable primitive protocol

The new benchmark uses PyTorch 2.12.1 with CUDA 12.6 on Windows 10, Python 3.10.11, and an NVIDIA GeForce GTX 950M with compute capability 5.0. Each synthetic tensor has four physical K/V heads, width 64, and 8,192 candidate tokens in float32. Selection admits 512 tokens (6.25%). Batch sizes are one and four. CPU rows use five warmups and 30 timed repetitions; CUDA rows use ten warmups and 50 repetitions. Every CUDA timing synchronizes before and after the operation.

Indexed gather uses evenly distributed selected token IDs. Interval packing uses eight intervals under the same 512-token budget. The hierarchy comparison holds intervals and output bytes fixed while moving sources among HBM, pinned CPU, and ordinary CPU. Compiler errors are artifact rows. The benchmark is a mechanism profile, not TTFT or throughput for a language model.

The layout sweep holds 8,192 total source tokens across four references and two layers. It packs 32-token pages in layer-major, reference-major, chunk-major, or block-major order, then restores an identical 512-token logical selection. Store construction is a cold cost; reported layout gather latency follows the same warmup and repetition protocol as the other device rows.

7.6 Inherited profiles

Two earlier artifacts answer adjacent questions. The Paper 2 public-API demo ran frozen Qwen3-0.6B over two QASPER and two HotpotQA examples. It records reference ingest, query encoding, routing, generation, K/V fractions, transfers, and peak GPU allocation. The Paper 4 lifecycle fixture ran eight deterministic resources through authenticated cold, warm, and hot states. Its answer proxy is required-document availability, not language quality. We do not pool these measurements with the new microbenchmark or compute cross-host speedups.

7.7 Execution-policy mechanism profile

We instantiate the same random-weight, three-layer tiny Llama configuration for five seeds and ingest two stable synthetic references. Every condition uses three active PRA layers, deterministic decoding, and at most three generated tokens. We compare request/shared at first and last routing layers, request/per-layer, token/shared at first and last routing layers, and token/per-layer. The profile records routing and selection epochs, temporal and layer Jaccard, selected-identity agreement, and generation wall time. It tests execution semantics and relative routing work, not language quality or production latency. EOS can terminate before three tokens, so token-level operation counts are averaged over the actual forwards.

7.8 RAG engine-mode protocol

Retrieval and selector quality belong to Papers 3.2 and 3.3. Here MultiHop-RAG is only a controlled engine workload: a fixed 50-question, seed-11 cohort and a tamper-evident receipt hold selected document intervals and token budgets fixed between realizations [8]. The engine comparison is Selected Context (visible text re-prefilled per request) versus Native Memory (the same selected text encoded as detached MLX K/V), each measured cold and warm. We report TTFT, total latency, throughput, ingestion time, visible and native tokens, retained K/V bytes, peak memory, and output parity. Selector and answer-quality artifacts remain available for audit but are not repeated here.

8 Results

8.1 Four-claim evidence map

Table 13 is the report’s claim ledger. Later mechanism and product results are evidence for one of these rows rather than additional independent claims.

Claim	Strongest current evidence	Present boundary
Correct live-history semantics	Same-resident-state controls qualify llama.cpp, vLLM-Metal, and the bounded vLLM CUDA path. Selected history is not re-encoded, and 100% is a semantic no-op.	Fresh-prefill/cache-reconstruction rows are diagnostics, not evidence of resident reuse.
Engine portability contract	vLLM-Metal passes 24/24 lifecycle checks; bounded vLLM CUDA adopts scheduler-owned pages with zero K/V copy, H2D transfer, or history re-encoding. HF, MLX, and SGLang now also pass their declared sparse consumer/lifecycle scopes.	HF remains an unfused batch-one Qwen path; the Metal kernel is limited to head dimension at most 256 and direct MLX batch one, while SGLang evaluates batched rows independently. CUDA still excludes partial pages, hybrid groups, and multiprocess execution.
Agent retention semantics	Task-03 restore-only replay recovers the efficient action in 3/3; task-04 changes whole-file rereading to targeted localization in 3/3. Typed causal policies preserve source and progress records.	Superiority to fixed token windows on official end-task outcomes is not yet established.
Coding-agent utility frontier	The Easy-3 No-PRA/PRA-100/PRA-90 matrix freezes success, calls, selected tokens, realized retention, K/V reuse, copy, and time; nine no-adaptor cells are configuration-ready.	No cells have executed; six learned-adaptor cells fail closed; 75% remains gated on three-task 90% success.

Table 13: Four-claim evidence map. Passing a mechanism gate does not silently promote task utility, portability, zero-copy behavior, or runtime benefit.

Four derived quantities remain deliberately separate:

$$G_{\text{context}} = C_{\text{full}}/C_{\text{selected}}, \quad G_{\text{session}} = T_{\text{selected needed}}/T_{\text{new}}, \quad (9)$$

$$G_{\text{native}} = C_{\text{selected}}/C_{\text{native}}, \quad Q_{\text{useful}} = (\text{requests/s})P(\text{quality pass}). \quad (10)$$

Here C must use one stated accounting boundary and T one stated materialized unit; the ratios are not interchangeable. In the scheduler-invisible CUDA vLLM fixture, scheduler-accounted tokens give $G_{\text{context}} = 276/36 = 7.67$ and $G_{\text{native}} = 36/20 = 1.80$; E2 carries no scheduler-visible source token, but the 20 query tokens remain. In the separate 50-turn agent trace, resource-body transmissions give $G_{\text{session}} = 275/21 = 13.10$. Because every CUDA row passes its quality gate, useful throughput equals 56.25 requests/s for E0, 45.46 for HOT E2, and 42.44 for WARM E2. Context and Native Gain therefore do not imply a throughput win.

8.2 RAG results: engine realization only

Papers 3.2 and 3.3 report retrieval, selection, evidence coverage, and answer quality. Their current policy keeps generic document selection unqualified. Paper 4.5 therefore asks only what the runtime does after a selection is frozen. On the 50-question 2K cohort, Selected Context and Native Memory

match every output and quality metric; detached MLX K/V introduces no additional quality change in this comparison.

Table 14: MLX-LM Qwen3-4B 4-bit realization metrics for 50 frozen 2K RAG selections on a 48 GiB M4 Pro. Times are milliseconds; cold includes native encoding, while warm Native Memory reuses retained immutable K/V.

Condition	Regime	TTFT p95	Total	Ingest	Visible	Native	Reuse
PRA_NATIVE_MEMORY_NO_ADAPTOR	COLD	2700.7	3120.6	2475.7	79	2026	0.000
PRA_NATIVE_MEMORY_NO_ADAPTOR	WARM	210.9	638.0	0.1	79	2026	1.000
PRA_SELECTED_CONTEXT_NO_ADAPTOR	COLD	2716.4	3024.3	2471.0	2105	0	0.000
PRA_SELECTED_CONTEXT_NO_ADAPTOR	WARM	2728.5	3030.7	2473.5	2105	0	0.000

Native Memory reduces mean visible prompt width from 2,105 to 79 tokens while carrying 2,026 selected native tokens. Cold native execution pays 96 ms more end-to-end than Selected Context (3.121 versus 3.024 s), a bounded 3.2% regression. Once the same selection is retained, mean TTFT falls from 2.640 s to 0.166 s and total latency from 3.031 s to 0.638 s; throughput rises from 10.56 to 50.21 tokens/s. Warm native active detail is 299 MB, while measured peak unified memory is 0.63 GB lower than the repeatedly prefilled Selected Context path. The native-memory and economic gates pass only for this repeated-selection regime; they do not override the selector qualification owned by Papers 3.2 and 3.3.

At the 4K boundary, contiguous selector-frozen realization remains exact. Cold Native Memory costs 6.313 s versus 6.109 s for Selected Context, a 3.3% regression. After retaining the exact selection, Native Memory costs 0.799 s versus 6.121 s, an 86.9% reduction, while moving 4,084 selected tokens out of the visible prompt. The cost is approximately 602 MB of active all-layer K/V. This is a repeated-selection benefit, not a comparison with a selected-text prefix-cache hit.

The MLX persistent-corpus mode changes the reuse unit from a complete selection to independently encoded 256-token chunks. Across 50 changing questions it reuses 22.0% of chunk lookups, materializes 80,117 unique native tokens rather than about 101,294 selected tokens, and reduces cumulative request wall time from 151.19 to 131.78 s (12.8%). Mean TTFT falls from 2.633 to 2.163 s. Retained detail reaches 6.38 GB, however, and generated behavior is not exact: official score changes from 0.440 to 0.380 while token F1 changes from 0.071 to 0.086. This is not a cache-corruption result. Each chunk is encoded independently and `combine_native_memories` preserves its source-local post-RoPE positions; concatenated Selected Context assigns one global sequence of positions. The experiment consequently measures a real chunk-resident representation policy, not transport parity. We mark the selection, native-parity, and economic decomposition gates `NOT_APPLICABLE_PERSISTENT_CORPUS`; position-aware composition remains required.

On the same M4 host, pinned MLX-LM Qwen3-8B 4-bit repeats the engine result: both selector-frozen text/native pairs are output-exact. Generic cold Native Memory costs 5.636 s versus 5.535 s, a 1.8% total-latency regression. Retaining the same native selection reduces warm total latency from 5.541 to 0.990 s (82.1%) and TTFT from 4.908 to 0.276 s. Active detail is approximately 285 MB. The 8B run therefore repeats the native-memory and repeated-selection result across model scale.

8.3 Stable prefix and session-delta contract

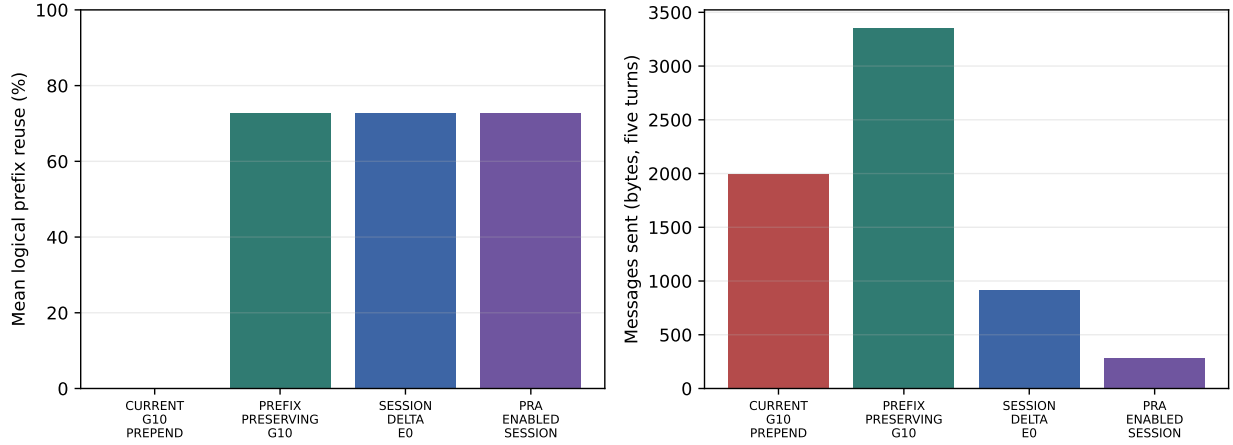


Figure 9: Five-turn deterministic gateway profile. The left panel reports logical reuse of the exact prior serialized message prefix after turn one; the right reports message bytes sent. Resource bytes are reported separately in Table 15.

Table 15: Controlled gateway session profile. Stable turns use the four turns after initial setup. Reuse is logical serialized-message reuse, not an engine-reported cache hit.

Condition	Stable turns	Reuse	Invalidations	Msg bytes	Resource bytes
CURRENT_G10_PREPEND	0/4	0.0%	4	1995	0
PREFIX_PRESERVING_G10	4/4	72.8%	0	3355	0
SESSION_DELTA_E0	4/4	72.8%	0	921	0
PRA_ENABLED_SESSION	4/4	72.8%	0	281	2297

The legacy message-zero G10 control changes the leading evidence block on every eligible turn, preserving zero of four prior prefixes and recording four invalidations. Moving evidence to the current turn preserves all four prior prefixes, with a mean logical reuse fraction of 72.8%. Full-history prefix-preserving G10 sends 3,355 message bytes because it still retransmits history. The session-delta E0 contract retains the same logical prefix while reducing message transport to 921 bytes; four turns reuse the prepared engine session.

The E1 contract sends 281 message bytes and 2,297 resource-envelope bytes, using resource additions, updates, and removals rather than repeating resource contents indiscriminately. Its combined delta bytes are not lower than the tiny text fixture because typed resource metadata and operation envelopes are explicit. This is a semantic and accounting result, not evidence that PRA transport is always smaller. All four conditions report physical prefix-cache hits as unknown. No vLLM/SGLang affinity, TTFT, throughput, or scheduler result is inferred from logical prefix stability.

Table 16: Five-seed repeated-session accounting. Turn one selects A/B/C; turn two selects B/D. Prefix-cache tokens, native reuse, and TTFT were not measured, so they remain explicit rather than being inferred from logical visibility.

Condition	Visible	New materialization	Logical reuse	Prefix cached	Native reused	Quality
full.context	72.0	72.0	0.0	NOT MEASURED	NOT MEASURED	1.0
selected.context.without.logical.reuse	88.0	28.0	0.0	NOT MEASURED	NOT MEASURED	1.0
session_aware.selected.context	72.0	12.0	16.0	NOT MEASURED	NOT MEASURED	1.0
native_memory	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT MEASURED	NOT MEASURED

Table 16 isolates the new ledger from retrieval and engine effects. Without logical reuse, Selected Context renders the selected B again and materializes 28 tokens across the two turns. Session-aware realization preserves B’s 16-token active occurrence and materializes only the 12 tokens of D, reducing total visible tokens from 88 to 72 while preserving the controlled quality check. Full Context also exposes 72 tokens in this fixture but counts all 72 as newly materialized. These are deterministic token-accounting results, not TTFT or attention-compute measurements; Native Memory remains NOT MEASURED in this cohort.

8.4 Typed transport avoids repeated record bodies

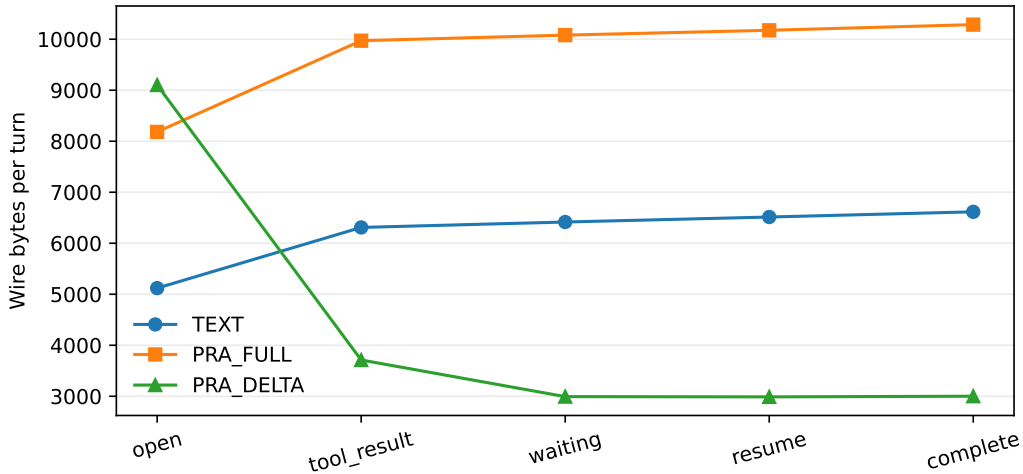


Figure 10: Per-turn JSON bytes for the five-turn task/tool workflow. PRA-FULL preserves explicit metadata but repeats all resource bodies; PRA-DELTA sends a body only for ADD or UPDATE and keeps unchanged resources logical.

Table 17: Negotiated transport contract over one deterministic long-running task. Byte totals are wire serialization, not model tokens or latency.

Mode	Total bytes	Resource bodies sent
TEXT	30,976	24 rendered copies
PRA-FULL	48,695	24
PRA-DELTA	21,795	8

PRA-FULL is larger than TEXT in this fixture because it carries explicit identity, views, provenance, task state, and authorization metadata in addition to complete bodies. PRA-DELTA reduces total wire

bytes by 29.6% relative to TEXT and 55.2% relative to PRA-FULL, while reducing body transmissions from 24 to 8. All five turns preserve selected identity, task status, and the provider-native tool schema. These are transport invariants, not evidence that logical PRA improves model quality. A matched model benchmark must hold selection, model, positions, profile, and generation fixed.

We therefore repeated the comparison with a real MLX-LM Qwen3-0.6B model on ten held-out QASPER selections. Each condition used the same chat template, selected records, task state, provider-native tool schema, greedy decoder, and 24-token budget. The gold answer remained in an out-of-band evaluator rather than request metadata. TEXT, PRA-FULL, and PRA-DELTA produced byte-identical model messages and identical outputs in 10/10 examples; their mean gold-answer log probability was also identical at -40.35625 . Mean TTFT was 507, 506, and 481 ms, respectively, but ten independent first turns do not support a latency-ordering claim. Their mean wire sizes were 5113, 9475, and 10919 bytes; first-turn typed metadata has a cost before delta reuse begins.

The long-session fixture makes that reuse visible. At 50 turns, including one forced reconnect and full resynchronization, TEXT sends 638,552 bytes, PRA-FULL sends 840,632, and PRA-DELTA sends 162,253. Delta transport is therefore 74.6% smaller than repeated TEXT and 80.7% smaller than PRA-FULL. It sends 21 resource bodies rather than 275 while preserving selection, task state, and tool schema.

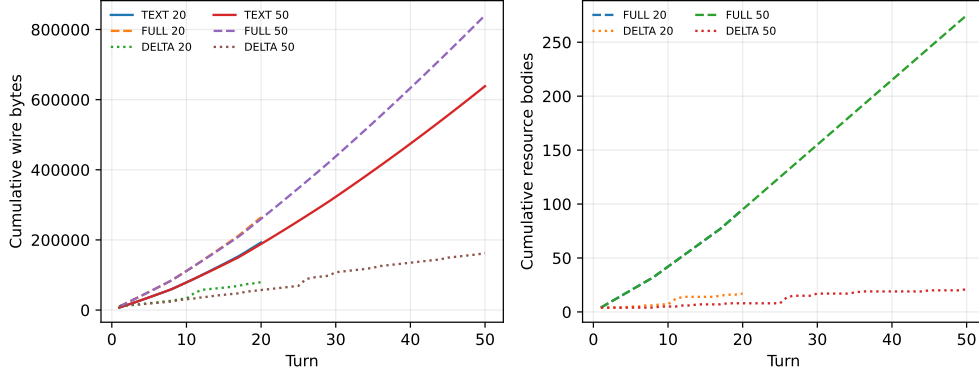


Figure 11: Cumulative wire bytes over 20- and 50-turn workflows. The reconnect forces one resource resynchronization; unchanged resources remain body-free on the other delta turns.

8.5 Selection sharing removes repeated routing work

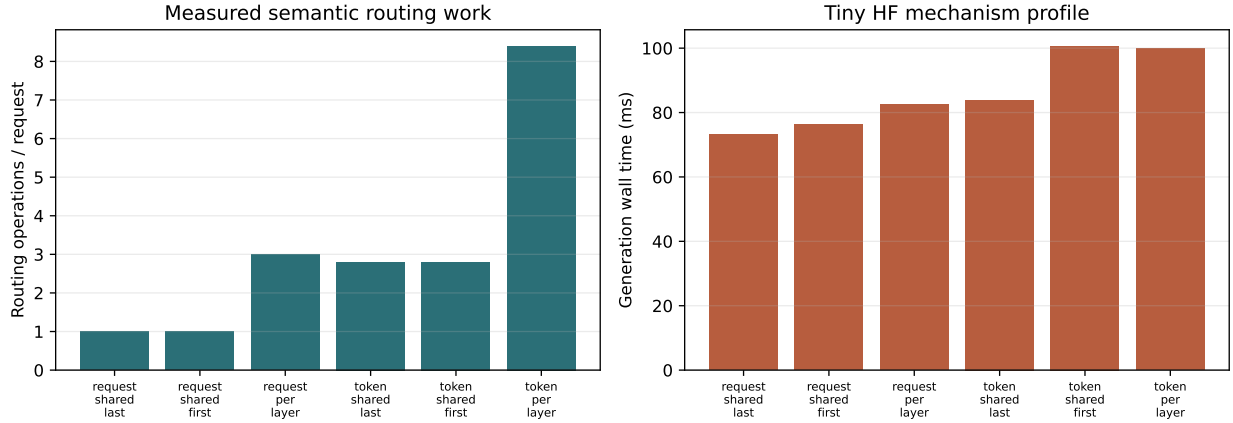


Figure 12: Five-seed tiny-HF mechanism profile. Routing operations validate the execution contract; wall time is host- and model-specific and is not a serving speed claim.

Table 18: Implemented policy points in the five-seed mechanism profile. Times are means over random-weight tiny models and do not measure answer quality.

Condition	Routing operations	Generation ms
Request/shared, last	1.0	73.1
Request/per-layer	3.0	82.6
Token/shared, first	2.8	100.7
Token/per-layer	8.4	100.0

Request/per-layer performs exactly three routing decisions, but amortizes them over generation. Token/shared and token/per-layer observe the same average number of autoregressive forwards; sharing reduces routing work by 3.0 $\times$, from 8.4 to 2.8 operations. Tiny-model wall times are effectively tied for the two token modes, which means Python and model overhead dominate at this scale. The routing-layer sweep also changed selected identities, confirming that routing layer is a semantic parameter rather than a scheduling-only knob. The stable selections within these short completions yield temporal Jaccard one; longer trained-model studies are needed before making a churn or quality claim.

8.6 Portable eager primitives

Table 19: Median warm latency in milliseconds. All measured outputs match the eager reference exactly. Each output contains 512 selected tokens per request.

Device	Primitive	Batch 1	Batch 4
CPU	indexed gather	0.133	0.835
CPU	interval pack	0.228	0.938
CUDA	indexed gather	0.253	0.478
CUDA	interval pack	0.415	0.613

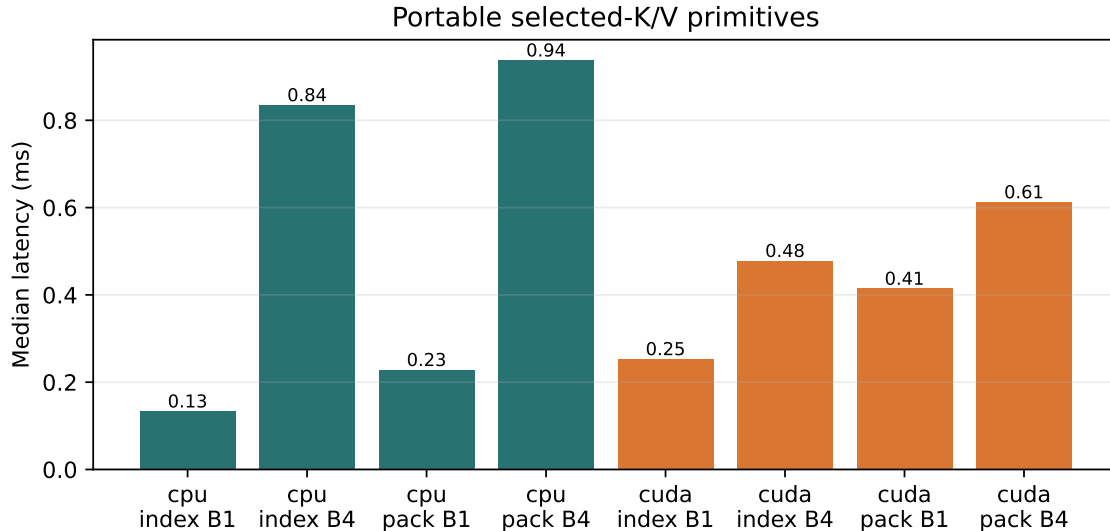


Figure 13: Portable primitive latency. This older GPU does not make small batch-one gathers faster than the CPU, but CUDA scales better at batch four.

The indexed CUDA path is 0.253 ms at batch one. The eight-interval pack is 0.415 ms because it launches and concatenates multiple slices. At batch four, CUDA indexed throughput increases to roughly 4.28 million selected tokens/s, whereas the CPU reaches about 2.45 million million. These values identify packing and fragmentation as optimization targets; they do not include routing or attention.

8.7 Residency dominates the selected payload

Table 20: Median CUDA interval materialization latency for an identical selected payload. Output size is 1 MiB at batch one and 4 MiB at batch four.

Source residency	Batch 1 (ms)	Batch 4 (ms)
HBM resident	0.424	3.371
Pinned CPU → CUDA	1.592	6.655
Ordinary CPU → CUDA	2.157	8.660

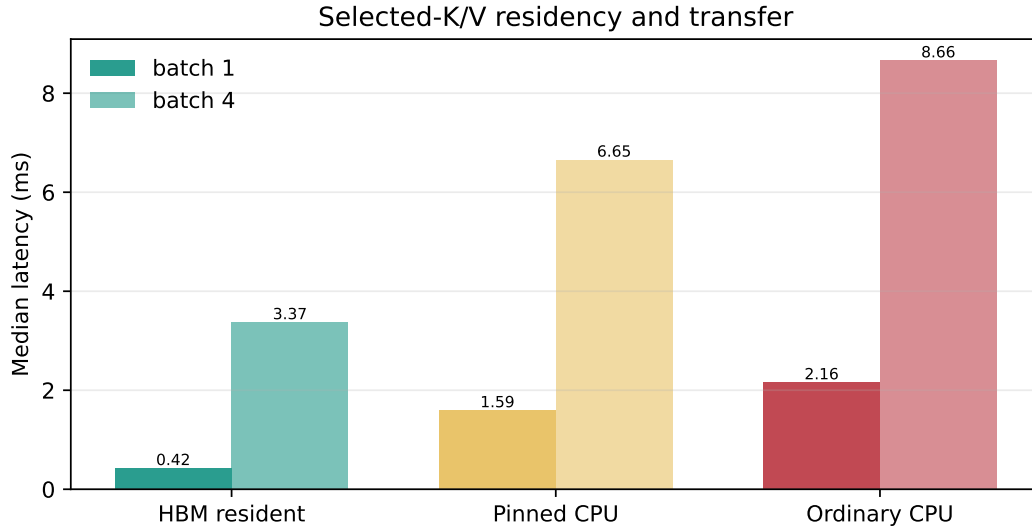


Figure 14: The same logical selection has different physical cost depending on residency. Pinned transfer helps, but does not erase the hierarchy boundary.

At batch one, pinned transfer is 3.75 times the HBM-resident cost, and ordinary CPU transfer is 5.08 times that cost. This is the central result of the initial systems gate. A configuration that moves inactive K/V off accelerator reduces residency, but request latency then depends on locality, cache reuse, and transfer overlap. Any future headline must report both memory and latency.

8.8 Physical layout is secondary to fragmented remapping

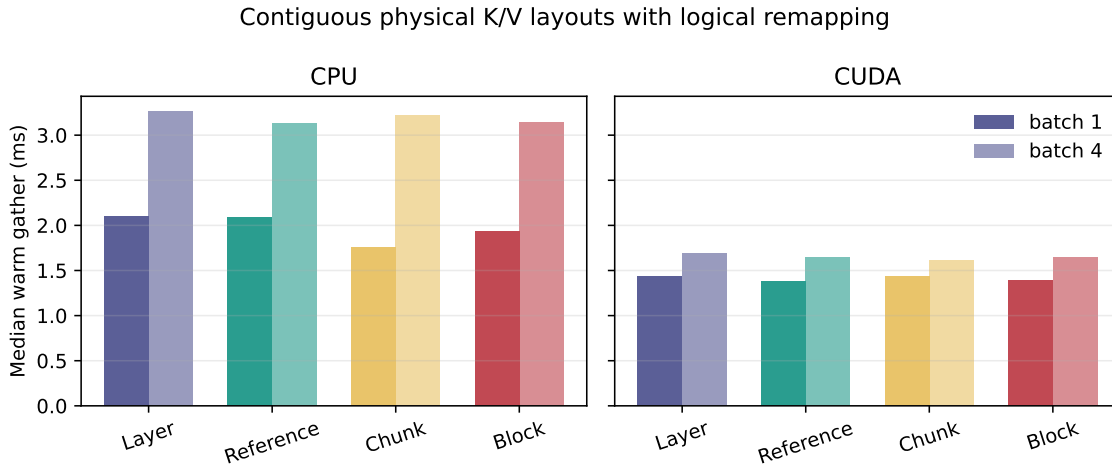


Figure 15: Warm materialization from four physically distinct contiguous stores. Every layout restores the same logical per-layer K/V before attention.

Layer-major, reference-major, chunk-major, and block-major stores place the same four-reference, two-layer native K/V in different contiguous orders. A compact placement index maps URI, layer, and logical page offsets back to physical ranges. All outputs match the unpacked reference exactly. On CUDA, the slowest and fastest medians differ by only 3.8% at batch one and 5.0% at batch four. The

nominal fastest layouts are reference-major and chunk-major, respectively, but those small differences do not justify a universal preference on this synthetic cohort.

More importantly, all layout-remap paths take roughly 1.4–1.7 ms on CUDA, several times the single-source interval pack. They reconstruct selected logical ranges across Python placement records and page fragments. The next optimization target is therefore indexed vectorized remapping and fewer launches, not a claim that one physical ordering has won.

8.9 Compilation is a negative capability gate

The `torch.compile` API is present, but neither local compiled path is valid. CPU Inductor cannot find the Microsoft C/C++ compiler. CUDA compilation rejects compute capability 5.0 because the Triton backend requires capability 7.0 or newer. No warm compiled latency is reported and no eager result is relabeled as compiled. Triton and custom CUDA therefore remain untested.

8.10 Earlier profiles locate larger costs

The new primitive measurements explain the cost of moving an already selected payload. Earlier end-to-end artifacts provide the complementary scale: how much time was spent before and after materialization when the public model API and authenticated resource lifecycle were exercised.

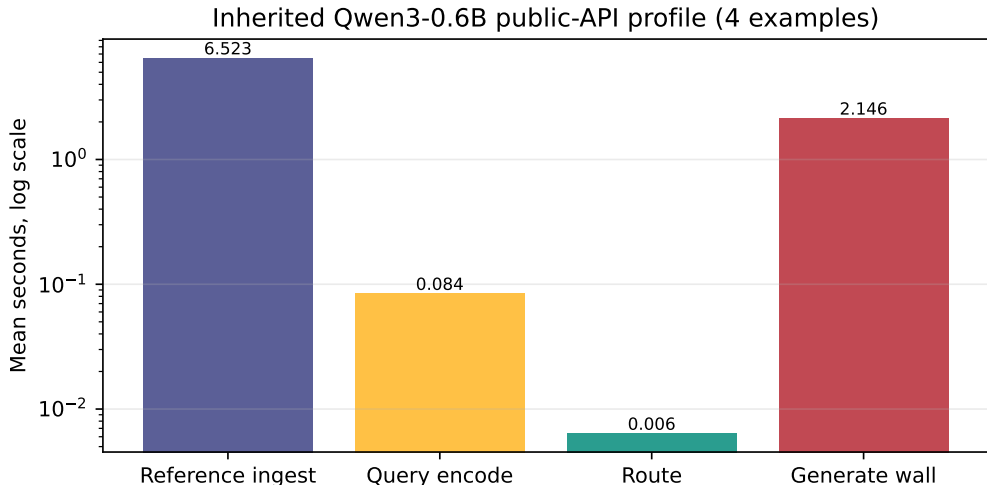


Figure 16: Inherited Paper 2 Qwen3-0.6B profile over four examples. The log scale shows that routing time is small relative to native reference encoding and generation on that protocol.

The inherited profile reports 6.40 ms mean routing time, 6.52 s mean reference ingest, and 2.15 s mean routed generation wall time. Mean evidence recall is 0.667 and materialized detail K/V is 0.066 of the candidate total. This supports optimizing cache reuse and encoding amortization before replacing the router with a custom kernel. The cohort contains only four examples, so it does not establish a general workload distribution.

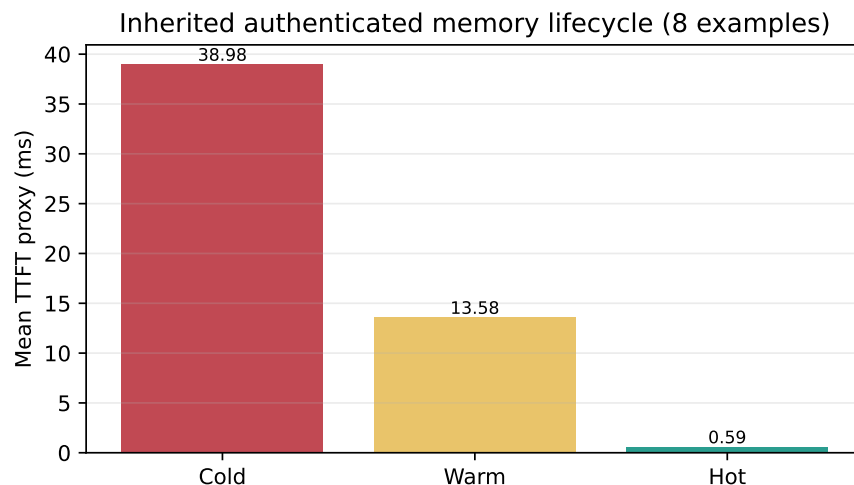


Figure 17: Inherited Paper 4 lifecycle proxy over eight examples. Cold includes fetch and native encoding; warm reuses native encoding; hot reuses selected K/V.

The lifecycle TTFT proxy falls from 38.98 ms cold to 13.58 ms warm and 0.59 ms hot. Required-document recall is 0.875. The result validates state transitions and shows the size of the reuse opportunity, but it is not an LM answer-quality or production-serving measurement.

8.11 Semantic storage lifecycle

A 120-turn mixed trace compares plain LRU, size-aware LRU, and weighted task/type-aware retention at the same 80 KB WARM quota. Weighted retention raises the utility-weighted hit rate from 0.700 to 0.796, preserves every long-running task and shared-document access, and reduces SOURCE reconstructions from 214 to 184. Its estimated context-resolution cost falls from 1,170 to 936 ms. These are policy-trace results; the extra Python scoring work is not presented as engine serving speed.

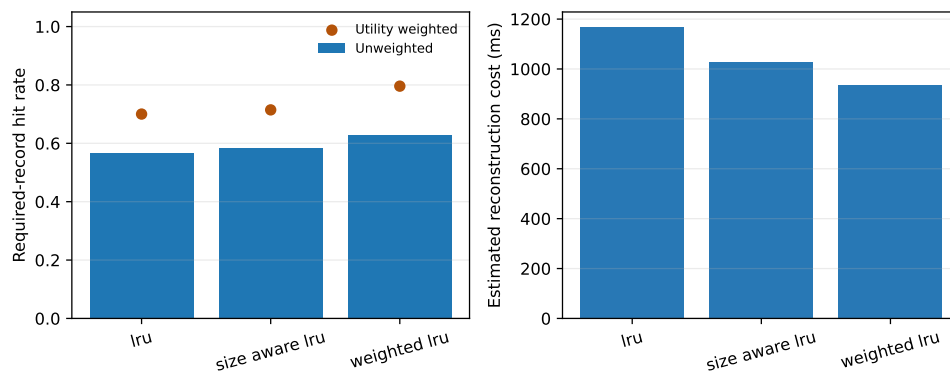


Figure 18: Equal-quota retention on a mixed task, document, and tool-result trace. Weighted policy protects active task state and shared documents while allowing reconstructable transient records to churn.

The runtime now makes the cache hierarchy a shared semantic contract rather than an engine-specific accident. Each typed record has an authoritative SOURCE representation and may have derived native detail in COLD, WARM, or HOT form. HOT means attention-ready and request-pinnable; WARM means lossless native detail suitable for fast promotion; COLD means capacity-

oriented persistence with lossless compression and optional, separately named K/V quantization. The physical HOT object remains an engine concern—vLLM pages, an independent SGLang object, or MLX arrays—but record type, task status, dependencies, reconstructability, grace periods, and quotas are decided once by `PRASStorageManager`.

The deterministic policy retains plain LRU, size-aware LRU, reuse-count, and reload-cost controls, and adds weighted LRU. Its score combines recency, frequency, reconstruction cost, typed-record prior, task state, authorized sharing, dependency count, byte cost, and reconstructability. Open, blocked, or waiting tasks enforce minimum WARM lifetimes. Completion schedules delayed compaction rather than deletion; an open dependent task cancels aggressive compaction. Session close releases transient native detail without treating a native-cache quota as permission to delete SOURCE. Newly encoded detail first observes a grace period, avoiding speculative persistence of short-lived tool and terminal records.

Table 21: Shared lifecycle controls over HF, vLLM, SGLang, and MLX labels with five seeds each (20 runs). Rates are policy outcomes under a fixed synthetic pressure trace, not end-task model quality.

Control	Baseline	Semantic lifecycle
Shared document retained under pressure	0	1
Open-task record retained	—	1
Dependent completed-task record retained	—	1
Transient persistence writes	20	0
COLD native-byte ratio	1.00 FP32	0.25 int8

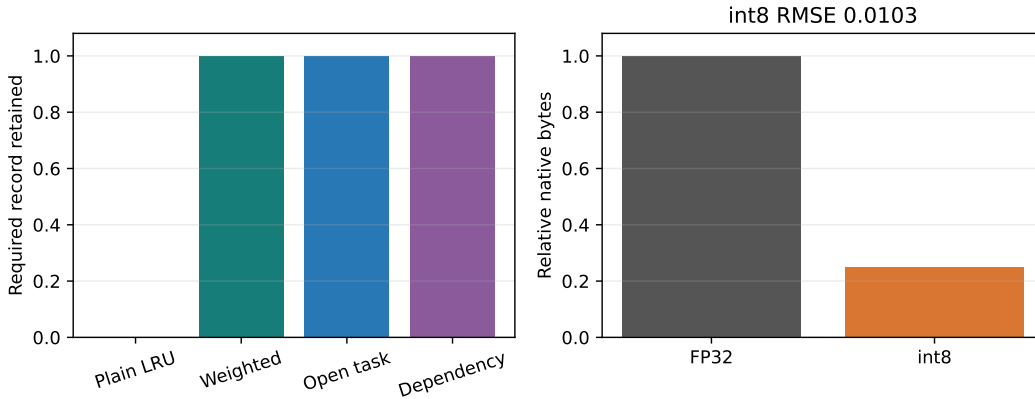


Figure 19: Engine-neutral retention controls and generic symmetric-int8 byte transport. The int8 result is a codec diagnostic; model-specific quality and active-attention bytes remain separate gates.

Lossless HOT→WARM→COLD→HOT recovery and SOURCE reconstruction are exact in 20/20 runs. Under a 24-byte pressure fixture, plain LRU loses the shared document in every run whereas the weighted policy retains it in 20/20; active-task and downstream-dependency protections also hold in 20/20. A persistence grace period reduces writes for the transient closure trace from 20 to zero. Generic symmetric int8 occupies 0.25 of FP32 bytes with 0.0103 RMSE, 0.28 ms median quantization, and 0.07 ms median dequantization on this host. This does not supersede the MLX model-specific int8 quality result, nor show that WARM and COLD improve generation latency. It establishes policy parity, exact lossless recovery, and a measurable reason to defer transient writes; natural lifecycle distributions remain open, while the following small probe begins engine-native promotion measurement.

The next probe replaces the in-memory HOT bridge with live attention objects: reserved vLLM-

Metal pages, SGLang MLX arrays backed by the built-in HiCache file API, and MLX arrays. Each of three natural QASPER selections is generated from HOT, demoted to lossless WARM, promoted and generated again, moved through int8 COLD, and finally recovered by a newly constructed lifecycle manager. MLX-format WARM storage is layer segmented: each K and V array has an atomic manifest entry and checksum, and a consumer profile can mmap only requested layers. HOT promotion retains the lower durable copy, so a process failure during an active request does not erase the recoverable representation.

Table 22: Three-example live-engine storage mechanism probe. Ratios include request-time promotion and generation but exclude background demotion and COLD conversion. Prefix is the mean common HOT/int8 output prefix in tokens.

Engine	WARM exact	int8 exact	int8 first	Prefix	WARM/HOT	COLD/HOT
mlx-lm	100%	0%	67%	1.3	1.97	1.84
sglang-mlx	100%	0%	67%	0.7	1.90	1.89
vllm-metal	100%	0%	67%	4.0	1.96	2.00

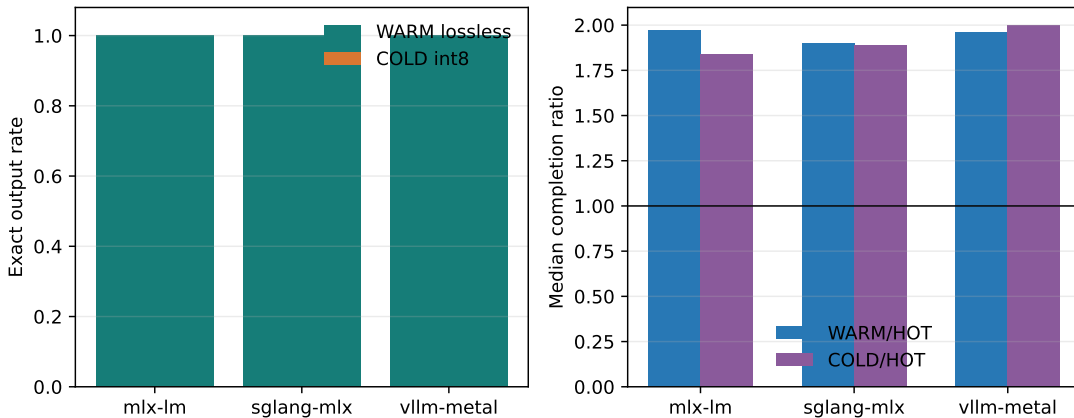


Figure 20: Lossless WARM preserves all three outputs per engine, whereas int8 COLD changes every 24-token sequence. The latency panel measures a tier hit at request time; background WARM and COLD transitions are reported separately.

WARM output is exact in 9/9 engine-example pairs, and restart promotion succeeds for every engine. Median WARM/HOT request ratios are 1.97 for MLX, 1.90 for SGLang, and 1.96 for vLLM; the corresponding COLD/HOT ratios are 1.84, 1.89, and 2.00. Median background WARM transitions cost 348, 158, and 362 ms, while COLD conversion costs 940, 701, and 1,016 ms. These are small mechanism cohorts, not tail-latency estimates. Int8 COLD is 0/9 sequence-exact, with first-token parity in 6/9 and short mean common prefixes. The signed F1 changes are noisy on three low-accuracy examples and do not rescue sequence equivalence. Named product profiles therefore keep lossless COLD as the default; int8 remains explicit, model-calibrated, and disabled unless its own quality gate closes.

We then scale the same live protocol to 20 examples per cohort: QASPER, HotpotQA, and 2Wiki-MultiHopQA at Qwen3-0.6B, plus QASPER at Qwen3-1.7B, for each engine. Five smaller QASPER cohorts add Llama-3.2-1B to all three engines and Gemma-3-1B to vLLM-Metal and MLX-LM. Lossless WARM reproduces HOT generation in 265/265 pairs and all 17 managers recover after restart. Diagnostic int8 COLD is exact in only 61/265 pairs. The pinned SGLang-MLX build rejects Gemma’s

mixed global/sliding-window topology before PRA attachment; this is a backend compatibility boundary, not a PRA parity failure. Table 23 keeps model and dataset identities separate; its percentiles describe finite cohorts rather than an online arrival distribution.

Table 23: Expanded live lifecycle matrix. Latencies include request-time promotion and generation; background demotion/conversion remains separate.

Engine	Model	Data	n	WARM exact	int8 exact	HOT p50	WARM p50	WARM p95
mlx-lm	Llama-3.2-1B-Instruct-4bit	qasper	5	100.0%	40.0%	191	275	281
mlx-lm	Qwen3-0.6B-4bit	2wikimultihopqa	20	100.0%	15.0%	328	658	710
mlx-lm	Qwen3-0.6B-4bit	hotpotqa	20	100.0%	15.0%	329	707	817
mlx-lm	Qwen3-0.6B-4bit	qasper	20	100.0%	15.0%	326	672	714
mlx-lm	Qwen3-1.7B-4bit	qasper	20	100.0%	20.0%	337	723	744
mlx-lm	gemma-3-1b-it-4bit	qasper	5	100.0%	100.0%	391	480	502
sglang-mlx	Llama-3.2-1B-Instruct-4bit	qasper	5	100.0%	40.0%	242	287	302
sglang-mlx	Qwen3-0.6B-4bit	2wikimultihopqa	20	100.0%	15.0%	310	525	565
sglang-mlx	Qwen3-0.6B-4bit	hotpotqa	20	100.0%	20.0%	320	543	592
sglang-mlx	Qwen3-0.6B-4bit	qasper	20	100.0%	15.0%	307	538	604
sglang-mlx	Qwen3-1.7B-4bit	qasper	20	100.0%	20.0%	374	614	632
vllm-metal	Llama-3.2-1B-Instruct-4bit	qasper	5	100.0%	60.0%	299	428	453
vllm-metal	Qwen3-0.6B-4bit	2wikimultihopqa	20	100.0%	30.0%	351	738	803
vllm-metal	Qwen3-0.6B-4bit	hotpotqa	20	100.0%	35.0%	366	737	832
vllm-metal	Qwen3-0.6B-4bit	qasper	20	100.0%	10.0%	360	714	779
vllm-metal	Qwen3-1.7B-4bit	qasper	20	100.0%	10.0%	383	769	824
vllm-metal	gemma-3-1b-it-4bit	qasper	5	100.0%	100.0%	457	585	623

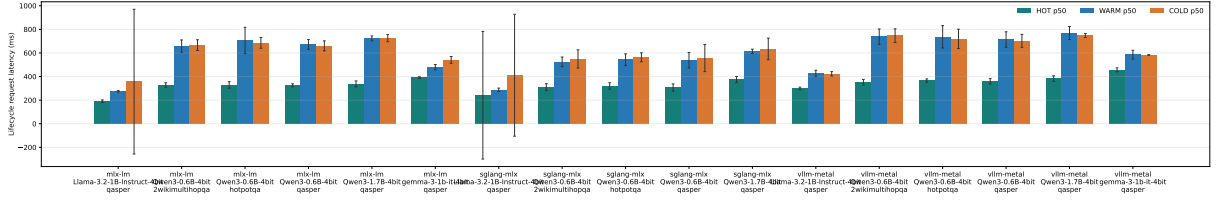


Figure 21: HOT, WARM, and COLD request p50 with p95–p50 upper whiskers. WARM recovery is reliable, but synchronous promotion remains visibly expensive.

Across cohorts, WARM/HOT p50 ratios range from 1.64 to 2.15. This changes the product interpretation: persistent native K/V is now a correctness-qualified backing representation, but request-path promotion is not yet economically qualified. Prefetch, overlap, and hot-set admission must remove that stall; quantized COLD additionally requires a model-and-workload quality gate.

The runtime now implements that next step without giving model-array mutation to background Python threads. An event-loop-owned coordinator deduplicates in-flight promotions by logical key, admits completed objects to a bounded hot set, and overlaps WARM reads with query-side work. Across five QASPER objects, MLX requires a 500 ms lead for 5/5 ready-at-demand and a 0.983 median demand/HOT ratio; SGLang reaches 5/5 by 350 ms and a 1.063 ratio. With no lead, the corresponding ratios are 2.123 and 1.561. All 70 paired outputs are exact. This demonstrates latency hiding under oracle lead, not a faster storage codec or a calibrated production predictor.

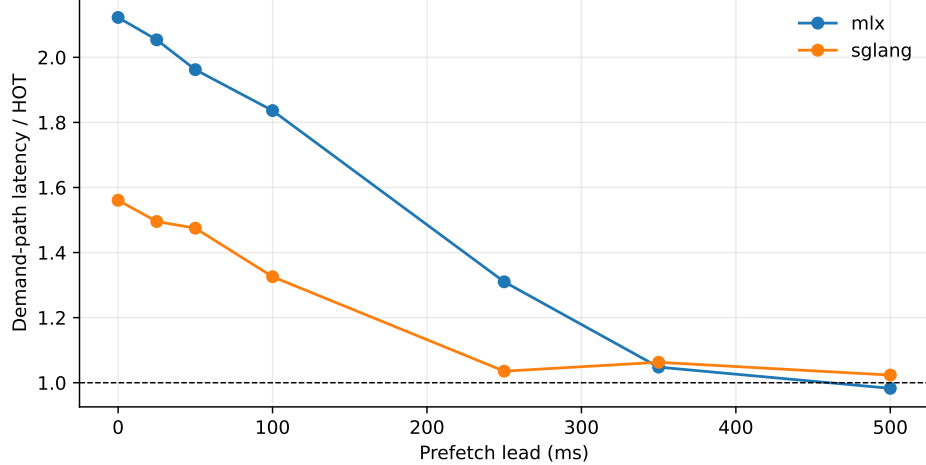


Figure 22: Scheduler-owned WARM promotion can hide request-path stall when useful lead is available. The dashed line is HOT latency.

A two-host SGLang storage control then separates transfer lead from placement. Five 125-token native objects reside on one 48-GiB Mac and are requested by a model-side coordinator on a second Mac. A 2,000-ms lead makes all 40 requests ready at demand; shorter leads leave a measurable remote-transfer stall. At concurrency 16 and 250-ms lead, affinity placement reduces shared-resource reads from 57.4 to 14.4 MB and raises control-plane throughput from 12.24 to 28.38 requests/s. For mixed resources it reduces reads from 272.7 to 71.8 MB and raises throughput from 2.25 to 11.77 requests/s. Tensor equality is exact. This is a controlled native-K/V storage result over tunneled HTTP, not yet an online SGLang generation or distributed-HiCache latency claim.

The same native executors now traverse the HTTP streaming gateway. SGLang delivers 3.04–3.20 requests/s and MLX 1.56–1.66 requests/s over concurrency one through eight. Tail latency grows with the intentionally serialized model runners; cancellation after first token and cleanup succeed, with no retained request-owned native state across 36 exercised sessions. The model locks are a correctness boundary: an audit found that MLX cache captures were views into reusable arrays. Owning evaluated K/V arrays at the capture boundary restores HOT and WARM isolation and prevents later requests from mutating retained memory.

A separate MLX finite-wave curve extends physical-tier concurrency to 16. Every one of 124 HOT/WARM requests is baseline-exact. At concurrency 16, a shared WARM resource requires one promotion, sustains 2.91 requests/s, and avoids 630 MiB of duplicate K/V; 16 independent WARM resources require 16 promotions, read 1.26 GiB, and sustain 1.22 requests/s. This quantifies the benefit of immutable physical sharing while also exposing the queue cost of an unbatched, serialized runner.

Long-session pressure adds 1,125 requests across QASPER, HotpotQA, and 2Wiki. With an eight-resource cyclic working set, HOT capacities two and four average 17 reloads per seed and reload every final revisit; capacity eight has zero reloads and evictions. Quality is invariant because selection is frozen. A 20-example QASPER selective-int8 study reaches only 4/20 exact generations when all K/V is quantized; restricting quantization to the late quarter reaches 14/20, still below the lossless product gate despite small aggregate F1 change.

A further 1,000-request QASPER matrix crosses HOT/WARM resource budgets $\{2, 8\}$ with local rotating windows $\{64, 256\}$. The small WARM budget produces 24–75% SOURCE starts, whereas WARM eight eliminates them. F1 is identical in all eight cells and window size has no material completion-tail effect. Full disk-backed WARM does not reduce resolve p95 relative to re-encoding these bounded sources, making reload-cost-aware admission a runtime requirement rather than an

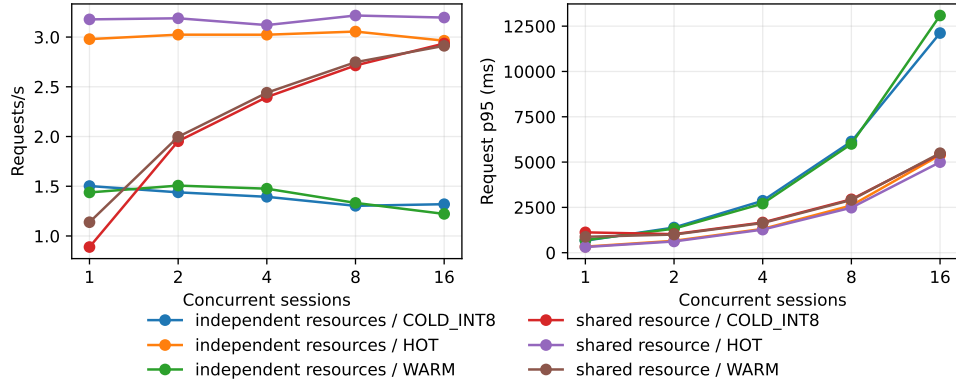


Figure 23: Shared and independent MLX resources through 16 concurrent sessions. The finite waves include promotion and serialized model queueing.

optional optimization.

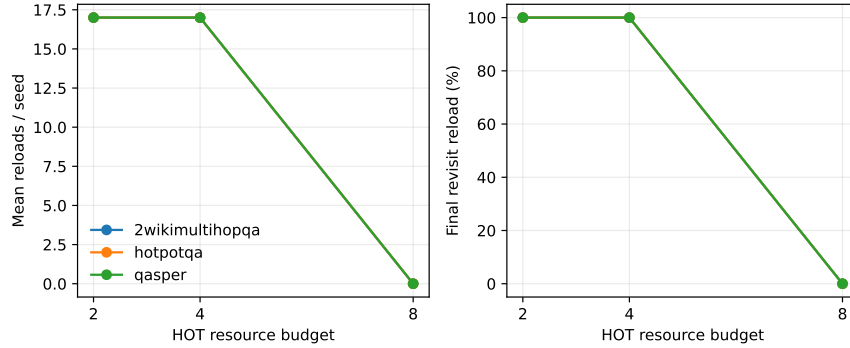


Figure 24: Sustained pressure exposes a working-set threshold rather than a quality effect: once all eight resources fit, reloads disappear.

8.12 Interpretation

The results separate three claims that are often collapsed. Selection can improve evidence availability and reduce visible context before any engine change. Native Memory can preserve the same selected evidence while moving it outside prompt accounting. Native Serving becomes economically useful only when the engine owns attachment, placement, reuse, and cleanup cheaply enough to offset its control-path cost. Exactness and token reduction close broadly; the economic gate remains engine- and workload-dependent.

9 Cross-Model, Precision, and Product Qualification

9.1 Common Qualification Contract

The qualification contract asks four separate attribution questions. First, No PRA versus Selected Context measures PRA routing and visible-context construction. Second, Selected Context versus Native Memory holds selected record IDs and intervals fixed and isolates native-K/V realization. Third,

Native Memory versus Native Serving holds selection and representation fixed and measures scheduler-owned placement, reuse, and lifecycle. Fourth, no-adaptor versus bundle comparisons hold execution mode fixed and isolate the exact model-specific bundle. We use stable condition identifiers for all seven mode/adaptor combinations. In particular, E0 is Selected Context and is already a PRA condition.

The available flagship engine cohorts primarily answer the second question. True matched No-PRA arms for Qwen3-4B, Qwen3-8B, Qwen3-14B, and Llama-3.2-1B remain **NEEDS_RUN**; exact bundle-versus-no-adaptor end-task arms are also absent. We therefore do not attribute the large E0/E2 visible-token difference to PRA versus ordinary inference. It is only the effect of realizing the same PRA-selected evidence as native memory.

All engine rows now answer one question: at fixed or acceptably changed quality, when does PRA reduce total serving cost? The benchmark computes each query, candidate set, selected IDs, and selected intervals once. A cryptographic selection digest is then reused by **FULL**, **E0_SELECTED**, **E2_HOT**, and **E2_WARM**; optional E3 conditions add prefetch, remote WARM, COLD, or SOURCE restoration without rerunning retrieval. Context size, shared versus independent resources, and concurrency 1, 4, 8, and 16 are manifest dimensions. This separates retrieval quality from representation and transport.

Every row pairs task quality with serving measurements. Binary workloads derive successful requests per second as request rate times task success and report successful tasks per accelerator-hour. Monetary cost remains null unless an auditable hourly-cost source is supplied. Visible-token reduction and active-K/V reduction remain separate. Tail percentiles, physical memory components, transfer bytes and time, cache events, reuse, hardware metadata, status, and provenance are first-class fields. The current legacy registry has no row with both request rate and task success, so its quality-adjusted-throughput field is honestly **NOT_MEASURED** rather than reconstructed after the fact.

Figure 25 distinguishes a validated level from a deeper observed mechanism. In particular, AirLLM has a live E2 mechanism smoke but remains E0-qualified until natural quality and larger-than-VRAM evidence close the gate; SGLang has local E3-like HiCache controls but lacks distributed, scheduler-owned E3 validation. Raw throughput is deliberately absent from this maturity axis.

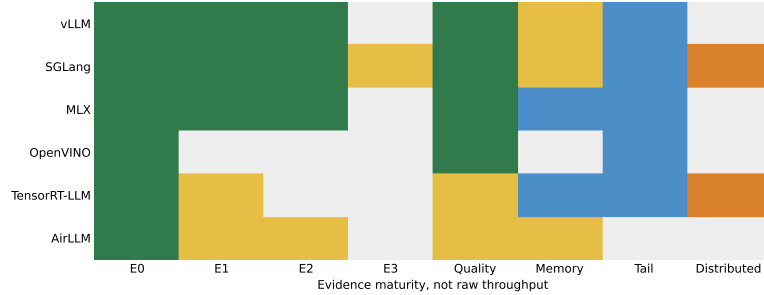


Figure 25: Cross-engine evidence maturity. Darker cells indicate stronger evidence; candidate mechanisms do not automatically promote validated level.

Table 24: Evidence-driven engine frontier. “Observed” may exceed “validated” when a mechanism exists but the corresponding quality, pressure, or scheduler gate remains open. Status words describe evidence, not numeric values.

Engine	Validated	Observed	Quality	Memory	Tail	Distributed
vLLM	E2	E2	natural	controlled	measured	not measured
SGLang	E2	E3	natural	controlled	model	candidate
MLX	E2	E2	natural	model	model	n/a
OpenVINO	E0	E0	natural	not measured	measured	n/a
TensorRT-LLM	E0	E1	controlled	measured	measured	candidate
AirLLM	E0	E2	controlled	controlled	not measured	n/a

Table 25: Implementation status on the measured branch and host.

Runtime	Status	Boundary
HF/PyTorch eager	measured	Native PRA model API and portable K/V primitives
Standalone gateway	contract tested	G00/G10/G01/G11, session lifecycle, prefix-preserving fallback, deltas, debug/close API, and HF-backed SSE
OpenAI-compatible HTTP	E0 implemented	Full-history safe default; typed engine profiles and explicit text fallback
DeepSeek/Pi agent bridges	contract tested	Typed event/RPC capture and explicit G10 fallback
<code>torch.compile</code>	blocked on host	API implemented; compiler/GPU gates recorded
Triton/custom CUDA	not run	Requires supported GPU and profiling evidence
vLLM	provider E0; companion E2	Default provider remains conservative; Paper 6 measures native APC and request isolation through concurrency eight
SGLang	provider E2	Companion Paper 6.1 measures native runner integration; local Radix-prefix and external-HiCache composition is controlled
MLX	provider E2	Companion Paper 6.2 measures native layer profiles, persistence, and concurrency; Paper 4.5 remains HF-centered
TensorRT-LLM	provider E0; native architectural	Companion Paper 6.4 measures CUDA selected-text load through concurrency 16; native attachment remains open
OpenVINO GenAI	provider E0	Companion Paper 6.3 measures CPU/GPU generation and continuous batching; the public native non-prefix attachment seam is blocked
FreeToken	E0/architectural	Remote fallback or planned provider; no native Paper 4.5 result

Table 26: Measured companion native-mechanism matrix. These rows calibrate engine realizations; they do not promote Paper 4.5’s HF-centered profile evidence.

Engine/model	Hardware	Profile	Evidence tier	Status
vLLM-Metal V1 / Qwen3-0.6B	Apple M5 / 16 GB / Metal 4	all-layer native K/V + APC, concurrency 1–8	natural qa matched selection	840/840 exact at 0.6B/1.7B; lossless WARM 80/80; int8 COLD open
SGLang MLX / Qwen3-0.6B	Apple M5 / 16 GB / Metal 4	Radix + selected K/V + built-in HiCache file storage	natural qa matched selection	840/840 exact; lossless WARM 80/80; distributed HiCache open
MLX-LM / Qwen3-0.6B	Apple M5 / 16 GB / Metal 4	all-layer selected K/V, four matched regimes	natural qa matched selection	840/840 exact; lossless WARM 80/80; int8 COLD open
MLX-LM / Qwen3-1.7B	Apple M5 / 16 GB / Metal 4	FP/int8 all-layer selected K/V	natural qa routed evidence materialization	routed natural QA; bounded residency curve; oracle gap remains
MLX-LM / Llama-3.1-8B	Mac16,7 / 48 GB / Metal	all-layer native concat; reduced profiles calibration pending	natural qa matched selection	60/60 exact; warm ratio 1.032; cold ratio 0.935
vLLM-Metal V1 / Llama-3.2-1B	Apple M5 / 16 GB / Metal 4	all-layer native K/V + APC, four matched regimes	natural qa matched selection	840/840 exact; lossless WARM 5/5; int8 COLD 3/5 (smoke; gated)
vLLM-Metal V1 / Gemma-3-1B	Apple M5 / 16 GB / Metal 4	global/local native K/V + APC, four matched regimes	natural qa matched selection	840/840 exact; lossless WARM 5/5; int8 COLD 5/5 (smoke; gated)
SGLang MLX / Llama-3.2-1B	Apple M5 / 16 GB / Metal 4	selected K/V + lifecycle manager	natural qa lifecycle smoke	lossless WARM 5/5; int8 COLD 2/5 (smoke; gated)
SGLang MLX / Gemma-3-1B	Apple M5 / 16 GB / Metal 4	mixed global/sliding-window topology	backend compatibility	blocked before PRA: per-layer window map unsupported
MLX-LM / Llama-3.2-1B	Apple M5 / 16 GB / Metal 4	all-layer selected K/V + lifecycle manager	natural qa lifecycle smoke	lossless WARM 5/5; int8 COLD 2/5 (smoke; gated)
MLX-LM / Gemma-3-1B	Apple M5 / 16 GB / Metal 4	global/local selected K/V + lifecycle manager	natural qa lifecycle smoke	lossless WARM 5/5; int8 COLD 5/5 (smoke; gated)
vLLM CUDA V1 / TinyLlama-1.1B-Chat-v1.0	NVIDIA RTX 5060 Laptop / 8 GB / WSL2	scheduler-invisible detached E2 HOT/WARM, concurrency 8	cuda matched detached pages	128/128 exact per condition; E2-HOT 45.5 vs E0 56.3 useful req/s; LMCACHE open

Table 27: Bounded E0 serving matrix. These selected-text baselines do not imply native non-prefix K/V support.

Engine/model	Hardware	Profile	Evidence tier	Status
vLLM CUDA V1 / TinyLlama-1.1B-Chat-v1.0	NVIDIA RTX 5060 Laptop / 8 GB / WSL2	selected-text E0 + APC, concurrency 1–16	serving bounded load context pressure	all controlled outputs exact; selected C16 91.9–93.5 req/s; native E2 reported separately
TensorRT-LLM 1.2.1 / TinyLlama-1.1B-Chat-v1.0	NVIDIA RTX 5060 Laptop / 8 GB / WSL2	selected-text E0, concurrency 1–16	serving bounded load context pressure	all controlled outputs exact; selected C16 79.6–81.9 req/s; native E2 source seam audited and stopped
OpenVINO GenAI 2026.3.1 / Qwen2-0.5B-Instruct-int4-ov	Intel i7-1355U / Iris Xe	selected-text E0, continuous batching 1–16	serving bounded load context pressure	selected controlled recovery exact; C16 8.4–14.2 req/s; native E2 blocked

The engine registry now includes bounded-load E0 rows for vLLM CUDA, TensorRT-LLM, and OpenVINO GenAI. On the shared RTX 5060 Laptop GPU, 66-token vLLM selection sustains 91.9–93.5 requests/s at concurrency 16 as the matched full prompt grows to 1,274 tokens; independent full-context throughput falls to 10.0 requests/s with 1.41 s TTFT p99. TensorRT-LLM reaches 79.6–81.9 requests/s for its 66-token selected condition at the medium pressure point. OpenVINO selection remains exact through concurrency 16 on Iris Xe, while full-context correctness varies with distractor pressure. These are engine realization measurements at E0. They strengthen the selected-text baseline that native E2 must beat, but do not promote any CUDA or OpenVINO row to E2.

Table 28: Cross-engine context pressure at concurrency 16 and the largest tested prompt. Ratios are normalized within engine; values above one favor selected text.

Engine	Workload	Sel./full tok.	Succ. S/F	Req/s S/F	Req. ratio	p99 S/F	Tail ratio
vLLM CUDA	Shared	66/1274	1.00/1.00	92.4/72.1	1.28×	46/68	1.46×
vLLM CUDA	Independent	66/1274	1.00/1.00	91.9/10.0	9.19×	47/1408	29.83×
TensorRT-LLM	Shared	67/1276	1.00/1.00	80.7/56.9	1.42×	58/149	2.57×
TensorRT-LLM	Independent	67/1276	1.00/1.00	80.3/52.6	1.53×	58/173	2.98×
OpenVINO	Shared	63/831	1.00/1.00	14.1/11.5	1.23×	235/483	2.06×
OpenVINO	Independent	63/831	1.00/0.82	10.8/10.2	1.06×	319/530	1.66×

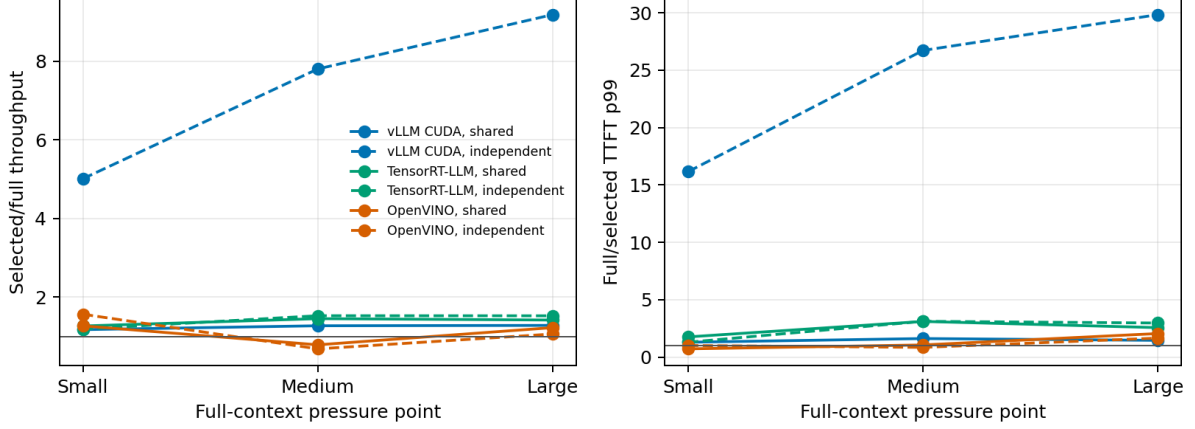


Figure 26: Within-engine throughput and TTFT-tail ratios over three context pressure points. Absolute rates are not compared across dissimilar hardware.

At the largest point, selected text improves shared-resource throughput by $1.23\text{--}1.42\times$ across all three engines and reduces TTFT p99 by $1.46\text{--}2.57\times$. Independent resources expose the largest difference on vLLM CUDA: selected/full throughput is $9.19\times$ and the TTFT-tail ratio is $29.8\times$ after the long-prefix scheduling boundary is crossed. TensorRT-LLM shows a steadier $1.53\times$ throughput and $2.98\times$ tail advantage. OpenVINO’s independent full condition recovers only 0.82 of targets at the largest point, so its $1.06\times$ throughput ratio must be read with the reported quality difference. The comparison therefore establishes a portable pressure-test method and a strong E0 baseline, not a hardware ranking.

Table 29: Frozen matched E0 selected-text versus E2 native-K/V quality. Each dataset contains 20 routed examples; selection is fixed before cold, warm, multi-query, and concurrency-eight engine consumption.

Engine	Dataset	Visible E0/E2	Reduction	Cold F1 E0/E2	Cold parity	All parity
mlx-lm	2wikimultihopqa	375/33	91.1%	0.067/0.067	100.0%	100.0%
mlx-lm	hotpotqa	415/39	90.6%	0.087/0.087	100.0%	100.0%
mlx-lm	qasper	399/28	93.0%	0.110/0.110	100.0%	100.0%
sglang-mlx	2wikimultihopqa	375/33	91.1%	0.074/0.074	100.0%	100.0%
sglang-mlx	hotpotqa	415/39	90.6%	0.084/0.084	100.0%	100.0%
sglang-mlx	qasper	399/28	93.0%	0.105/0.105	100.0%	100.0%
vllm-metal	2wikimultihopqa	378/33	91.2%	0.079/0.079	100.0%	100.0%
vllm-metal	hotpotqa	417/39	90.6%	0.074/0.074	100.0%	100.0%
vllm-metal	qasper	400/28	93.0%	0.101/0.101	100.0%	100.0%

Table 30: Macro E2/E0 execution-cost ratios; lower is better. Cold includes one-time ingestion, warm and multi-query use median completion, concurrent uses inverse requests/s, and E2 usable is milliseconds.

Engine	Cold	Warm	Multi-query	Concurrent	E2 usable ms
mlx-lm	1.130	1.155	1.148	1.132	51.2
sglang-mlx	0.976	1.136	1.140	1.122	55.0
vllm-metal	0.985	1.004	0.993	0.981	86.0

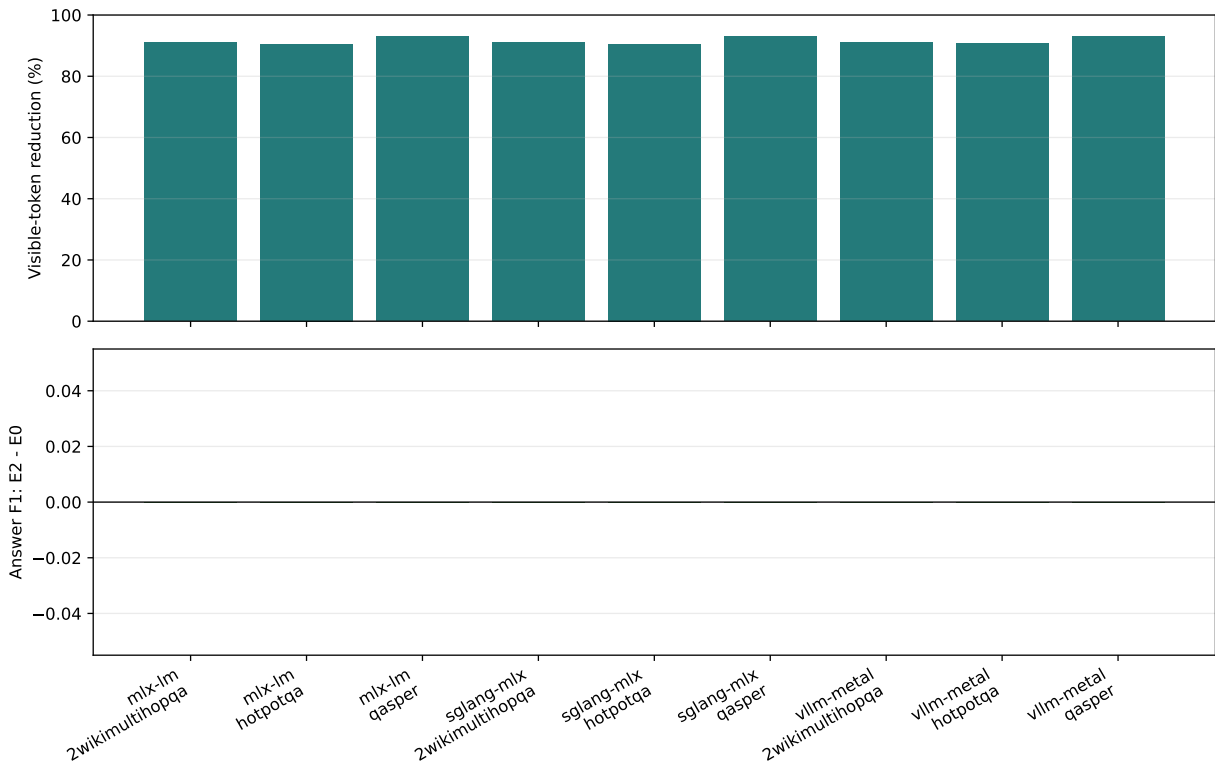


Figure 27: Holding routed evidence fixed separates visible-token reduction from semantic preservation. After restoring E0 through APC and capturing E2 from the same source prefill, all three engines retain every paired output and F1.

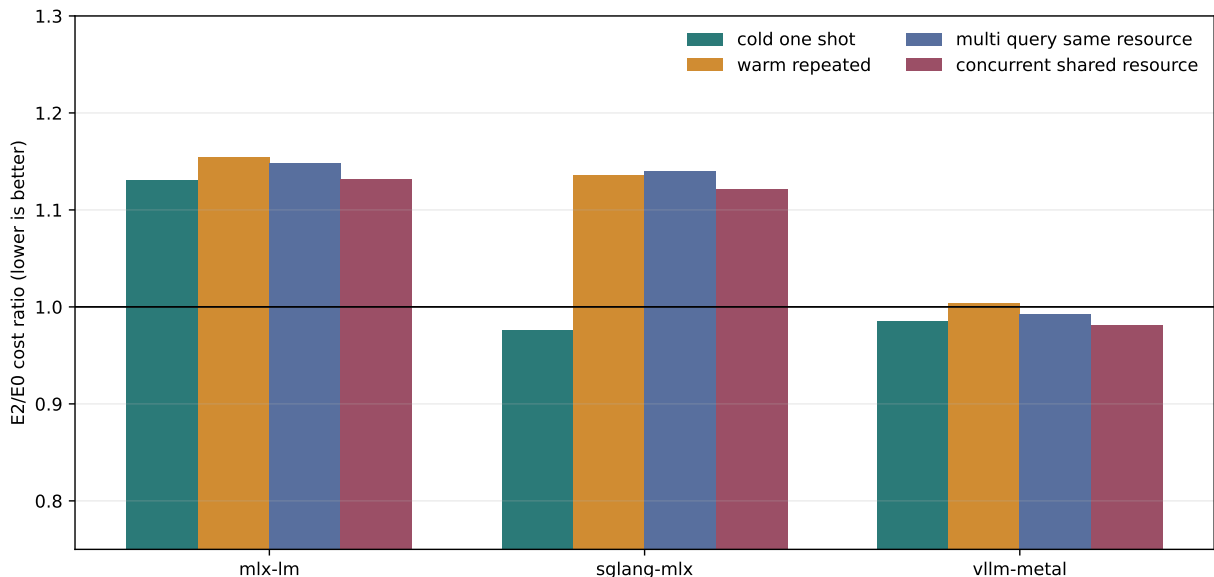


Figure 28: Representation economics after freezing retrieval. Ratios below one favor native E2. The current SGLang path wins only cold ingestion-inclusive cost; vLLM is near parity; MLX and SGLang still pay unfused decode overhead in reused and concurrent regimes.

The matrix now distinguishes companion evidence from generic provider defaults. On vLLM-Metal, selected native pages recover 75/75 answers and reach 34.41 requests/s at concurrency eight; after the first request in a native-selection namespace, APC supplies 128–144 prompt tokens. Alternating selected and ordinary rows satisfy their expected outcome in 40/40 cases. On SGLang-MLX, an 81-token shared-prefix hit composes with external HiCache: selected A/C recover 10/10, cleanup B recovers 0/5, and every selected request contains exactly one memory copy. A separate five-seed bridge uses SGLang’s built-in file-storage backend: fresh adapters hit and recover 5/5, with 38.7 ms mean writes, 122.2 ms mean cold L1 restoration (36.2 ms median), and 0.013 ms warm lookup. This is a local backend-contract result, not distributed HiCache. On MLX-LM, routed ordinary and native K/V agree on 60/60 original-answer QA pairs; evidence recall@4 ranges from 0.348 to 0.775, so the companion result retains an explicit open oracle gap. These are pinned controlled realizations, not portable claims that the default SDK provider may advertise unconditionally.

The shared matched benchmark makes the product boundary sharper. MLX-LM and SGLang-MLX preserve all 840 E0 outputs while moving 90.6–93.0% of visible prompt tokens into native K/V. Their current selected-cache wrappers remain 12–16% more expensive in reused or concurrent regimes. vLLM-Metal also preserves 840/840 outputs with zero pairwise F1 change. Its four macro cost ratios lie between 0.981 and 1.004. An earlier 593/840 result compared full-prefix E0 prefill against query-only E2 prefill; a five-example audit shows that APC-restored E0 and native E2 match 5/5 while full-prefix and APC-restored E0 themselves differ in 2/5. The SDK therefore treats all three pinned paths as semantic mechanism passes while keeping their economic status experimental. Engine labels alone never select these capabilities.

For product evidence we normalize each run into explicit staged conditions rather than exposing only paper-level E0/E2 labels: Selected Context (the frozen PRA selection rendered as visible text), Native Memory without a learned adaptor (the same selected source materialized as native K/V), and a mode-matched condition for an exact learned-adaptor bundle. No PRA is a separate ordinary-inference condition and appears only when actually run. The qualification runner pins model revision, tokenizer, quantization, engine, and selector output. It executes QASPER, HotpotQA, and 2Wiki separately and

records the source artifacts and checksums in the card. Thus a result for MLX 8-bit is never inherited by MLX 6-bit, bitsandbytes 8-bit, or another checkpoint.

Table 31 reports the new exact-identity cohorts. The MLX runs contain 20 examples per dataset and five dataset seeds (60 unique questions per identity). The smaller CUDA bitsandbytes run contains five examples per dataset (15 total) and is intentionally presented as controlled, not production-scale, evidence. Native Memory is promoted only when exact sequence agreement is at least 95% and mean token-F1 change is no worse than -0.01 . This gate is stricter than average-score similarity: it rejects the Gemma path despite its modest -0.0036 F1 change because only 3/60 sequences match exactly.

Table 31: Exact-identity selected-text (E0) versus native-K/V (E2) bundle qualification. The selector output is frozen between conditions. “Promote” means Native Memory with BALANCED becomes the exact-identity default.

Model / quantization	N	Exact	E0 F1	E2 F1	Visible Δ	Completion Δ	Gate
Llama-3.2-1B MLX 8-bit	60	60/60	0.1254	0.1254	-91.5%	-0.2%	Promote
Gemma-3-1B MLX 8-bit	60	3/60	0.0735	0.0699	-91.1%	$+5.9\%$	Candidate
Qwen3-4B MLX 8-bit	60	60/60	0.1315	0.1315	-91.5%	$+1.9\%$	Promote
Qwen3-8B MLX 6-bit	60	60/60	0.1582	0.1582	-91.5%	$+1.4\%$	Promote
Qwen3-8B MLX 8-bit	60	60/60	0.1440	0.1440	-91.5%	$+0.6\%$	Promote
Qwen3-14B MLX 8-bit	60	60/60	0.1494	0.1494	-91.5%	$+0.3\%$	Promote
Qwen2.5-1.5B BNB 8-bit	15	0/15	0.3991	0.0179	-91.7%	$+203.5\%$	Candidate

The passing Qwen and Llama rows preserve output exactly while hiding about 91.5% of selected source tokens from the visible prompt. Median TTFT changes are -6.9% for Llama, -9.2% for Qwen3-4B, $+0.7\%$ for Qwen3-8B 6-bit, -5.7% for Qwen3-8B 8-bit, and -2.4% for Qwen3-14B 8-bit. Mean completion overhead within the Qwen 8-bit family falls from 1.9% at 4B to 0.6% at 8B and 0.3% at 14B. This is a useful scaling observation, but not yet a scaling law: the cohort and host are fixed and the models have different output distributions.

The two rejected rows identify different integration boundaries. Gemma’s mixed local/global attention geometry needs architecture-specific native-memory calibration. The eager bitsandbytes path preserves evidence recall and removes 91.7% of visible tokens, but loses 0.3813 combined token F1 and takes roughly three times as long end to end; Selected Context therefore remains its default. Neither failure is converted into an unspecified missing value.

The public catalog applies the same schema to 16 current bundles. Each card enumerates every declared engine, mode, and profile and labels absent work by cause: `NEEDS_RUN`, `NO_QUALIFIED_ADAPTER`, or `CALIBRATION_PENDING`. It preserves `NOT_MEASURED` when no stronger cause is established. Five Qwen/Llama quantized identities now carry promoted Native Memory evidence; Gemma MLX 8-bit and Qwen2.5 bitsandbytes 8-bit carry measured candidate failures. The generated site evidence matrix remains the exhaustive view.

These results do not show that a learned adaptor is necessary. The promoted BALANCED profiles use parameter-free routing and all eligible consumer layers. The Qwen3-4B bundle includes an opt-in learned router whose QASPER recall gain does not transfer to HotpotQA; it is not substituted for a missing end-task adaptor arm. Other exact identities state `NO_QUALIFIED_ADAPTER`. The machine-readable audit retains this distinction for cards, CLI reports, and Control Plane views.

9.2 Precision and quantization robustness

Precision is now part of the canonical evidence key rather than an informal model-name suffix. Each row records a broad numerical family (for example, `INT4`) and the concrete encoding (for example, `MLX-4bit`); serving, feature-extraction, and adaptor-parameter precisions remain separate. This prevents evidence for MLX 4-bit weights from silently qualifying AWQ, GPTQ, GGUF, BF16, or another

conversion of the same source model. Converted artifacts additionally admit source and tokenizer revisions, quantization configuration hashes, conversion recipes, and artifact checksums.

The current normalized ledger contains 53 planned or observed exact-precision rows. Five have matched Selected-Context and Native-Memory end-task measurements; 46 planned rows still require execution. Table 32 shows only measured rows, while the machine-readable supplement retains all missing and blocked states. Qwen3-14B is the sole present within-model cross-precision comparison: both MLX 4-bit and MLX 8-bit preserve token-F1 between Selected Context and Native Memory, although the 4-bit cohort has only 15 pairs and the 8-bit cohort has 60. These observations support exact-encoding qualification, not a general claim that PRA is precision invariant.

Table 32: Observed exact-precision Selected-Context versus Native-Memory rows. Missing FP32/BF16 and learned-adaptor cells are omitted here but retained as explicit states in the 53-row generated ledger.

Model	Encoding	Evidence	Pairs	$\Delta F1$	Vis. SC	Vis. NM
Qwen/Qwen3-14B	MLX-4bit	ENGINE.QUALIFIED	15/15	+0.0000	315.5	34.3
Qwen/Qwen3-14B	MLX-8bit	ENGINE.QUALIFIED	60/60	+0.0000	396.3	33.5
Qwen/Qwen3-4B	MLX-8bit	ENGINE.QUALIFIED	60/60	+0.0000	396.3	33.5
meta-llama/Llama-3.2-1B-Instruct	MLX-8bit	ENGINE.QUALIFIED	60/60	+0.0000	390.9	33.2
google/gemma-3-1b-it	MLX-8bit	CONTROLLED	3/60	-0.0036	395.3	35.2

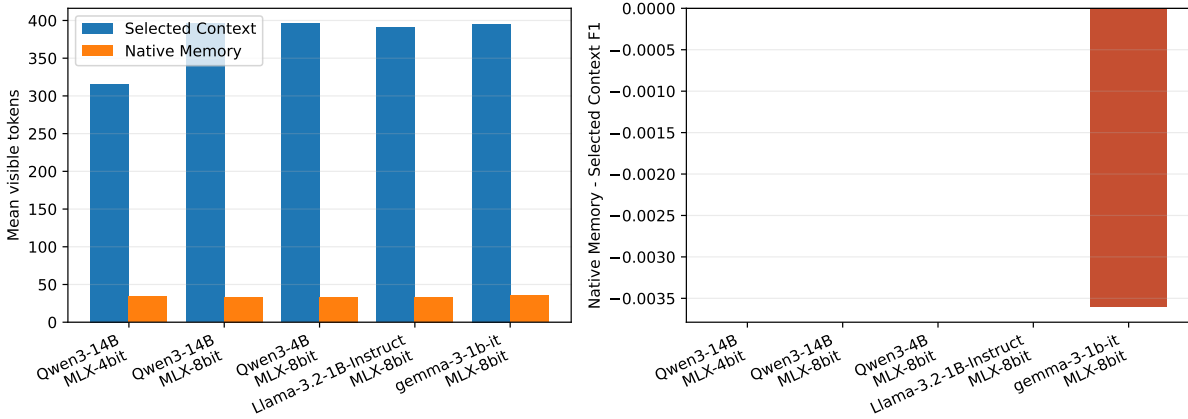


Figure 29: Observed visible-context accounting and Native-Memory-minus-Selected-Context token-F1 change by exact precision encoding. The plot deliberately excludes planned rows; absence is not rendered as zero.

The public workflow is reproducible through **pra model qualify-precision**. It resolves an immutable model identity, classifies staged memory gates, validates imported evidence against the exact precision key, and emits the canonical JSON, CSV matrix, card fragment, and LaTeX table from one record. Every current local bundle now declares **supported_precisions**; the catalog exposes size, family, encoding, engine, mode, profile, datasets, and evidence tier. No learned-adaptor arm is inferred from the presence of a structural bundle.

A matched M5 study isolates precision-specific MLX cost on ten fixed MultiHop-RAG questions. Table 33 reports the deployed No-PRA and Native-Memory pipelines: one-shot native execution costs 5.8–6.1% more total latency for BF16, 10.0–10.7% for INT8, and 14.8–16.1% for INT4. The row retains the quality delta only as a guardrail. Because this historical cohort changed both selector and representation, it is not a selector-frozen transport comparison; Papers 3.2 and 3.3 own its retrieval interpretation.

Table 33: Matched Qwen3-4B three-rung precision study on one M5 host ($n = 10$, seed 11). “No PRA” is standard RAG context packing and “Native” is generic PRA with detached native K/V; no learned document-RAG adaptor or intervening Selected Context arm is qualified.

Encoding	Candidates	No-PRA F1	Native F1	Δ_{NM-NP} F1	Task Δ_{NM-NP}	NM/No-PRA latency
MLX-bfloat16	20	0.1233	0.1214	-0.0019	+0.00	1.058
MLX-bfloat16	50	0.1177	0.1249	+0.0072	+0.00	1.061
MLX-8bit	20	0.0916	0.0881	-0.0035	+0.00	1.107
MLX-8bit	50	0.0903	0.0939	+0.0036	+0.00	1.100
MLX-4bit	20	0.0944	0.0768	-0.0176	-0.20	1.148
MLX-4bit	50	0.0906	0.0815	-0.0091	-0.10	1.161

The small, one-seed cohort is directional engine evidence. A selector-frozen non-Qwen replication, Qwen3-14B BF16, and peak-memory measurement remain `NEEDS_RUN`; absent adaptor arms remain `NO_QUALIFIED_ADAPTER`.

The expanded confirmation raises each engine to 149 unique questions and 2,086 paired requests. Every engine is 2,086/2,086 exact, for 6,258/6,258 pairs overall; the pooled Wilson 95% lower bound is 0.9994. Visible-token reduction is 90.4–93.0%. The macro ratios in Table 34 show why semantic and economic gates remain separate: vLLM-Metal is within 0.9% of parity in every regime, SGLang-MLX pays 5.4–7.4% on reused paths, and MLX-LM pays 7.2–13.9% while using the same frozen evidence.

Table 34: Expanded selector-frozen E2/E0 cost ratios over 149 unique questions and 2,086 paired requests per engine. Lower is better.

Engine	Cold	Warm	Multi-query	Concurrent	E2 usable ms
mlx-lm	1.043	1.139	1.133	1.072	109.9
sglang-mlx	0.980	1.074	1.071	1.054	117.1
vllm-metal	1.009	0.997	1.003	0.991	149.7

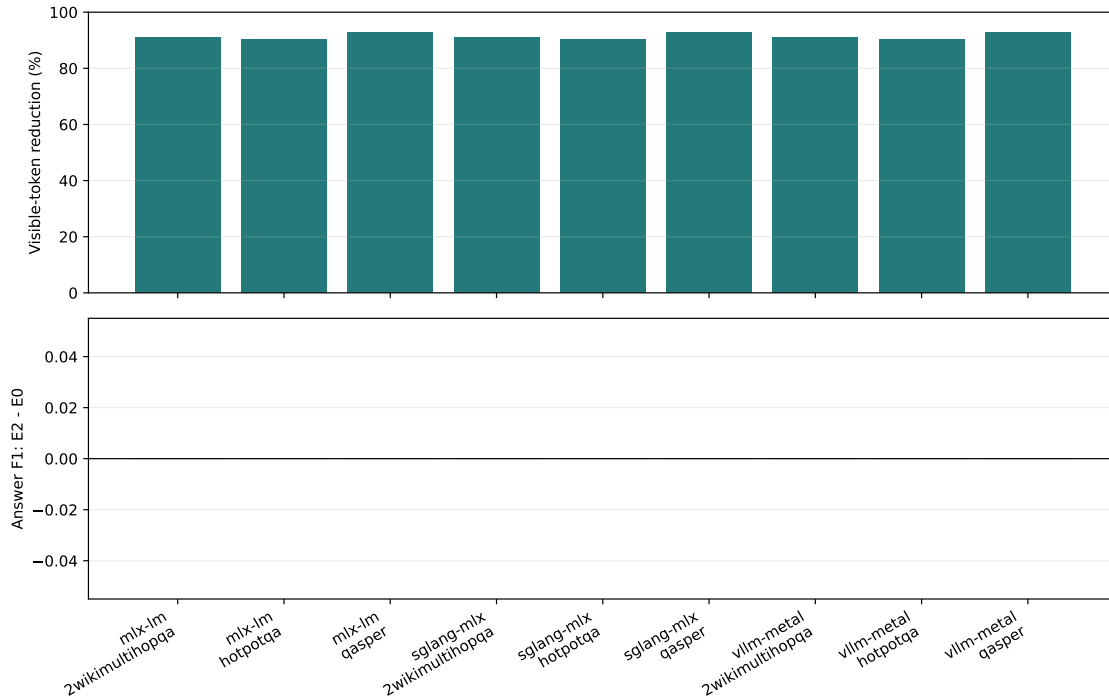


Figure 30: Expanded frozen-selection quality and visible-token accounting. Every engine preserves every paired output; confidence reflects finite sample size.

The vLLM audit also establishes a product invariant that is easy to miss at an HTTP boundary. APC hashes visible prompt state, but native PRA evidence is hidden from that state. A native provider must therefore derive an opaque APC salt from tenant scope, ordered logical-memory identity, selected-token count, position policy, and consumer layers. Stable selections retain warm reuse; changed selections cannot inherit query K/V produced under another memory. The SDK exposes this derivation as an engine helper and tests equal-selection reuse, changed-selection isolation, geometry sensitivity, and identifier non-disclosure. A five-example capture/replay audit preserves E0’s first token in 5/5 cases at the nominal position base. Holding source-ingestion and query-prefill geometry fixed raises complete 24-token parity from 3/5 to 5/5; the full corrected cohort then reaches 840/840.

The product surface includes installable package metadata, Hugging Face loading, runtime config serialization, direct and external reference registration, session isolation, callable and folder-backed capability registration, lazy view activation, typed resource discovery, safe execution, compact result ingestion, selected replay, address search, cursors, size-adaptive native result routing, lazy native-region promotion, generation, inspection, benchmark commands, and two executed notebooks. The canonical `pra` CLI exposes separate `model`, `adapter`, `profiles`, `bundle`, `runtime`, `agent`, `gateway`, and Hub-only `hf` namespaces. `pra-hf` remains a deprecated alias for one release cycle. The `pra gateway serve` command launches the independently testable gateway with health, capability, logical-request, and OpenAI-compatible endpoints. A deployment can therefore discover unsupported paths before serving.

9.3 Model onboarding, calibration, and community distribution

The product workflow treats architecture adaptation, learned routing adaptation, profile calibration, engine realization, and Hub distribution as different operations. `ModelInspector` reads configuration metadata without loading weights. `StructuralAdapterBuilder` emits a versioned declarative map-

ping for conventional Qwen, Llama, and Gemma decoder attention. **ModelValidator** records V0-V9 independently, so discovering Q/K/V projections cannot be reported as semantic success. The later stages cover disabled-PRA parity, native capture, visible-prefix equivalence, source-relative position geometry, cached decode, grouped-query layout, selected-region consumption, and generation.

ProfileCalibrator packages measured rows from the existing benchmark registry. A small-cohort optimum remains `QUALITY_MAX_CANDIDATE` with `SMOKE/CALIBRATION_PENDING` evidence; only benchmark-scale validation may assign `QUALITY_MAX`. `PRAModelBundle` then combines immutable base model identity, structural and learned adapters, profiles, evidence, runtime compatibility, engine realizations, and provenance without copying base-model weights. Hub pull and push are optional transport operations under `pra hf`; generic Transformers execution does not depend on Hub login. The public catalog currently contains nine canonical `EInnovator/prax` bundles. In addition to Qwen3-0.6B, Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-32B, Llama-3.1-8B, and Gemma-3-1B, it includes matched general-instruction and code-instruction Qwen2.5-1.5B identities. A declarative manifest orders the Collection by measured usefulness rather than publication date: Qwen3-14B and Qwen3-32B lead, while the 0.6B artifact is explicitly a research/reference bundle. Bundle discovery checks the base model, revision, engine, and immutable bundle revision before AUTO may resolve it; checksum validation precedes managed runtime loading. A release gate requires one qualified recommended profile and rejects routing-only headlines, unpaired absolute quality, profile conflicts, and revision or quantization mismatches. The published learned routers remain dataset-qualified research profiles rather than universal defaults.

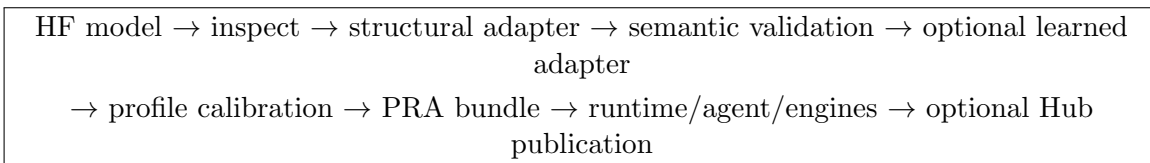


Figure 31: The model-onboarding path. Each arrow is a separately inspectable service boundary rather than an implicit side effect of one CLI callback.

9.4 Engine-neutral runtime providers

The canonical launch path is `pra runtime serve` with a model and `-e ENGINE`. Click parses the request, `RuntimeManager` resolves a provider, and that provider translates a common model, revision, PRA bundle/profile, device, session, and cache contract to the upstream engine. Repeated `--engine-arg KEY=VALUE` values preserve engine-specific escape hatches without cloning upstream CLIs. Providers return serializable handles and distinguish process start from readiness. Inspection reports static declarations; doctor checks report installed dependencies and live endpoints separately.

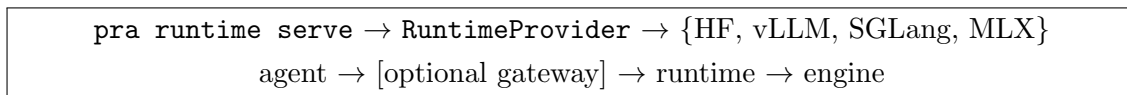


Figure 32: One runtime command preserves the separation between agent policy, optional gateway mediation, and engine-native execution.

The default vLLM provider deliberately advertises E0 unless the measured native bridge is installed. Paper 6 now establishes both a controlled Metal V1 path and a version-bounded CUDA 0.28 worker hook with scheduler-invisible detached pages. The CUDA cohort is exact at 128 requests per condition, but HOT and WARM E2 trail E0 useful throughput and LMCash was not exercised. This is measured native capability, not an unconditional production default. SGLang and MLX advertise the companion

papers’ measured native mechanism support under their pinned environments. Paper 6.1 additionally closes local shared-prefix/HiCache composition at the runner boundary, while distributed HiCache and E3 scheduler-aware PRA remain false. The shared capability object feeds runtime inspection, gateway negotiation, agent launch, and benchmark records.

9.5 Agent-level configuration and interactive surfaces

Low-level `pra.yaml` controls model/context execution. A separate, versioned agent profile controls model endpoint, workspace, durable sessions, tool approval, skill directories, MCP declarations, task policy, and generation budgets. Resolution is explicit CLI options, selected named profile, default profile, project config, user config, then package defaults. The short option `-p` selects an agent profile; `-P` independently overrides PRA. Credential files remain server-side references and are redacted from session, inspection, benchmark, and model-card output.

Agent profiles now independently select `context.transport=auto|pra|text`, text-fallback permission, and required features. The CLI exposes `--context-transport` and `--allow-text-fallback/--no-text-fallback`; `-P/--pra` continues to select the model/runtime PRA profile and is not overloaded with wire semantics. The launch summary, TUI `/runtime` command, and web session diagnostics report requested and resolved transport, capability source, gateway mode, integration level, resource-delta support, and native-K/V status without including record bodies.

`pra agent chat` is the stable TUI reference and can resolve a default profile without model flags. `pra agent run` provides a noninteractive form. The optional FastAPI/WebSocket UI is another presentation layer over the same `PRAAgent`, `AgentLauncher`, session, task, and authorization services. It supports multiple durable conversations and sequence-based replay after reconnect. A dangerous tool call remains blocked until the backend receives a typed one-shot approval; the browser cannot bypass host policy. The UI binds to loopback by default and is experimental rather than a scientific contribution or production security claim.

The fleet Control Plane follows the same separation at deployment scope. REST, streamable-HTTP or stdio MCP, the built-in Control Plane agent, and an embedded operator CLI call one transport-neutral `ControlManager`. Its fleet, Registry, deployment, action, qualification, observability, context, experiment, and audit managers own authorization and policy; engine and Registry clients remain behind backend protocols. A declarative operation catalog maps semantic operation IDs to permissions, side effects, and risk, and drives REST route exposure and MCP tool discovery. A disabled operation is therefore absent from OpenAPI or MCP discovery rather than merely failing after dispatch. The default MCP surface is disabled; when enabled it is read-only except for plan construction, with apply and experiment submission separately denied.

Operational mutation uses a common plan-apply transaction. The plan records current and projected state, impact, reversibility, restart requirement, required permission, confirmation requirement, tenant, expiry, and idempotency key. Plans and completed results are durable, so REST, MCP, and CLI retries share the same idempotency decision. Manager-owned audit records add roles, permission, request and trace IDs, transport, before/after state, and outcome. An official MCP SDK protocol test initializes both stdio and authenticated streamable HTTP, discovers the configured tools, and invokes the same fleet manager used by REST. This is interface validation rather than a serving-quality or latency claim.

9.6 PRA under agentic coding workloads

The coding-agent campaign uses end-task success as the admission gate and context economy only as a secondary objective. Terminal-Bench 2.1 is delegated to Harbor 0.22.0 with frozen task identifiers, fresh task containers, official verifiers, and preserved agent-native context policy. An oracle first passed the downloaded `filter-js-from-html` task, establishing that task setup and grading were functional.

OpenCode 1.18.26 with Ollama/Qwen3-14B then completed all five No-PRA smoke trials without harness exceptions, but solved 0/5 tasks while passing 7/12 individual verifier tests and consuming 100,883 cumulative input tokens. We therefore did not promote that pairing into a PRA profile sweep.

The follow-up campaign makes that sequencing executable rather than relying on operator convention. Each published baseline pins source and model revisions, harness, cohort, scaffold, engine, precision, context and step limits, decoding policy, and official grader. A resumable scheduler writes a durable receipt after every state transition. Gateway pass-through, Selected Context, and native-PRA cells cannot launch until their exact No-PRA dependency is marked `BASELINE_REPRODUCED`. A smoke subset or a changed engine, quantization, or context window remains `BASELINE_ATTEMPTED`; it can qualify infrastructure but cannot authorize a PRA efficacy claim.

Gateway placement is not synonymous with PRA execution. We therefore factor the promoted agent experiment into two binary ownership axes. The ordinary controls are direct engine access and G00 pass-through to that same ordinary engine. The native comparison is (i) an agent sending typed selected context directly to a PRA-capable engine and (ii) the same agent using a G11 PRA gateway in front of that engine. This defines two experiments that must not be conflated. In *G11-equivalence*, the complete request and selected-resource envelope are replayed, and matching resource digests make the difference an estimate of gateway mediation. The direct run writes an ordered selection fixture keyed by a digest of the complete agent-visible request. Replay has no ordinal or approximate fallback: a changed trajectory aborts the equivalence cell instead of admitting a different selection. In *G11-end-to-end*, each placement owns its normal session and selection path; this estimates product behavior, with selection and transport jointly present. Both fix the engine, model revision, task and repeat, budget, profile, and official grader. Easy-50 contains both the fail-closed frozen replay and a separate route-owned product cell. G10 remains a separate selected-text fallback and is not relabeled as native execution.

Every Easy-50 arm fixes a 4,096-token per-turn completion ceiling in addition to the 32,768-token context and 50-step limits. This ceiling is part of the execution fingerprint. It bounds pathological continuations when a backend does not emit its normal end marker; ordinary shorter actions are unchanged, and a ceiling hit is retained as a protocol event rather than a PRA quality outcome.

The executable version-2 matrix records connection, engine-PRA and gateway-PRA flags, gateway mode, physical engine target, and comparison group in every row. It refuses native labels unless endpoint preflight reports effective native K/V, and it completes each direct task before launching its gateway mate. Paired reports count outcome agreement, regressions, recoveries, token traffic, and wall time. Mini-SWE-agent and Qwen Code are the first enabled external harnesses; OpenCode, Pi, OpenHands, Aider, SWE-agent, and endpoint-configurable commercial agents enter only after their own No-PRA baselines and protocols are admitted. The first-party PRA Agent is deliberately last and runs both direct and through G11. That pair serves two purposes: comparison with external agent loops and diagnosis of PRA Agent session, tool, typed-record, or transport faults when its outcomes diverge. Different agents remain separate experimental units because their prompts and tools differ.

The 40-cell Terminal-Bench direct/G11 matrix is a transport qualification, not the principal efficacy experiment. Its two enabled harnesses each resolved only 1/10 No-PRA tasks, and both resolved the same `fix-git` identity. At that floor, one outcome change dominates the relative rate. The matrix can still reveal protocol failures, crashes, isolation errors, duplicate native materialization, and accounting regressions. PRA efficacy is instead tested on the admitted Easy-50 cohort described below, whose direct ordinary baseline resolved 14/50 tasks. Its preregistered execution order is corrected G00, direct native PRA at 50%, frozen G11 equivalence, G11 end to end at 50%, matched 50% truncation, and G10 at 50%; lower budgets follow only when that comparison is informative.

The first capable reproduction target is TIGER-Lab FIM-14B under its published R2E-Gym configuration [9, 5]: all 500 SWE-bench Verified instances, an ordinary vLLM endpoint, BF16 weights, temperature zero, a 65,536-token limit, 40 normal steps with an absolute cap of 100, and official SWE-

bench container grading. The reported three-seed mean is 29.20%. The available CUDA host has only 8 GiB of device memory, so it cannot host this exact 14B cell. A quantized FIM-7B llama.cpp run on the 48 GiB M4 is therefore recorded only as a No-PRA distributed-path calibration. This distinction prevents a convenient but non-identical local configuration from being mislabeled as reproduction.

The calibration completed two seed-42 instances through an ordinary llama.cpp endpoint, the unmodified R2E-Gym scaffold, and official SWE-bench container grading. It resolved `django_django-13821` and did not resolve `sympy__sympy-19954: 1/2`, with a Wilson 95% interval of 9.5–90.5% and no grader errors. The resolved task terminated naturally after 31 model calls; the unresolved task reached the 40-step limit. Across both tasks the agent made 72 calls, accumulated 1,123,772 prompt tokens and 11,878 completion tokens, and recorded 607.4 seconds of trajectory time. The small cohort does establish a functioning distributed model-agent-container-grader path and a nonzero task signal. It does not estimate population accuracy: llama.cpp, Q4_K_M, two instances, and a 32,768-token context differ from the published vLLM/BF16/500-instance/65,536-token setup. The scheduler therefore assigns `BASELINE_ATTEMPTED`, and no gateway or PRA cell is admissible yet. The next fixed No-PRA cohort must contain enough tasks to estimate a nontrivial success rate and stable trajectory costs before any treatment is considered. Its receipts retain cumulative and maximum prompt tokens, a conservative unique-context estimate, repeated-context fraction, calls, tools, trajectory length, patch size, wall and model time, termination, and official grader outcome. Retrospective normalization of the two trajectories finds 50,106 maximum-prompt tokens across tasks versus 1,123,772 cumulative prompt tokens, a conservative 95.5% repeated-context estimate under the accumulating R2E-Gym scaffold. This load is a primary future treatment target, but savings will count only beside a matched solved-task rate. TTFT and prefill remain explicitly unmeasured when the endpoint does not expose them.

Increasing code-model capacity did not close this admission gate. OpenCode with Ollama/Qwen3-Coder-30B scored 0/1 on `query-optimize` and passed 4/6 verifier tests. It made five tool calls, read the source query, and wrote a candidate, but retained a correlated subquery. The official performance verifier consequently consumed almost its full 30-minute allowance (1,791.2 seconds total task wall time). The run used 50,876 cumulative input tokens and had no harness exception. This distinguishes a model-level optimization failure from gateway or tool-transport failure, but still supplies no quality floor for comparing PRA profiles.

Holding the model, gateway, engine, and `query-optimize` task fixed exposed substantial harness effects even before PRA was enabled. Table 35 reports the one-task diagnostic. OpenCode and Pi both read and wrote candidate SQL, but neither met exact-output and runtime requirements. OpenHands entered a repeated-continuation loop and did not create the requested file. The OpenHands row uses version 0.57.0 because Harbor’s adapter expects a module path absent from 1.11.0. These sparse rows qualify execution paths; they are not an agent ranking.

Table 35: Same-model No-PRA Terminal-Bench mechanism gate. Input tokens marked N/R were not exposed by the pinned harness integration.

Agent	Success	Tests	Input tokens	Wall time (s)	Tool path
OpenCode 1.18.26	0/1	5/6	20,996	466.0	read + write
Pi 0.73.1	0/1	4/6	5,219	478.1	read + write
OpenHands 0.57.0	0/1	2/6	N/R	489.9	no solution file

We then expanded the stronger-model gate to a preregistered ten-task Terminal-Bench 2.1 cohort. Qwen3-Coder-30B Q4_K_M served every trial through Ollama on a 48 GiB M4 Pro, while Harbor and the official verifiers ran on a separate 16 GiB Apple host. Mini-SWE-agent 2.4.6, Qwen Code 0.23.0, and Aider 0.86.2 each attempted the same ten tasks, for 30 completed No-PRA trials. Mini-SWE-agent

and Qwen Code each solved `fix-git`; Aider solved none. Thus aggregate success was 2/30, or 6.7% (Wilson 95% interval 1.8–21.3%), but only one of ten unique tasks was solved by any harness. Table 36 reports the observed workload. These rows are not an agent ranking because each harness retains its own prompt, tools, and loop policy.

Table 36: Expanded Qwen3-Coder-30B No-PRA admission gate. Token totals cover only harness integrations that report usage; N/R means not reported, not zero.

Harness	Runs	Success	Input tokens	Tool calls	Wall (h)
Mini-SWE-agent	10	1/10	2,562,781	271	1.14
Qwen Code	10	1/10	4,154,315	143	1.84
Aider	10	0/10	N/R	N/R	1.04
Observed total	30	2/30	6,717,096 [†]	414 [†]	4.02

[†]Coverage is 20/30 trials because Aider’s pinned Harbor adapter does not expose token or turn telemetry.

Of the 28 unsuccessful trials, 21 completed with an official score below the success threshold, six reached the agent timeout after usable model activity, and one reached the verifier timeout. Retaining timeouts as outcomes avoids silently improving the apparent success rate. The larger model and broader harness set therefore establish substantial real coding-agent traffic—the 20 token-reporting trials alone accumulated 6.72 million input tokens—but not a stable enough quality floor for a treatment comparison.

This negative gate matters to the product claim. Token reduction from a model that no longer completes coding tasks is not a PRA benefit. A code-capable agent/model pairing must first establish nonzero success and enter the preregistered 20–80% admission band, with 30–70% preferred. The expanded cohort’s 6.7% result falls below that lower boundary, so the scheduler correctly withheld treatment launches. Only after admission may identical tasks compare No PRA, PRA Selected Context without an adaptor, and Selected Context with the immutable Runtime Bundle under BALANCED. Table 39 makes the unrun conditions explicit: `BLOCKED` is missing evidence caused by the quality or reproduction gate, not a synthetic zero or an observed PRA failure. The normalized schema retains official score, verifier counts, turns, tool calls, cumulative context, TTFT and ITL distributions, decode output tokens/s, model-call requests/s, queue and inference time, resource use, and cost for that subsequent paired analysis.

Difficulty-qualified local admission. We next froze an easier cohort before inference by filtering SWE-bench Verified revision `c104f84` to its `<15 min fix` human-difficulty label, ordering eligible identifiers by a salted SHA-256 digest, and taking the first 20. The ordered cohort digest is `021b532a...3bd75fb`. This is a local capability calibration, not a claim of reproducing the H100 study below. Qwen3-Coder-30B Q4_K_M, served by Ollama 0.32.7 on an M4 Pro, used the unmodified mini-swe-agent 2.4.6 backticks scaffold, a 32,768-token context, 50 steps, temperature zero, and the official SWE-bench 4.1.0 Docker grader. The model resolved 9/20 tasks, or 45.0% (Wilson 95% interval 25.8–65.8%). This falls inside the preregistered 30–70% preferred band and therefore admits the nested Easy-50 treatment frontier.

The 20 trajectories made 525 model/tool turns and accumulated 4,286,673 prompt tokens and 84,145 completion tokens. Summing each task’s maximum prompt gives 226,423 tokens; the remaining 4,060,250 tokens are a conservative 94.7% repeated-context estimate under the accumulating scaffold. Four submissions had empty patches, two had patches rejected by the official grader, and two reached the step limit. These failures remain in the denominator. This baseline is already a materially stronger test than the floor-regime Terminal-Bench cells: the agent is competent, while repeated context is large enough for selection to matter. No PRA quality or savings conclusion is drawn from the admission

row alone; those require the identical Easy-50 tasks under pass-through, matched truncation, and PRA Selected Context.

The nested Easy-50 No-PRA expansion resolved 14/50 tasks, or 28.0% (Wilson 95% interval 17.5–41.7%), and retained one one-hour agent timeout as an unresolved result. Thus the larger estimate contracts from the optimistic 9/20 calibration prefix but remains above the preregistered 20% treatment floor. Across 1,229 model/tool turns it accumulated 10,115,822 prompt tokens and 200,677 completion tokens. The per-task maximum prompts sum to 526,997 tokens, leaving 9,588,825 tokens, or 94.8%, as conservative repeated context. Eight tasks produced empty submissions and 13 produced patches rejected by the official grader; all remain in the denominator. The pass-through and matched-budget conditions reuse these exact 50 identifiers and execution settings.

An initial G00 pass-through attempt produced a nominal 0/50, but it is not a quality observation. Every trajectory stopped on its first model call with an `InternalServerError` and an empty submission. The downstream URL had been supplied in the conventional OpenAI form ending in `/v1`, while the adapter appended a second `/v1`; the resulting upstream HTTP 404 escaped the request handler and appeared to the harness as a disconnected server. The same implementation also discarded generation options and reconstructed a reduced completion envelope. We therefore mark all 50 rows `INVALID_TRANSPORT` and exclude them from PRA deltas. The repaired G00 path normalizes both origin and `/v1` API roots, pins and advertises the served model, preserves OpenAI request and response fields, returns structured upstream errors, and uses an explicit upstream timeout. A benchmark run now must pass health, mode, model-catalog, and live one-token generation probes before its first task. The clean corrected parity gate then reran three tasks previously solved by the direct baseline and resolved all 3/3 again with the official grader, zero timeouts, and zero grader errors. It used 69 model calls, 403,703 prompt tokens, and 9,503 completion tokens, versus 63 calls, 458,690 prompt tokens, and 8,573 completion tokens in the corresponding direct trajectories. This tiny selected-on-success diagnostic establishes end-task transport parity, not an efficiency advantage or a population-level quality estimate; the two runs followed different trajectories despite temperature zero, and gateway latency was confounded by network placement and a shared single-slot model server. It clears the transport gate for a fresh corrected Easy-50 comparison.

9.6.1 Claim 1: correct live-history semantics

Native profile qualification exposed three agent-history correctness bugs. First, mini-swe-agent can reject a malformed assistant action and append a format-error user observation without retaining the rejected action; the live KV sequence must therefore support exact-prefix backtracking rather than assume append-only history. Second, independently selecting message fragments can retain two assistant actions while omitting the intervening tool observation, which Qwen’s chat template rejects. Third, treating only the latest observation as mandatory can still discard the assistant command that produced it. On the task-01 trajectory this changed the second generated response and immediately caused a format error. The corrected selector ranks historical atomic causal turn bundles: one assistant action plus all following user/tool observations up to the next action, while the complete active action–observation tail is mandatory. The task statement remains pinned, selected bundles retain causal presentation order, and the complete reconstructed request must pass the physical model’s chat template before resource materialization. Intermediate frozen replay of the admitted task-01 trajectory validated every one of its 31 user-ending request prefixes at each of the 75%, 50%, and 25% profiles (93 rendered requests total). This validates template grammar, not causal completeness or task efficacy; both fragment-level runs and profile attempts made before the active-tail correction are excluded from solve-rate and efficiency deltas and must be rerun. The revised segmenter also treats message/tool records as the first physical boundary. Short records remain intact; overlap never crosses records; and an oversized observation splits at file headers, stack frames, test cases, or diff hunks before falling back

to token windows. Every child retains a stable parent record and causal-group ID plus return-code, mutation, and result metadata. Child slices are disjoint before budget accounting, while physical engine packing preserves their logical identities. Once engine correctness qualifies, the matched segmentation ablation will compare fixed token windows, record-aligned messages, causal bundles, and hierarchical causal bundles with selective regions inside oversized observations. The engine path also stopped deleting the pinned selected-history sequence when its digest changes. It submitted the replacement text to the same owned slot and let llama.cpp retain the exact common token prefix while trimming and re-evaluating the divergent tail. A physical M4 probe cached 2,450 of 2,455 replacement-prefix tokens and evaluated five tokens versus 2,455 under cold materialization. This is useful composite-prefix reuse, but it does *not* qualify agent-history PRA: any selected text after the common prefix is encoded again, and several selected records are collapsed into one physical resource. We therefore reclassify this path as a rematerialization diagnostic.

The first corrected active-tail efficacy result separates those layers. On task 01, both the same-engine plain control and the 100% live-prefix PRA arm resolve the task in 31 model calls with exact trajectories. The 75% selector matches them through two generated actions, then diverges when it drops the oldest completed turn on request 3; it submits after 23 model calls but its nonempty one-line patch fails to apply, for 0/1 officially resolved. Estimated logical-to-physical context falls by 23.2%. Independently, resource-delta telemetry records 83,644 selected-prefix token-exposures but only 33,600 newly evaluated resource tokens, with 43,852 exact common-prefix cache hits, three identity hits, 18 delta updates, and no physical K/V copy. These counters describe ordinary longest-common-prefix reuse of changing selected text, not arbitrary selected-record K/V reuse. This selector profile also fails its task-quality gate. The row is retained only as a diagnostic and does not admit an end-to-end gateway or PRA efficiency claim. The task-01 50% arm also resolves 0/1: it reaches the 50-step limit with no patch while reducing estimated logical context by 47.0%. Its 167,908 resource-managed selected-prefix token-exposures require 57,301 newly evaluated tokens; 83,775 tokens hit exact delta prefixes, 13 requests are identity hits, 34 are delta updates, and no K/V copy occurs. The 25% arm likewise resolves 0/1 at the 50-step limit with no patch. Its pinned active tail limits realized estimated savings to 67.6%, but delta/identity reuse reduces newly evaluated resource tokens to 19,193 across 116,572 resource token-exposures. A frozen-selection cache on/off factorial was planned, but the Easy-50 matrix is now stopped until the live-prefix subset implementation passes the correctness gate below.

An initial frozen factorial appeared to fail a stricter consumption gate. The cache-off 50% anchor submitted an unresolved nonempty patch after 44 model calls and 507.9 s. The cache-on arm replayed the same selected records and logical requests for three calls, but its third response differed and request 4 consequently had no matching frozen digest. A subsequent 100% audit found that this was not yet a pure prefix-cache factorial: cache-on continued the live full-prefix slot, whereas cache-off routed the lossless selected history through detached resource materialization. We therefore retain the run only as an implementation diagnostic and exclude its partial score and runtime ratio.

The repaired adapter treats 100% retention as a semantic no-op. If no live slot is consumed, it validates and reconstructs the complete ordinary chat prompt and cold-prefills that prompt directly; it does not enter detached resource materialization. With llama.cpp revision 458681e1, batch 256, microbatch 1, and Qwen3-Coder-30B Q4_K_M, a frozen 30-turn trajectory then matched cache-on and cache-off on all 30 continuations while all 29 eligible cache-on requests reported physical hits. More importantly, the autonomous task-01 control now matches on all 24 logical-request hashes, ordered-selection digests, and complete responses. Both arms emit the same 689-byte, 14-line patch and resolve under the official grader. Cache-on retains 120,848 of 126,772 physical prompt tokens (95.3%) and completes the agent in 100.3 s, versus 2,792.5 s cache-off, a 27.85 $\times$ single-task speedup. This is a qualified strict *prefix-cache versus cold-prefill* control, not a PRA speedup: 100% retention selects no subset and the cache-on arm remains on its ordinary live slot.

Agent-history K/V correctness reset. We stopped the Easy-50 treatment matrix after recognizing that the composite resource design violated the product invariant. For an agent trajectory, PRA must select cells already encoded in the session’s live prefix cache; it must not tokenize and prefill the selected transcript again. The replacement llama.cpp path names one pinned live source sequence plus multiple stable record ranges. Unified-cache `seq_cp` adds request membership to those existing cells without copying K/V, preserves their original positions, and removes request membership without deleting source membership. Newly evaluated query and generation cells are committed back to the source. The final sampled token, which llama.cpp returns before evaluating it into K/V, is explicitly reported as pending and bridged in the next suffix.

On an M4 Pro with Qwen2.5-0.5B-Instruct Q4_K_M, a 20-turn same-state gate forked the control and treatment from identical source K/V on every turn. All 20 continuations matched token-for-token as the source grew from 11 to 343 tokens; zero selected tokens were re-encoded and every suffix commit succeeded. The record-aware agent wrapper then exercised a genuinely sparse turn: from a 531-token live source it attached 26 K/V tokens (4.90%), evaluated none of the selected history text, copied no K/V, and returned the pinned key correctly. A separate rejected-action path removed the discarded branch by rolling the source from 27 to its 23-token common prefix, then committed the recovery suffix with zero selected-text re-encoding and returned the correct recovery answer.

The first current-build real-agent gate uses predeclared baseline-success task 02, `django_django-15368`; task 01 was not substituted after both its matched current plain and selected-K/V executions became 0/1 under the revised binary. The task-02 ordinary-prefix-cache control and the 100% live-K/V arm both resolve 1/1 in 14 model calls, emit the same 673-byte, 13-line patch, and match on all 14 assistant responses byte-for-byte. After the initializing request, the live arm continues the canonical same-slot sequence on every turn: 54,686 resident K/V token-exposures are reused, only 2,996 wire-suffix tokens are evaluated, selected-history text is never re-encoded, and no physical K/V copy occurs. Wall time is 141.4 s for the plain control and 184.1 s for the instrumented live arm, so this row establishes correctness, not speedup. An earlier replay of task 01 also matched both controls through 19 responses, then failed closed because an `ls -la` observation carried different file timestamps and therefore a different exact request hash. That is a frozen- observation harness defect, not a model-response divergence; replay fixtures must freeze tool observations or key them by stable causal identity.

At 90% budget the same task passes through four exact full-retention actions. The first sparse request attaches 3,047 of 3,221 resident source tokens and drops the oldest completed causal turn while retaining the task, active tail, and two most recent turns. Response 5 changes in wording but issues the same command; the subsequent trajectory edits the intended line, then submits an invalid 503-byte patch after 12 calls and resolves 0/1. The run realizes only 10.2% estimated logical-context saving. This is the first measured sparse trajectory failure on the corrected K/V mechanism, but it is not yet attributed solely to selection. In the frozen consumption control, live K/V and fresh prefill match exactly on the four full-retention requests. At the first sparse request, compact fresh prefill consumes 3,543 tokens while the live-K/V request spans 3,717 logical positions: the 174 omitted tokens leave positional holes because the native path deliberately preserves original record coordinates. The responses differ slightly in reasoning text but issue the same command. Thus “identical selected records” is insufficient to isolate K/V construction unless positional presentation is fixed too.

A position-preserving fresh-evaluation control now closes that narrower correctness question. It cold-prefills the frozen 3,221-token source, attaches the identical 3,047 selected K/V entries to a clean request sequence at their original positions, and evaluates the same 496-token wire suffix. Its response is byte-for-byte identical to the live-K/V response at the first sparse request (matching SHA-256 `f9b4fceb64011a3c79a0c607f8e0b5efec23e3a32c89ea0d9db6d66b8cd45cdf`). Consequently, the native path is reusing the intended K/V subset without selected-text re-encoding or K/V-construction drift. The earlier compact-fresh difference is a positional-consumption effect. The task-level 0/1 result

can therefore be assigned to the sparse treatment as a whole, but selection loss and original-position versus compact-position consumption remain separate factors for the next frozen factorial.

The paired stochastic control strengthens this single-seed result. After an exact replay of the four source-building turns, five predeclared sampling seeds (101, 202, 303, 404, and 505; temperature 0.7; 128-token continuation cap) fork from the same 3,047 selected cells. Each live-built continuation is byte-for-byte identical to its position-preserving cold-built counterpart; the five seeds produce five distinct response hashes, and all five pairs match. This clears the multi-seed live-versus-fresh K/V-construction gate for this model, request, and physical slot geometry. It does not establish quality at 90% retention, because both arms consume the same sparse state.

Cross-model replication exposes a necessary qualification dimension. With Qwen3-14B Q4 and its default model template, turn 2 fails closed before inference because the template inserts an empty thinking wrapper when an assistant record is final, then removes that wrapper after a later user record arrives. The completed assistant record is therefore not token-prefix-stable, and its old K/V cannot represent the next transcript. With llama.cpp’s stable ChatML template, the same probe builds a four-turn causal source twice with identical response hashes, attaches 3,069 of 3,205 source cells, and obtains byte-identical live-built versus cold-built continuations for sampling seeds 101, 202, and 303. Model identity alone is consequently insufficient: agent-history qualification binds the model, tokenizer, chat template, and engine geometry. The direct endpoint now runs a four-record prefix-stability probe at startup and rejects an incompatible template before the first costly agent prefill.

That control exposed a separate occupancy confound. With both request slots empty, two frozen full-transcript prompts produce identical response hashes under cache-on and cache-off execution. Replaying the exact physical selected- record payloads is also exact. If the native readiness probe first leaves an unrelated seven-token resource pinned in sequence 0, however, the cache-off response to prompt 2 changes reproducibly even though sequence 1 is erased, reports zero cache hits, and receives the same 1,857 prompt tokens. The changed hash exactly reproduces the earlier apparent cache mismatch. This is unified- KV physical-occupancy sensitivity, not evidence that the selected transcript or ordinary prefix cache is wrong. Readiness probes are now ephemeral and release their resource sequence before measurement.

The first concurrent sparse-session attempt then exposed a related ownership bug rather than clearing the gate. The wrapper assigned canonical histories to distinct source slots, but built them as ordinary prefix-cache requests without setting llama.cpp’s persistent PRA pin. Under unified-KV scheduling, `try_clear_idle_slots()` consequently purged idle source slot 1 while the other session was active: the slot-3 request attached correctly, whereas the slot-1 request failed closed with “source has no encoded context.” The fix marks only canonical live-history sources with `pra_pin_resource`; ordinary no-PRA controls remain unpinned.

After that repair, two Qwen3-14B/ChatML sessions independently built three-turn, 1,262-token live histories in source slots 1 and 3. Their simultaneous 75%-budget requests used destination slots 0 and 2, each attached five records and 662 existing K/V cells, re-encoded zero selected-history tokens, and copied no K/V. Both complete source trajectories and both sparse target responses were byte-identical to a separately rebuilt sequential control. Concurrent target wall time was 182.9 s versus 296.0 s sequential, but this 1.62 $\times$ observed scheduling ratio is diagnostic only: an unrelated virtual machine and endpoint security substantially contended for the host during the run. The result clears cross-session slot ownership and behavioral-isolation correctness for this qualified configuration; it is not a sparse-runtime speed claim.

The small-model rows and the task-02 result are mechanism and single-task correctness gates, not Easy-50 population-level efficacy evidence. A strict frozen replay now resubmits the successful task-02 arm’s recorded physical payloads and observations: all 14 assistant responses, including mutation, verification, and submission turns, reproduce byte-for-byte. This removes live filesystem timestamps and other tool nondeterminism from the replay verdict and clears the single-task frozen-agent gate.

Together with the qualified Qwen3-14B replication, this clears the current single-task, multi-seed, cross-model construction, and concurrent-session isolation gates. The wrapper additionally serializes same-session requests, makes exact transport retries idempotent, rejects a different generation from the same logical head as a commit conflict, applies resource deltas transactionally, and tombstones a session before draining and erasing it. A direct Qwen3-14B probe confirms those semantics: an exact retry returns the cached identical response; two simultaneous requests from one logical parent produce exactly one HTTP 200 commit and one HTTP 409 conflict; DELETE completes in 4.4 ms after the winning request, and later traffic receives HTTP 410. This clears retry, same-session commit, and completed-request termination. It does not clear active cancellation by itself. To close that gap while preserving the public non-streaming API, the wrapper consumes llama.cpp’s completion stream internally, retains the live upstream response handle, and closes it when the session DELETE arrives. A repeated direct probe observes the upstream stream, returns HTTP 499 `generation_cancelled` to the interrupted request, returns HTTP 200 to DELETE in 1.54 s, and records llama.cpp cancelling and releasing the active task.

The eviction/offload gate uses a one-pair Qwen2.5-0.5B configuration so slot reuse is unavoidable. The wrapper saves a 24-token canonical sequence as a 295,916-byte llama.cpp slot image, erases both sequence memberships, and the next pressure session occupies the released source slot. Restoring the first session into that same pair, with its PRA pin set atomically by the restore operation, yields a byte-identical continuation to the never-offloaded control. No selected text is submitted for prefill, and the checkpoint file is deleted after successful restore. This clears explicit disk demotion, capacity reuse, restore identity, and cleanup for the tested runtime.

That final mechanism gate uses ten alternating-order pairs on the same small model and fixed synthetic trajectory. Ordinary full-prefix continuation reads all 1,495 resident cells; sparse live-K/V attaches 391 cells (26.2%) across five causal records, with zero selected-text re-encoding and no K/V copy. The one-token target response is identical in every pair. Median engine prompt time falls from 174.0 to 161.4 ms ($1.08\times$ ratio of medians; $1.09\times$ median paired ratio), while endpoint wall time falls from 294.7 to 266.7 ms ($1.10\times$; $1.11\times$ paired); sparse wins all ten prompt and wall pairs. This qualifies an engine-specific sparse runtime benefit at this geometry, not an Easy-50 task-quality or large-model speedup. The llama.cpp live-agent path therefore clears the construction, concurrency, lifecycle, offload, and sparse- runtime gates. A campaign-level lock prevents the resumable Easy-50 scheduler from launching treatment cells before that status is explicitly promoted. The gate is now **QUALIFIED** only for a llama.cpp wrapper with prefix caching enabled and a configured slot checkpoint directory; cache-off wrappers and HF, MLX, vLLM, and SGLang remain unqualified. The restarted efficacy sequence operates on the 14 baseline-success instances, one instance at a time: each instance must first pass the 100% semantic-no-op control before its 90% treatment can run, and lower retention remains contingent on that result.

9.6.2 Claim 4: observed coding-agent utility frontier

The restarted task-01 row illustrates why that protocol separates mechanism parity from current task capability. Its 100% live-K/V execution makes 30 model calls, re-encodes zero selected-history tokens, copies no K/V, and reuses 234,035 resident K/V token-exposures. A frozen ordinary-prefix replay of the same recorded requests and tool observations reproduces all 30 assistant responses byte-for-byte. The independently autonomous plain run instead matches the first 22 responses and first differs after an `ls -la` observation carries a changed directory timestamp; with frozen observations, that apparent divergence disappears. Both autonomous arms nevertheless score 0/1 because this current model/binary pairing submits explanatory prose rather than an applicable unified diff. We classify that as a matched capability failure, not a PRA regression, and do not spend a lower-retention efficacy row on it. The sequence advances to the next instance that reproduces its admitted baseline solve;

historical membership in the 14-task stratum and current paired success are reported separately.

Task 02 then provides the first complete restarted efficacy pair. Its fresh 100% arm again resolves 1/1 in 14 calls with a 673-byte patch, 54,686 resident K/V token-exposures, zero selected-history re-encoding, and no K/V copy. The frozen ordinary-prefix control again reproduces 14/14 responses exactly, with the same response hashes as the earlier qualification run. The admitted 90% arm again resolves 0/1 in 12 calls and submits the same invalid 503-byte patch shape. Across eight sparse requests it records 37,793 selected K/V token-exposures, zero selected-history re-encoding, no K/V copy, and only 10.16% estimated logical-context saving. Replicating both outcomes strengthens the assignment of the loss to the sparse treatment rather than a transient engine defect. Because the predeclared 90% quality gate fails, lower-retention task-02 rows are withheld and the sequence advances to task 03 at 100%.

Task 03 also clears its 100% gate: it resolves 1/1 in 13 calls with a valid 716-byte patch, while its frozen plain-prefix replay reproduces all 13 responses byte-for-byte. At 90%, it still resolves 1/1 and emits the same patch, but requires 25 calls—a 92.3% action increase—while reducing estimated logical context by 17.75%. Its 21 sparse requests expose 186,379 selected resident K/V tokens while re-encoding zero selected-history tokens and copying no K/V. This is a quality pass but a consumption-efficiency warning: preserving the final answer does not imply preserving the path used to reach it. The observed 497.9-versus-940.4 s wall times are retained only as contended provenance and are not a speed comparison. Task 03 therefore advances to 75% retention. At 75%, it resolves 0/1 after 23 calls and submits an invalid 479-byte patch while reducing estimated logical context by 23.23%. Its 19 sparse requests still re-encode zero selected-history tokens and copy no K/V, so this is a task-quality failure on a physically correct mechanism. Profiles below 75% are consequently withheld for this instance, and task 04 becomes the next 100% gate.

This also changes the comparison with ordinary prefix caching. Prefix caching can reuse only the common leading run and must either keep every middle turn or evaluate a replacement suffix when selection changes. Live-record PRA keeps the canonical full session once, adds request membership to selected existing cells, and in the sparse smoke exposes only 26 of 531 source cells to the new request. Thus a selection change no longer implies thousands of re-prefill tokens. It may still change the number of K/V cells read by attention and the amount of resident memory, so four quantities remain disjoint: logical token retention, selected K/V attachment, newly evaluated tokens, and wall time. The first three now have physical telemetry; no speedup is claimed until a matched agent run shows that reduced K/V reads dominate membership and residency costs.

The same frozen agent history also shows why transcript rematerialization cannot be assumed equivalent across engines. Table 37 reports diagnostic token parity; continuation scopes differ and the rows are neither speed nor quality rankings. The bounded HF arm is exact despite small logit changes, while MLX, vLLM, and SGLang each expose prompt-dependent token changes. MLX precision is non-monotonic, so quantization alone does not explain the effect. Product adapters must preserve the live K/V state that generated the trajectory, or explicitly qualify a strict rematerialization mode; they must not silently substitute one for the other.

Table 37: Frozen real-agent cache/rematerialization diagnostics. “Scope” is the number of agent turns and continuation cap. Different models and scopes make these correctness probes, not engine comparisons.

Engine	Model	Scope	Exact turns	First divergence
llama.cpp 458681e1	Qwen3-Coder-30B Q4	30, ≤ 4096	30/30	—
HF 5.17 / PyTorch 2.13	Qwen2.5-1.5B BF16	6, 8	6/6	—
MLX 0.31.3	Qwen3-0.6B BF16	10, 16	6/10	2
MLX 0.31.3	Qwen3-0.6B 8-bit	10, 16	9/10	6
MLX 0.31.3	Qwen3-0.6B 4-bit	10, 16	8/10	7
vLLM 0.29.0 V1	Qwen2.5-1.5B BF16	10, ≤ 256	5/10	4
SGLang 0.5.19	Qwen2.5-1.5B BF16	10, ≤ 256	3/10	2

Task 04 clears 100%: 1/1 in 13 calls, a valid 875-byte patch, 59,325 resident K/V exposures, zero re-encoding or copy, and 13/13 exact frozen responses. Its 90% arm also resolves 1/1, but takes 34 calls—21 more than the 100% arm— while reducing estimated logical context by 11.80%. Across 30 sparse requests it exposes 248,815 selected resident K/V tokens, re-encodes no selected history, and copies no K/V. As on task 03, end-task success therefore conceals a large interaction-efficiency regression; contended wall time is provenance only.

9.6.3 Claim 3: agent retention semantics

The first-divergence audit also rules out ordinary sampling temperature as the proximate explanation: both task-03 and task-04 arms sent `temperature=0`. At the request producing the first changed executed action, both nominal-90% selectors omit exactly logical messages 4–5: the assistant command that displayed the target source file and its full-source observation. Because the old whole-bundle packer treated 90% as a ceiling, the 1,151- and 1,152-token bundles exceed their remaining budgets by only 224 and 167 tokens but are dropped completely. Realized retention collapses to 62.01% for task 03 and 57.72% for task 04. The corrected floor policy keeps the bundle and realizes 97.39% and 96.15%, respectively. In task 03 the request-6 logical prompt is byte-identical to the existing full-history prompt; in task 04 response wording has already diverged at request 5 although the executed command is unchanged.

A frozen restore-only counterfactual on the same 48 GiB M4 Pro now makes this localization causal at the next-action level. For each task it compares the recorded sparse request-6 prompt, that prompt plus only messages 4–5, and its matched full logical request, with three temperature-zero repeats. On task 03, the sparse condition chooses a whole-file read in 0/3 matches, whereas restoring only the source-view action–observation bundle recovers the recorded 100% targeted `grep` command in 3/3. On task 04, the sparse condition again chooses a whole-file read; restoration changes it to a target-specific `grep` in 3/3, but does not recover the separate 100% trajectory’s immediate `sed` edit. This is strict causal recovery for task 03 and directional recovery for task 04: under task 04’s frozen 90%-trajectory request-5 wording, source evidence is important but insufficient. Commands are deterministic in every cell, although task-03 reasoning wording changes once without changing its command. These K/V counters describe ordinary contiguous llama.cpp prefix-cache reuse during repeated prompt controls, not sparse PRA attachment. A target-bearing 256-word child span exists in each large observation, motivating both a `recent_source_turns` floor and hierarchical within-record selection rather than either dropping or retaining the entire source view.

9.6.4 Claim 2: engine portability contract

The cross-engine expansion follows the same qualification ladder already required of llama.cpp. Its first rung requires that ordinary and PRA continuations fork from the same engine-resident K/V, 100% selection is a semantic no-op, selected history is never tokenized or prefilled again, source positions and stable causal-record identities survive, and request membership does not mutate source ownership. HF and vLLM-Metal now clear request-owned lifecycle controls; the bounded vLLM CUDA scheduler alias clears them in its declared scope. The newest MLX and SGLang disjoint-consumption candidates also clear their declared numerical, allocation, and lifecycle gates through the shared interval-addressed Metal consumer described below. The earlier rows in Table 37 compare incremental execution with an independently reconstructed cache and consequently remain rematerialization diagnostics, not this qualification.

The corrected same-state probes pass for HF and MLX. On the frozen task-02 agent transcript, Transformers 5.17/PyTorch 2.14 with Qwen2.5-0.5B-Instruct FP16 on MPS and MLX-LM 0.31.3 with Qwen3-0.6B 4-bit each match the ordinary continuation on all 10 tested turns with zero maximum logit delta. Both paths select the complete resident cache as a view, report zero selected-history re-encoding, and copy no K/V. A subsequent requested-90% sparse-position probe supplies stronger but deliberately separate evidence. Across seven eligible turns per engine, HF and MLX both match an independent reference consuming the identical selected K/V and original source positions: 7/7 exact token sequences and zero maximum logit delta, with no selected-text re-encoding. A corrected causal-floor rerun at the same repository revision also passes 7/7 for each engine. Its selected/source ratios range from 90.32% to 95.66% (mean 92.88%), so the nominal 90% setting is now a floor rather than the 67.1–94.4% ceiling behavior of the superseded planner. The packed reference paths clear semantic sparse-position correctness and cross-engine logical-selection agreement; selection, consumption, and lifecycle are evaluated separately below rather than inheriting qualification from those results.

The corrected vLLM-Metal bridge now runs matched release packages: vLLM 0.29.0+cpu and vLLM-Metal 0.29.0, with native shared-library and Metal-library hashes captured from the active environment and source revision 73908058. Full retention matches 10/10 turns, and the seven eligible requested-90% turns match an identical selected-page reference. A real request-owned lifecycle gate on task-02 turn 4 retains 2,944/3,104 page tokens (94.85%) with holes and original position extent 3,104. All 24 checks pass: two requests alias the exact source block IDs, active memory does not increase, request attachment records zero copy events, selected history is not re-encoded, finish/cancel/error release is exact-once, stale generations are rejected, and termination and tombstones are enforced. Lossless offload and restore move 712,002,099 bytes and preserve the source fingerprint and output; that residency transfer is reported separately from zero-copy request attachment.

The first vLLM 0.28 CUDA worker-side prototype was useful but copied 804,651,008 bytes across the seven-turn frozen probe. Source audit localized the problem: an external connector match returns only a token count, after which `KVCacheManager` ordinarily allocates a fresh request table. The replacement integration therefore runs one boundary earlier, at `Scheduler.get_local_prefix_cache_hit` before `allocate_slots`, and installs the source request’s existing `KVCacheBlock` objects in the authoritative scheduler table. On an RTX 5060 with vLLM 0.28 and Qwen2.5-1.5B, task-02 turn 4 retains 2,944/3,104 tokens (94.85%); two consumers are same-subset exact and K/V-copy, host-to-device, and selected-history-re-encoding counters are all zero. A separate three-page mechanism smoke exercises concurrent completion, real cancellation, injected scheduler error, stale-generation rejection, borrowed and deferred eviction, and exact-once release: five aliases are installed and five are released, with final page reference counts zero. This is a bounded zero-copy result, not a general vLLM claim: it requires in-process V1, complete pages, and one homogeneous K/V group. Partial pages, hybrid groups, multiprocess scheduler/worker execution, the full seven-turn trajectory, and runtime economics remain unqualified.

SGLang-MLX likewise keeps selected-history re-encoding at zero while preserving Radix/source-

owner isolation, borrower-safe removal, eviction and offload, exact-once terminal release, stale-generation rejection, restoration, and tombstones. Its replacement consumer is a fused interval-addressed Metal kernel. Rather than passing six selected slices as independent buffers, the kernel receives the canonical source K/V once plus a compact interval table, keeps original source positions in the cache contract, and performs one online softmax while reading only admitted ranges. For 2,907/3,081 selected tokens (94.35%), selection pack bytes remain zero. First-layer peak delta falls from 12,199,774 to 19,371 bytes, only 0.163% of the 11,907,072-byte selected layer, and the candidate is final-logit exact against a packed-value oracle preserving the same segment boundaries and using the identical consumer. All combined consumption and lifecycle checks now pass. This qualification remains bounded: SGLang’s multi-request path evaluates each selected row separately and rejects a batch that mixes ordinary and disjoint cache protocols.

The shared lifecycle registry has been exercised over real task-02 selected cache objects for HF, MLX, and SGLang. Two borrowers share one canonical object; eviction is rejected while borrowed; cancelling one borrower preserves the other; idle objects offload; lossless restore reproduces their fingerprints; active termination cleans up; and tombstones prevent stale recreation. These ownership results remain independent of consumer numerical and allocation gates even though all three declared scopes now pass them.

HF wires ownership into its synchronous model request and passes concurrent borrowing, exact-once finish/cancel/error release, stale-generation, offload/restore, termination, and tombstone controls. On Qwen2.5-1.5B FP16 on the RTX host, task-02 turn 4 retains 2,907/3,081 tokens (94.35%), preserves original positions, re-encodes no history, and uses zero persistent interval- pack bytes. The corrected two-pass consumer scales the request-local query, uses native SDPA on each storage-alias K/V view, and combines their probability mass in FP32. It casts or copies no selected K/V: cumulative K/V scratch and maximum K/V tile bytes both fall from 337,125,376 and 524,288 to zero. Bounded score/probability state remains explicit at 34,062,336 logical bytes. Against a packed-value oracle with the identical causal segments, positions, and consumer, maximum logit delta is 0.0, clearing the predeclared 0.001 gate. The path is correct but unfused; no latency advantage is claimed.

MLX-LM uses the same Metal consumer directly on Qwen3-0.6B 4-bit. For task-02 turn 4, six intervals retain 2,907/3,081 tokens with zero selected-history re-encoding, zero selection pack bytes, and zero allocator delta. The kernel addresses those intervals in the original canonical arrays and reduces first-layer peak delta from 12,511,756 to 19,371 bytes (0.163% of the 11,907,072-byte selected layer). The packed-value matched-consumer oracle is final-logit exact (0.0 versus the 0.005 gate), and ownership, exact-once release, stale-generation, offload/restore, termination, and tombstone checks all pass. Lossless offload separately serializes 706,726,451 bytes. The earlier 1.0 cross-consumer logit delta remains a useful warning: changing from MLX’s dense SDPA to a different reduction while also changing storage did not isolate selection correctness.

This correction adds a fifth entry to the superseded-evidence ledger. A same-subset comparison must hold the consumer fixed. The older HF, MLX, and SGLang rows simultaneously changed storage realization and attention reduction; their logit deltas therefore diagnose cross-kernel rounding, while their measured K/V copies and transients remain valid implementation defects. The replacement oracles are intentionally allowed to pack values, but preserve the candidate’s logical segment boundaries and original positions and execute the same consumer.

The agent SDK now carries a typed, request-scoped retention contract under `pra.metadata.retention_policy`. It controls the minimum number of recent complete turns, the minimum newest records retained inside dense turns, separate source/progress/mutation/verification recency floors, the oversized-record chunk target, the dense-turn threshold, causal action–observation pairing, and causal-bundle round-up. Short records remain intact; large tool observations split at natural file, test, stack, diff-hunk, or line boundaries; child spans retain stable parent-record and causal-group identities. This makes target and realized retention distinct protocol fields rather than undocumented proxy behav-

ior. The new `recent_progress_turns` control conservatively pins explicit diagnoses, hypotheses, root causes, and proposed fixes together with their causal observations; generic verbose reasoning is not promoted. A frozen task-04 regression verifies that its exact request-5 progress state and latest source evidence both survive the 90% policy.

9.6.5 Claim 4: locked next-stage matrix

After engine qualification, the locked task-level matrix contains no gateway arm: No PRA, 100% PRA, and 90% PRA. PRA is crossed with an orthogonal adaptor condition (no adaptor versus an immutable model-adaptor bundle) while holding the selected records, retention budget, and physical materialization policy fixed. Exact no-adaptor parity at 100% is an implementation gate; a learned adaptor is an explicit model intervention and is judged by its frozen identity and task outcome rather than assumed to be response-identical. The two 75% rows (without and with the adaptor) are admitted independently only when all three predeclared tasks pass at 90% in their matching engine, selector, materializer, and adaptor cell.

The executable Easy-3 card freezes the first three historical baseline-success instances and expands them task-major into 15 cells: one No-PRA control and four PRA cells (100% and 90%, each without and with a frozen adaptor) per task. Temperature is zero, seed is zero, top- p is one, the direct llama.cpp engine runs on the 48 GiB M4 host, and mini-swe-agent runs separately. Receipt preparation yields nine configuration-qualified no-adaptor cells and six blocked adaptor cells; no task execution is counted by this preparation step. Here “adaptor” has the narrow scientific meaning of a learned causal-record routing/selection model applied before the K/V subset is frozen. Structural model mapping, prompt guidance, memory-consumption LoRA, and frozen replay are not adaptor arms. Because no certified Qwen3-Coder-30B bundle or typed injection path exists, the six learned-adaptor cells fail closed rather than silently falling back to the no-adaptor policy.

The first 90% retention-window diagnostic keeps two mutation and two verification turns fixed while varying completed-turn recency. The H10 and H5 arms reproduce the same first seven commands; H5 first removes history on request 8 and immediately takes a different, unproductive source view. H4 first selects on request 7, and H3 on request 6. H10 is the strongest selection candidate: it reaches the target edit and realizes 10.0% estimated context saving, but then repeats source inspection instead of verifying or submitting. H5 also enters a discovery cycle, while H4 and H3 were stopped once their boundary was localized. These intentionally terminated diagnostics are not solve-rate rows. They show that shortening the history window does not repair the task and can lose progress earlier; the next controlled variable is consumption policy—post-mutation diff inspection, focused verification, and submission after a successful check—rather than more selector training.

Exact interaction-history audit localizes the reduced-context failure more sharply. The legacy 100% live-prefix-cache arm is response-for-response identical to the successful plain trajectory. In every legacy active-tail arm, however, the first changed command is action 4: the selector retains the five task segments and current action–observation tail but drops the first two completed causal turns. The plain agent advances to locating `Value`; the treated agent re-inspects `CharField` and later repeats similar source views. Thus the early loss is a progress-memory selection bug, not a chat-template failure. Because the reduced arms rematerialized selected text, this audit does not test the replacement selected-live-K/V mechanism.

We repaired that policy with a v4 *progress spine*. In addition to the pinned task and active tail, it pins the two latest completed causal turns and the latest source-content, mutation, and verification turns, queries retrieval with both the task and active state, and restores all records to causal presentation order. Source-content means the latest causal bundle whose command actually displays file contents (for example `cat` or `sed -n`); a later path-only `find/grep` discovery does not displace that evidence. This new count is exposed and recorded as `recent_source_turns`; applying value 1 to the

frozen task-03 and task-04 request-6 payloads retains messages 4–5 in both cases. Autonomous and one-bundle counterfactual replays remain required before attributing recovery to this change. All 31 baseline request prefixes again pass the physical Qwen chat template at 75%, 50%, and 25%. On the same admitted task, v4 at 75% resolves 1/1: the first four commands remain exact, action 5 merely takes a narrower source view, a failed stale-line edit is recovered, the patch is created at action 31, and the exact submission occurs at action 33. Estimated logical-context saving is 22.3%. By contrast, v4 at 50% resolves 0/1 after 50 actions with no submitted patch despite 45.9% estimated saving. Its selected records contain the exact earlier **CharField** path, yet the model launches redundant whole-tree searches and later corrupts a method boundary while editing. This is a consumption/edit-execution failure with the required state present, so the 25% arm is withheld by the predeclared quality gate.

The v4 runs also expose the rematerialization boundary. At 75%, 135,757 selected resource-token exposures require 57,754 newly evaluated tokens while 78,003 tokens hit exact composite-prefix deltas; no physical K/V copy is observed. These are not selected-live-K/V reuse metrics. Nevertheless, the run takes 608.0 seconds under severe host memory pressure, so it is not admitted as a latency improvement over the earlier plain run. The runtime now writes a paired JSONL history for every call containing the exact logical payload, transformed physical payload, selected-resource digest, and model response. This makes future plain, G00, direct-native, and G11 deviations attributable at the first differing interaction rather than only at the final SWE-bench score.

That trace then exposed two G11 integration faults before they could be counted as task failures. First, the llama.cpp wrapper advertised resource-delta support but rendered only the newly supplied ADD/UPDATE bodies; on request 3 the native prefix therefore collapsed from the complete 2,576-token trajectory to the 565-token new turn and the response diverged. The wrapper now retains a tenant-session-model resource inventory, reconstructs every ordered active record before chat rendering, and fails closed when an unchanged body is missing. Second, G11 treated the intentional movement of completed chat turns from inline messages into detached records as a destructive history rewrite. It closed the live engine session, preserving outputs but losing the 2,293, 2,576, and 2,684 cached tokens observed on requests 2–4. The selected-history projection is now explicit; G11 retains that session while the native adapter still compares exact token IDs and backtracks safely on a genuine rewrite.

The corrected frozen G11 replay is exact against the direct composite-resource path on all 33 requests: logical-request digests, ordered selected records, response text, prefix-hit state, cached-token counts, native-token counts, wire-token counts, and the no-K/V-copy flag all match. Aggregate engine accounting is consequently identical: 135,757 selected-resource token exposures, 57,754 evaluated resource tokens, 78,003 resource-prefix hits, 7,553 sequential-prefix hits, 143,310 native tokens, and 8,950 wire tokens. It emits the same 689-byte patch and the official grader resolves the task, for direct/G11 transport parity at 1/1 and 22.3% estimated logical-context saving. The grader initially attempted to repull the x86-only evaluator image as ARM after agent cleanup and failed before patch application; regrading the unchanged prediction under the declared `linux/amd64` platform resolved it. The runner now propagates that platform contract to both agent and grader. Because the G11 run took 1,189.7 seconds on the already swap-saturated host versus 608.0 seconds for direct, no gateway latency ratio or native selected-K/V claim is admitted from this pair.

Table 38: Corrected G00 end-task parity gate on three tasks selected because the direct baseline solved them. Equal 3/3 outcomes validate transport semantics; the selected cohort and differing trajectories preclude accuracy, token-saving, or latency claims.

Path	Resolved	Model calls	Prompt tokens	Completion tokens
Direct Ollama	3/3	63	458,690	8,573
PRA Gateway G00	3/3	69	403,703	9,503

We therefore preregistered a stronger controlled SWE-bench campaign from an independent local-model study [2]. The checked-in benchmark card preserves its ordered seed-0 cohort of 50 SWE-bench Verified instances at source revision `8f894c2`; a canonical LF-joined digest `20acb5f7...1818131` prevents accidental resampling. The source used mini-swe-agent 2.4.0 with its unmodified text/backticks scaffold, SWE-bench 4.1.0 Docker grading, vLLM 0.22.1, 16,384 model tokens, 40 steps, temperature zero, BF16 weights, and FP8 K/V on one H100 80 GB. Its reported controls are 7/50 for Qwen3-Coder-30B-A3B-Instruct and 19/50 for Gemma-4-31B-it. These are external reproduction targets, not PRA results.

The executable contract keeps both reference cells and all treatments disabled until a qualifying host is recorded. The currently reachable local CUDA device has 4 GB, and the known 48–64 GB Apple hosts cannot reproduce vLLM/H100/BF16. Moreover, the source does not publish immutable model/tokenizer revisions, package hashes, or grader-image digests. The runner records these as source-provenance limitations, separately from actual local configuration deviations. Current Qwen and Gemma HF revisions are pinned for the new run, as are installed package-manifest digests. The campaign can test reproduction of every reported setting and score, but it cannot claim binary identity with the historical artifacts. Any actual configuration deviation still leaves the cell partial and locked. After an admitted Gemma result of at least 20%, the scheduler orders gateway pass-through, matched truncation, and PRA at 50%, 25%, and 12.5% physical-context budgets. The first ten ordered IDs separately define a BF16/Q8/Q6/Q5/Q4 precision diagnostic; its result cannot be reported as fixed-50 reproduction evidence. The treatment path is nevertheless executable rather than a placeholder. A small OpenAI-compatible proxy leaves B1 payloads unchanged. For matched truncation and B2 it keeps system messages and the current user observation mandatory, then allocates the same residual budget to prior agent-visible history. Truncation keeps the recent suffix; B2 divides earlier messages into bounded records, ranks them with typed, BM25, and deterministic embedding channels under reciprocal-rank fusion, and sends the selected records through the gateway’s G10 text-realization path. A recency fallback fills only unused budget. Request receipts separate logical, selected, physically visible, and avoided whitespace-token estimates from engine-reported tokenizer counts and timings. The normalizer also reads mini-swe-agent’s agent-visible trajectory files: it records exact cumulative prompt and completion tokens, maximum prompt tokens, a conservative repeated-context estimate, model and tool calls, trajectory duration, termination, and patch size. Stable task hashes join these rows to request traces without benchmark labels. TTFT and prefill remain null when the endpoint does not expose them. This implementation readiness is not treatment evidence while the baseline gate is closed.

Table 39: Canonical coding-agent admission evidence. Qwen treatment conditions were withheld when their No-PRA cohorts remained below the configured quality floor. Easy-20 is the first admitted local baseline; its treatments use the nested Easy-50 cohort and are reported separately. The nonzero FIM-7B smoke remains withheld because its engine, precision, cohort, and context differ from the published baseline. Deltas remain blocked rather than being computed across inadmissible cells.

Cohort	Metric	No PRA	Selected Context	Selected + bundle
Qwen3-14B, five tasks	Official success	0/5	BLOCKED	BLOCKED
Qwen3-14B, five tasks	Verifier checks	7/12	BLOCKED	BLOCKED
Qwen3-14B, five tasks	Input tokens	100,883	BLOCKED	BLOCKED
Qwen3-Coder-30B, one task	Official success	0/1	BLOCKED	BLOCKED
Qwen3-Coder-30B, one task	Verifier checks	4/6	BLOCKED	BLOCKED
Qwen3-Coder-30B, 30 trials	Official success	2/30	BLOCKED	BLOCKED
Qwen3-Coder-30B, 30 trials	Unique tasks solved	1/10	BLOCKED	BLOCKED
Qwen3-Coder-30B, 30 trials	Input tokens [†]	6.72M	BLOCKED	BLOCKED
Qwen3-Coder-30B, Easy-20	Official success	9/20	ADMITTED	ADMITTED
Qwen3-Coder-30B, Easy-20	Input tokens	4.29M	ADMITTED	ADMITTED
Qwen3-Coder-30B, Easy-50	Official success	14/50	PENDING	PENDING
Qwen3-Coder-30B, Easy-50	Input tokens	10.12M	PENDING	PENDING
FIM-7B Q4, two tasks	Official success	1/2 attempted	BLOCKED	BLOCKED

[†]Reported by 20/30 trials.

9.7 Distributed observability and semantic context telemetry

The product runtime now carries one W3C trace context across the agent, gateway, PRA realization path, and inference adapter. This is not a new coupling between those services: each remains a distinct OpenTelemetry service (“**pra-agent**”, “**pra-gateway**”, and “**pra-runtime**”), and an engine-native provider contributes child spans only when its pinned integration explicitly enables them. Engines with Prometheus but no native tracing are scraped directly; engines exposing only timing APIs receive wrapper spans and normalized metrics. Thus common PRA telemetry does not pretend that every engine exports the same physical facts.

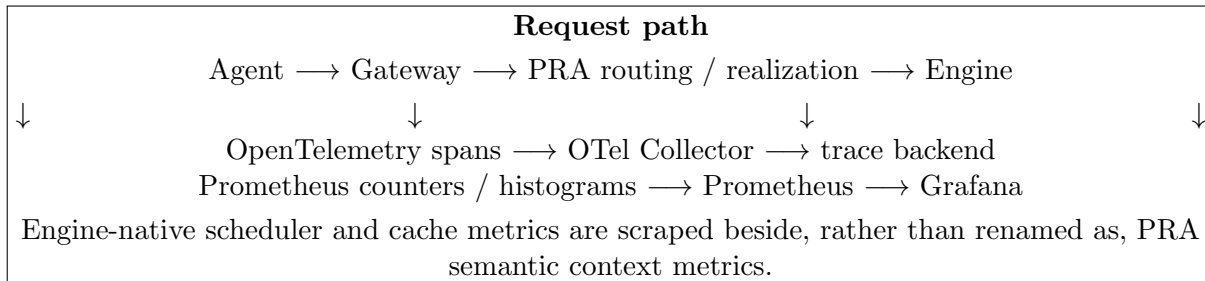


Figure 33: The observability boundary. W3C context joins request causality, while Prometheus and Grafana correlate semantic context decisions with engine-native serving behavior.

A representative trace is rooted at **pra.agent.turn**; its **pra.gateway.request** child contains session resolution and protocol translation, followed by **pra.context.route**, selection and realization, then **pra.engine.request**. The attributes separate source and selected tokens, already-visible and newly materialized records, prefix reuse status, native attached bytes, storage tier activity, and queue/prefill/decode timing. This decomposition answers whether a high TTFT arose in routing, WARM promotion, native attach, engine queueing, or ordinary model prefill.

Privacy and overhead are policy, not deployment accidents. The absent or disabled configuration

Telemetry family	Normalized observations
Context economy	Source, selected, newly materialized, and visible-reuse tokens
Conventional reuse	Prefix-cache status and physically confirmed cached tokens
Native Memory	Selected/attached resources, HOT/WARM resources, bytes, and attach time
Storage	HOT/WARM/COLD bytes, promotion, demotion, eviction, reload, and reconstruction
Serving	Request latency, TTFT, ITL, completion time, errors, and successful throughput

Table 40: PRA’s normalized metrics complement engine-native scheduler, cache, device, and transfer telemetry. Missing physical observations remain unknown.

Mode	Median $\mu\text{s}/\text{op}$	Ratio	Attributes evaluated
off	5.339	1.00×	0
metrics	17.223	3.23×	0
sampled otel	46.121	8.64×	70000
detailed otel	79.494	14.89×	70000

Table 41: Telemetry instrumentation microbenchmark. This is an overhead diagnostic, not model-serving qualification.

follows a genuine no-op path: no exporter, listener, background thread, identifier hash, or telemetry-only attribute construction. The default content policy exports no prompts, documents, tool payloads, credentials, or raw resource identifiers. Prometheus labels are restricted to bounded deployment dimensions; private request, session, user, task, and URI identities never become labels.

Table 41 isolates instrumentation cost from model serving. It times 10,000 span-plus-counter operations in seven repetitions on the development host. Ratios look large because the disabled baseline is only a few microseconds; they are not model-latency multipliers. The result confirms the key mechanism: the off and metrics-only paths evaluate zero span-attribute callables, whereas tracing evaluates them. All primary performance evidence in this paper remains telemetry-off.

The open artifact includes baseline Agent, Gateway, runtime, multi-engine, and engine-specific Grafana dashboards plus profile-gated Collector/Prometheus/ Grafana Compose deployment. Host-native MLX and embedded runtimes use the same collector through a documented hybrid-host arrangement.

10 Negative Results and Next Gates

The current evidence is intentionally incomplete. The local HF compilation probe uses an old CUDA GPU, while companion experiments span two Apple hosts, one newer CUDA laptop GPU, and an Intel GPU. Bounded async promotion, multi-session pressure, prefetch lead, and request-tail distributions are now measured; production continuous batching, a stable upstream native-page ABI, live LMCache, fused segmented attention, online cancellation under load, and sustained multi-tenant serving are not. The LRU fixture validates byte accounting but is not a production cache policy study. The inherited language-model cohort is too small for quality claims. The new capability and compact-result workflows and Paper 8 task/session integration are executed contract tests, not a new latency or answer-quality experiment. The primary interactive shell remains single-agent. The SDK now includes Paper 9’s experimental lineage, selective visibility, generic effect, resource-validity, and native-reuse contracts so external harnesses can wrap their own child loops; Paper 9 retains ownership of the subagent benchmark and qualification claims. Learned long-term memory remains scoped to Paper 10. Model-resident result routing currently requires an isolated model session; safe concurrent multi-tenant partitioning of model-resident result K/V remains an engine-level gate. The multi-axis HF bridge serializes high-level

generation on one wrapped model to isolate temporary adapter state. It is a correctness reference, not a continuous-batching implementation. The cache key now rejects physical reuse across changed source revisions, positions, materialization layouts, or scopes, and a concurrent fixture covers logical tenant isolation, pressure, and cancellation. It does not establish hard GPU partitioning or production multi-tenant throughput. Phase selection and portable E2/E3 engine integration remain open. The G01 parser recognizes explicit system, tool, and attachment boundaries; it does not claim semantic recovery from arbitrary opaque traffic.

Papers 3.2 and 3.3 own the document-RAG retrieval, routing, transfer, and quality evidence. Paper 4.5 imports their product boundary without reproducing those experiments: immutable contiguous-block reuse is qualified, while independent record composition and reduced materialization remain **CALIBRATION_PENDING**.

The Paper 4.5 gateway and agent-transport profiles are deterministic transport experiments. They do not by themselves measure physical prefix-cache hits, continuous batching, or engine queueing. Companion experiments now measure local physical reuse and controlled two-host SGLang affinity, but the latter is not wired through online generation. The reported gateway reuse fraction counts exact serialized messages, not tokenizer tokens. Engine profiles are conservative defaults and require runtime probing or explicit configuration before stronger capabilities may be advertised. Companion engine studies do not change this profile’s evidence tier and are intentionally kept outside Paper 4.5’s HF-centered metric tables.

The cross-model cohort contains only three one-token continuations per model. It is sufficient to expose exact disabled behavior, K/V topology, position, lifetime, and coarse semantic drift, but not to estimate broad language quality. The Llama row uses a public weight mirror because the pinned official repository was inaccessible. Gemma is partial rather than passed: its global-only adapter cannot make external K/V visible to unchanged local sliding layers. Sparse profiles are declared per family and are not assumed to preserve quality. The layer calibration reuses that diagnostic cohort. Its 102 rows vary profiles, not independent examples, so they identify gross transport drift and set conservative SDK defaults but cannot estimate downstream answer quality. Qwen last-24’s target-token gain is especially vulnerable to this small-cohort selection effect. It is consequently exposed only as **QUALITY_MAX_CANDIDATE** with pending calibration, not as **QUALITY_MAX**. Paper 3 owns retrospective placement evidence on MuSiQue and 2Wiki; a future calibration should add held-out QASPER, HotpotQA, typed-result, and long-generation workloads before any sparse profile becomes a default.

The next experiment should move to a compute-capability 8.0 or newer GPU with a pinned software stack. It should first reproduce eager parity, then measure `torch.compile` cold and warm paths. Only measured hot spots should move to Triton. Layout and hierarchy sweeps should cross contiguous versus indexed selection, whole chunks versus intervals, GQA head count, batch heterogeneity, and warm-cache locality. End-to-end runs should report TTFT, inter-token latency, throughput, peak HBM, off-device bytes, temporary bytes, transfers, and reload amplification together.

The next policy experiment should expand the trained-model retrieval smoke, hold selected-token budgets fixed, and compare answer quality, TTFT, inter-token latency, selection churn, and routing-layer disagreement over a powered cohort. The implemented phase/shared handoff closes the cache-correctness prerequisite: the routing probe sets `use_cache=false`, the seeded selection is consumed by one cache-producing prefill, and selected detail is never inserted into the ordinary local cache. A request guard binds selected keys to tenant and session identity and rejects cross-scope or duplicate attachment. Its remaining gate is workload-scale calibration and production multi-tenant throughput rather than mechanism availability.

Deeper engine integration remains outside Paper 4.5. Companion engine papers have established SGLang and MLX native mechanism baselines and controlled vLLM generation on Metal and CUDA. A shared matched benchmark now closes E0/E2 semantic parity for all three engines. The corrected vLLM-Metal geometry reaches 3,360/3,360 exact cross-family pairs, while expanded Qwen confirma-

tions preserve 2,086/2,086 per engine. SGLang and MLX still expose unfused decode overhead on reused paths. Those results remain environment-specific and do not turn the generic provider into an unconditional E2 claim. Native APC and offline concurrency now pass on pinned Metal and CUDA paths when cache identity includes native selection and geometry; online queued concurrency, LM-Cache, broader positions, masks, GQA, and heterogeneous batching must pass before portable speed comparisons. Controlled off-node WARM transfer and affinity now pass outside the SGLang generation path; engine-owned distributed placement remains open. If useful behavior requires semantic residency, retrieval-aware scheduling, or cross-request PRA block sharing, that work belongs to the deep engine paper.

11 Conclusion

PRA’s selected-memory fraction is an algorithmic fact, not a systems result. This paper turns that distinction into code and measurement. One runtime now connects the public model API, authenticated memory lifecycle, typed resources, lazy tool and skill disclosure, safe execution, compact result backing, selective replay, durable tasks and sessions, physical K/V planning, caching, profiling, and backend handoff. Typed results build full native indexes only within configurable token/byte ingestion budgets; larger records retain cheap recovery interfaces and promote selected regions lazily. Its policy lattice makes routing frequency and physical lifetime inspectable, while the gateway permits ordinary and PRA-aware agents and engines to coexist without sending native tensors over the logical wire. The agent no longer flattens selected records before remote execution: AUTO preserves typed context through G10, G11, and direct PRA endpoints, while an ordinary endpoint receives one deterministic text rendering. In the five-turn task/tool fixture, PRA-DELTA cuts wire bytes by 29.6% relative to TEXT and sends 8 rather than 24 resource bodies, without changing selected identity or task metadata. At 50 turns, including reconnect and resynchronization, the reduction grows to 74.6% versus TEXT and 80.7% versus PRA-FULL. A ten-example MLX-LM QASPER cohort then preserves exact model input, output, and gold-answer log-probability across TEXT, PRA-FULL, and PRA-DELTA in 10/10 cases. The protocol result is therefore behavioral as well as structural. Conventional sequential-prefix K/V and detached PRA resource/native state are now separate capability and lifecycle axes. The tested gateway preserves every eligible prior prefix in its five-turn fixture and sends typed message/resource deltas through a prepared session, while correctly leaving physical cache hits unknown. The HF semantic reference now passes full-prefix logit equivalence on the tested Qwen and Llama checkpoints, plus record-boundary deduplication and request-long decode-lifetime gates. Gemma validates native MQA mechanics but identifies a global/local topology boundary rather than a full semantic pass. Layer placement is now a versioned calibration result rather than an implicit “PRA layer” convention. Separate address, detail-K/V, routing, and consumer sets make persistent storage and request-time computation independently auditable; missing detail can never trigger a silent profile downgrade. The eager primitives are exact and inexpensive in isolation, but off-device transfer multiplies their cost on the measured host. Compilation remains a negative gate. Companion engine runs establish pinned native realizations but still leave portable E2/E3 integration unmeasured. The coding-agent gate sharpens the unit of reuse: arbitrary token windows are insufficient for multi-turn state, while causal action–observation bundles, source evidence, mutation state, and verification state require explicit identities and recency floors. Restoring one omitted source-view bundle causally recovers task 03’s efficient command and directionally improves task 04, where current-hypothesis state is additionally required; the typed `recent_progress_turns` floor now preserves that state in the frozen 90% regression. HF, MLX, SGLang-MLX, and vLLM-Metal now pass their declared consumer and lifecycle scopes. HF’s two-pass path eliminates K/V casts and is matched-consumer logit exact, but remains unfused and carries bounded score state. A bounded vLLM 0.28 CUDA scheduler hook additionally demonstrates genuine existing-page aliasing with zero K/V copy, host-to-device transfer, or selected- history re-

encoding on task-02 turn 4 and its lifecycle smoke. It is not yet a general product path: multiprocessing execution, hybrid K/V groups, partial pages, full-trajectory coverage, and runtime economics remain open. MLX and SGLang now consume the canonical source plus interval descriptors through one fused Metal kernel: their measured first-layer transient falls from roughly one selected K/V layer to 19,371 bytes and both are matched-consumer logit exact. End-to-end agent runs and broader batch/model geometry remain open. Under the RAG workload, frozen contiguous MLX selections are output-exact: retained K/V cuts total latency by 78.9% for Qwen3-4B at 2K, 86.9% at 4K, and 82.1% for Qwen3-8B, while cold native execution is 1.8–3.3% slower. Independent chunks remain an experimental composition mode because their outputs are not exact. The practical conclusion is conservative: cache locality, physical residency, and context lifecycle deserve attention before kernel novelty, and every future speed claim must match semantics while accounting for the whole request.

A Reimplementation Guide

The stable public entry points live in `src/prahf/runtime.py`. A reimplementation needs five small contracts. First, represent warm K/V as equal-shaped tensors $[B, H_{kv}, T, D]$. Second, express selections as stable URI, layer, and half-open token intervals. Third, merge overlaps before applying a hard global token budget. Fourth, pack selected tensors by consumption layer without expanding grouped-query heads. Fifth, emit stage times and physical bytes independently of routing-quality metrics.

The model adapter only needs `add_reference`, `generate`, and `inspect`. The thin serving request contains selected identities and materialized-token metadata, never semantic scores. Tool execution receives a separate authorization object after discovery. These boundaries allow another runtime to reproduce the interface without adopting the Python implementation.

Paper 8’s finalized geometry adds one further invariant. A frozen selection is an immutable set of source-local anchors. Expansion may change the interval width or choose the complete selected record, but it cannot cross record boundaries or change source identity. Overlap is merged before budgeting, and each consumer resolves the same logical interval to that layer’s own projected K/V. The query starts after the longest source extent, while separate records retain source-relative positions. Raw intervals are retained for diagnostics; the canonical union is the only geometry materialized. The named `paper3_default`, `paper7_selected_detail`, and `paper8_full_record_diagnostic` profiles resolve to ordinary runtime configuration. The public sequence is `route`, `freeze_native_selection`, `plan_native_materialization`, then `generate_with_native_plan`. Task operations follow the analogous control boundary: parse untrusted proposals, validate them against a copy of the task graph, and only then replay accepted events through the session service.

A.1 Agent transport implementation map

The semantic/wire split is implemented in `src/prahf/agent_transport.py`. `AgentTurnContext` keeps messages, context records, tool records, skill records, provider-native tool schemas, and task metadata independent of transport. `context_record_to_wire_resource` is the only logical projection; `render_text_messages` is the only text compatibility renderer. `CapabilityNegotiator` distinguishes a reachable ordinary server from an unavailable endpoint and caches the version-1 feature handshake. `NegotiatedRemoteBackend` resolves the mode, computes message and resource deltas, sends the OpenAI-compatible envelope, and clears its acknowledged inventory on refresh.

The call site is `src/prahf/agent.py`. `_turn_context` separates conversation records from detached records and projects active task state; `_generate_turn` delegates either to the negotiated backend or the canonical embedded text renderer. Thus selection and tool authorization remain agent responsibilities, while rendering and network serialization remain transport responsibilities.

The public wire dataclasses and protocol-version check are in `src/prahf/deployment.py`. Gateway capability composition and G10/G11 mediation are in `src/prahf/gateway.py`; acknowledged message/resource inventory, body-free unchanged operations, and resynchronization state are in `src/prahf/gateway_session.py`. The compatibility matrix is executable in `tests/test_agent_transport.py`; the five-turn byte fixture is `experiments/paper4_5_runtime/run_agent_transport_protocol.py`.

A.2 HF native-reference implementation map

The four-role resolver is in `src/prahf/layer_profiles.py:1--430`. Lines 27–77 define independent encoding and missing-detail policies; lines 81–178 normalize all, suffix, fractional, evenly spaced, and explicit profiles; lines 181–244 validate the L_A, L_{KV}, L_R, L_C invariants and serialize provenance. The stable public fields and registry precedence are applied in `src/prahf/config.py:19--420`. The versioned model entries live in `src/prahf/model_profiles/layer_profile_registry.json`.

Lines 328–510 of the HF `injection.py` module separate reference payloads. Every injected layer can produce compact identity metadata for cross-layer mapping, address layers retain their search representation, and only detail layers retain native K/V. The cache record exposes `has_detail_kv` in `src/prahf/memory.py:99--140`; materialization rejects an address-only chunk before attention in `src/prahf/core.py:232--270`. Selecting a chunk ID at L_R never licenses reuse of L_R ’s tensor at L_C : the identity is resolved to the destination layer’s independently projected K/V.

The complete geometry is concentrated in `src/prahf/native_geometry.py:59--374`. Lines 59–107 define four materialization modes and the three named research profiles. Lines 130–169 make raw and unique interval accounting, source lengths, and query offset part of the immutable plan. Lines 189–238 clamp expansion to one record; lines 310–374 normalize overlap before slicing each consumer layer’s own K/V.

The high-level path is in `src/prahf/model.py:932--1191`. `plan_native_materialization` separates routing anchors from selected chunk, asymmetric window, selected-record, or full-scope geometry. `native_next_token_logits` is the E0 diagnostic. `generate_with_native_plan` is the public fixed-plan path, and `debug_decode_with_native_plan` spells out prefill, local cache handoff, and one-token decode calls. All three use `src/prahf/hf/injection.py:494--529` to start the direct query after the longest active source extent.

At attention time, `src/prahf/hf/qwen.py:340--352` prepends selected native K/V to local K/V and records compact request-lifetime state before the family-native eager kernel. Llama and Gemma 3 inherit this shared path while overriding only family symbols and native scope. The trace owner is defined in `src/prahf/hf/adapters.py:107--129` and aggregated at `src/prahf/hf/injection.py:228--241`; no full K/V or attention tensor is retained for telemetry.

Portable streaming begins in `src/prahf/deployment.py:227--341`. The worker owns logical references, the cancellation criterion, and cleanup. Gateway mediation and SSE framing are separate at `src/prahf/gateway.py:131--231`, so a transport disconnect closes the iterator without exposing backing content. Tenant-scoped cache identity and local eviction are in `src/prahf/runtime.py:568--672`. Cache identity includes source revision, positional and materialization signatures, and scope; equality is the reuse predicate rather than a heuristic URI match. Engine capability fields separately expose interval materialization, request lifetime, residency, tenant isolation, phase selection, and scheduler hints. An independent implementation should fail a required unsupported field before reference registration.

A.3 Gateway session implementation map

The independent capability axes begin at `src/prahf/deployment.py:38`: the capability record carries engine type, PRA level, prefix-cache mode, session state, incremental-message support, re-

source deltas, affinity, and explicit handles. Typed request serialization starts at line 129, and the profile-parameterized OpenAI adapter at line 242. The versioned defaults and validation are in `src/prahf/engine_profiles.py:1--90`; adding an engine requires an enum member and one conservative registry row before any feature override.

Ephemeral state is isolated in `src/prahf/gateway_session.py:186--434`. `resolve_turn` compares explicit full or delta history, derives resource operations, and computes the affinity key. `commit` advances fingerprints only after a successful decode. It stores resource IDs, digests, and metadata, never content tensors. The gateway execution order is visible at `src/prahf/gateway.py:147--457`: resolve and invalidate, prepare once, transform G00–G11, execute, commit trace state, and close explicitly. HTTP capability, session-debug, and session-delete handling follows at lines 460–538. The same registry entry points are surfaced by `src/prahf/runtime.py:1194--1220`, preventing the HTTP process from becoming the sole owner of session semantics.

A.4 Execution and deployment code map

Line references identify the artifact version accompanying this paper. `src/prahf_torch/execution.py:61--107` defines the four policy axes and their validation; lines 358–425 define tensor-free identities and row-local plans; lines 428–504 own request epochs, churn, and trace summaries; and lines 507–632 separate selection reuse from opaque materialization lifetimes. The important invariant is the type boundary: a selection controller returns URI/chunk spans, while only an engine resolver may return tensors or block handles.

The canonical typed-result admission policy is implemented at three layers. `src/prahf/adaptive_context_runtime.py:82--236` defines global and per-record token/byte limits, inheritance, deferral, and serialization. `src/prahf/progressive_context.py:58--66` defines the four lifecycle states; lines 347–391 define the public audit and lazy-region records; and lines 602–773 perform idempotent size-gated full indexing and selected-region native encoding. The facade in `src/prahf/runtime.py:1119--1268` applies automatic preparation at ingestion and exposes preparation, lazy promotion, and lifecycle inspection. This order is important: policy is resolved before model-native construction, and a gate rejection preserves the backing identity and recovery interfaces.

`src/prahf/hf_execution.py:49--174` is the family-independent HF bridge. At the routing layer it turns the current hidden representation into a logical plan. For shared scope it remaps the source-layer hits through `map_chunk_identities_to_layers`; each destination therefore receives its own native grouped-query K/V. The public request lock and precedence entry point are in `src/prahf/model.py:710--895`. Qwen, Llama, and Gemma call the same bridge from their thin attention adapter.

For a distributed implementation, begin with `src/prahf/deployment.py:100--174`: reproduce that JSON request without adding tensor fields. At lines 175–182, expose capabilities, session setup, generation, streaming, and close as separate methods. Then reproduce deterministic mediation in `src/prahf/gateway.py:24--178`: negotiate first, transform according to G00–G11 second, and append component-local trace events last. The HTTP process starts at line 180; the CLI binding is `src/prahf/gateway_cli.py:35--59`. This order keeps an unsupported native request from being mistaken for successful text fallback. The agent bridge implementation is `src/prahf/agent_plugins.py:1--253`; its family-specific parsers terminate in the common logical request builder rather than engine calls. Frozen interval expansion and layer-native slicing are in `src/prahf/native_geometry.py:1--374`; the SDK facade delegates through `src/prahf/runtime.py` so users do not access adapter internals.

B Artifact Map

Artifact	Role
src/prahf/runtime.py	Unified facade, plans, materializer, cache, profiler, HF adapter, thin vLLM contract
src/prahf/runtime_providers.py	Provider registry, canonical runtime configuration, conservative capabilities, managed handles, readiness, and upstream launcher translation
src/prahf/onboarding.py, src/prahf/bundle.py	Structural inspection/adaptation/validation, profile packaging, portable bundles, and the optional Hub boundary
src/prahf/cli.py	Thin canonical pra command tree and deprecated pra-hf aliases
src/prahf_torch/execution.py	Engine-neutral policy axes, logical plans, precedence, lifecycle, and analytical routing count
src/prahf/hf_execution.py	Request-owned token/shared and token/per-layer HF execution bridge
src/prahf/deployment.py, src/prahf/gateway.py	JSON wire types, engine capabilities, E0/E1 adapters, G00–G11 mediation, and HTTP process
src/prahf/engine_profiles.py, src/prahf/model_profiles/engine_profile_registry.json	Versioned engine/cache/session defaults with conservative capability resolution
src/prahf/gateway_session.py	Tenant/session/model-isolated ephemeral engine handles, canonical and serialized prefix state, resource versions, history/resource deltas, affinity, and invalidation
experiments/paper4_5_runtime/run_gateway_session_profile.py	Five-turn prefix/session contract profile, two-cache architecture, traces, tables, and figures
experiments/paper4_5_runtime/run_agent_transport_scaling.py	Deterministic 20/50-turn TEXT, PRA_FULL, and PRA_DELTA wire scaling with resynchronization and body-transmission accounting
experiments/paper4_5_runtime/run_agent_transport_model_backed.py	Held-out model-backed G10 transport parity, task/tool visibility, gold-answer log-probability, TTFT, and execution-message checks
experiments/paper4_5_runtime/build_product_matrix_v2.py	Strict engine–model–profile–hardware product matrix with explicit evidence tier, per-metric status/provenance, qualification level, and missingness fields
experiments/paper4_5_runtime/build_engine_qualification.py	Selector-frozen E0/E2/E3 manifest, evidence-driven maturity registry, table, and cross-engine figure
experiments/paper4_5_runtime/run_storage_policy_trace.py	Weighted-versus-LRU retention trace with task/shared reuse, reconstruction, and estimated context-cost accounting
docs/papers/shared/results/paper4_5_runtime/prefix_preservation_results.csv	Canonical per-turn logical-prefix and transport measurements; physical hits remain unknown
src/prahf/agent_plugins.py	DeepSeek Harness and Pi typed-event bridges
src/prahf/native_geometry.py	Frozen record-bounded intervals and layer-native K/V plans
experiments/paper4_5_runtime/run_execution_policy_profile.py	Five-seed multi-axis mechanism profile and trade-off figure
experiments/paper4_5_runtime/run_cross_model_validation.py	Restartable Qwen/Llama/Gemma semantic gates, physical accounting, reuse and multi-tenant controls, and cross-model figure
experiments/paper4_5_runtime/run_layer_profile_calibration.py	Corrected cross-model contiguous/sparse profile sweep, storage accounting, registry selection, and quality–cost figure
src/prahf/layer_profiles.py	Four-role layer schema, normalized selection, registry precedence, lifecycle, and missing-detail policy
src/prahf/profile_benchmarks.py	Versioned model/workload profile registry, normalized metrics, missingness status, and resolution trace
src/prahf/model_profiles/pra_profile_benchmarks.json	Packaged semantic and physical profile registry

Artifact	Role
experiments/paper4_5_runtime/build_profile_matrix.py	Deterministic registry and manuscript-table generator
src/prahf/runtime_benchmark.py	Structured eager/compiled and hierarchy microbenchmark
src/prahf/agent_resources.py	Typed resource identities and discovery
src/prahf/agent_execution.py	Schema validation and independent host authorization
src/prahf/agent.py	Task-aware turn orchestration and persistent agent facade
src/prahf/agent_profiles.py	Versioned named profiles, precedence, credential redaction, runtime mapping, and shared agent launcher
src/prahf/agent_web/	Optional FastAPI/WebSocket multi-session presentation, reconnect replay, one-shot approval, and safe detached lifecycle
src/prahf/session_service.py	In-memory and atomic local logical session persistence
src/prahf/subagent_context.py	Durable agent lineage, selective cross-stream visibility, resource validity, and auditable reuse decisions
src/prahf/subagent_harness.py	Experimental harness-neutral spawn/stop/resume and payload/native-result reuse callbacks
src/prahf/task_context.py, src/prahf/task_scope.py	Versioned task DAG, provenance, scope admission, and working-set lifecycle
src/prahf/toolsets.py, src/prahf/tui.py	Reusable tools and the resumable coding-agent terminal
experiments/agents/	Frozen Terminal-Bench/SWE-bench manifests, pinned agent adapters, official-Harbor normalization, and paired analysis
experiments/paper4_5_agent/	Published-baseline cards, resumable campaign state, distributed result import, official SWE-bench normalization, and hard No-PRA reproduction gates
experiments/paper4_5_agent/benchmarks/swebench_verified_fixed50.json	Digest-protected external seed-0 cohort and source provenance
experiments/paper4_5_agent/runners/swebench_verified.py	Gated mini-swe-agent baseline/treatment chunk runner, host preflight, official-grader normalization, and agent-visible trajectory telemetry
experiments/paper4_5_agent/context_treatment.py	Matched truncation and agent-visible Selected Context proxy with per-request budget telemetry
docs/papers/shared/results/paper4_5_runtime_productization/coding_agents/	Official task locks, canonical trajectories, verifier records, normalized rows, and qualification summaries
experiments/paper4_5_agent/summarize_baseline.py	Wilson uncertainty, trajectory distributions, and explicit insufficient-cohort gate
docs/papers/shared/results/paper4_5_runtime_productization/rag/paper3_2_runtime_policy.json	Imported contiguous/independent/partial RAG runtime qualification boundary
src/prahf/capability_sdk.py	Callable and skill-folder ingestion, bounded palettes, and exact capability activation
src/prahf/typed_context.py	Type-aware compact views and retrieval addresses with explicit token/ratio compression budgets
src/prahf/large_record_index.py	Independent typed, BM25, and embedding indexes, bounded fusion, exact selectors, and lifecycle accounting
src/prahf/adaptive_context_runtime.py	Scoped backing, serializable token/byte and per-type ingestion policy, materialization, transport accounting, and cursors
src/prahf/progressive_context.py	Stable record views, native-index lifecycle audits, size-gated full backing, and lazy selected-region native encoding
src/prahf/external_memory.py	Resolver-neutral cold/warm/hot lifecycle and scoped caches
prahf-demo/prahf_runtime_productization.ipynb	Executed user workflow

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.
- [2] Serbulent Av. Local LLM coding benchmark: Controlled open-model agent evaluation. <https://github.com/serbulent-av/agent-coding/blob/main/benchmarks/local-llm-coding-benchmark.md>, 2026. Repository revision 8f894c2284b9f73a515024d7c1f32e4d0fb14a04; accessed 2026-09-05.
- [3] FlashML.org. Freetoken quickstart. <https://github.com/FlashML-org/FreeToken/blob/main/docs/quickstart.md>, 2026. Accessed 2026-08-27.
- [4] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *arXiv preprint arXiv:2311.04934*, 2023.
- [5] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environment generation and hybrid verifiers for scaling open-weights SWE agents. *arXiv preprint arXiv:2504.07164*, 2025.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: <https://arxiv.org/abs/2309.06180>.
- [7] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
- [8] Yixuan Tang and Yi Yang. Multihop-rag: Benchmarking retrieval-augmented generation for multi-hop queries, 2024.
- [9] Yubo Wang, Jiarong Liang, Yuxuan Zhang, Xuye Liu, Cong Wei, Yuyu Zhang, Ping Nie, and Wenhui Chen. Function-aware fill-in-the-middle as mid-training for coding agent foundation models. *arXiv preprint arXiv:2607.12463*, 2026.
- [10] Jiayi Yao, Hanchen Li, Yuhang Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.
- [11] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.